Cheatsheet · 37 chapters on one page

Machine Learning cheatsheet

Every formula, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

221 key ideas | 392 formulas | 143 tables | 202 tips and traps

STEP 1 Foundations · Start here

1.1 What machine learning is: learning from examples

what ML is, the learning loop, kinds of learning, the vocabulary, generalisation and your first scikit-learn models.

KEY IDEAS

- **Machine learning** Learn the rule from examples instead of writing it. Mitchell: performance P on task T improves with experience E.
- **The loop** Guess, measure the loss, adjust parameters against the slope (gradient descent), repeat. Learning rate sets the step size.
- **Kinds** Supervised (labels: regression for numbers, classification for categories), unsupervised (no labels), reinforcement (rewards), plus semi and self supervised.
- **Words** Features are inputs (X), label is the answer (y). Parameters are learned, hyperparameters are chosen.
- **Generalise** Judge a model on unseen test data. High training score with low test score means overfitting.
- **scikit-learn** model = Model(), model.fit(X, y), model.predict(X_new), model.score(X_test, y_test). X is 2D.

FORMULAS

A model Exam favourite

$$\hat{y} = f(x, \theta)$$

x is the input, θ (theta) are the parameters, ŷ (y hat) is the prediction.

Lesson: The maths of learning

Mean squared error (loss for numbers)

Exam favourite

$$MSE = (1/n) \Sigma (y_i - \hat{y}_i)^2$$

Average of the squared gaps between true answers y and predictions ŷ over n examples.

Lesson: The maths of learning

Accuracy (score for categories)

$$\text{accuracy} = \text{correct predictions} / \text{all predictions}$$

Between 0 and 1. Higher is better.

Lesson: The maths of learning

One gradient descent step Exam favourite

$$\theta_{new} = \theta - \eta \times (\text{slope of the loss at } \theta)$$

η (eta) is the learning rate. Move against the slope, because the slope points uphill.

Lesson: The maths of learning

Learning as a search

$$\theta^* = \text{the } \theta \text{ that makes the average loss on the training data smallest}$$

Training is finding θ*. Generalisation asks if θ* also works on new data.

Lesson: The maths of learning

TABLES

Table: kind · what the data looks like · what it learns · everyday example

Kind	What the data looks like	What it learns	Everyday example
Supervised	Examples with the right answer (a label)	A map from inputs to answers	Spam filter, house prices, face unlock
Unsupervised	Examples with no answers	Groups, structure, unusual points	Customer groups, odd card payments
Semi supervised	A few labelled examples and many unlabelled ones	Uses the unlabelled data to help the labelled task	Photo apps that ask you to name a face once, then find it everywhere
Self supervised	Raw data where the label is hidden inside the data itself	Predict a hidden part from the rest	Language models that guess the next word
Reinforcement	No fixed data. An agent acts and gets rewards	A policy: what to do in each situation	Game playing, robots, ad choice

Lesson: The kinds of machine learning

Table: word · plain meaning · in the house example

Word	Plain meaning	In the house example
Example (sample, row, instance)	One case in the data	One house
Feature (input, attribute, column)	A measured fact we give the model	size_m2, rooms, age_years
Label (target, output, y)	The answer we want to predict	price
Dataset	All the examples together	The four houses
X and y	The table of features and the list of labels	X has 4 rows and 3 columns, y has 4 numbers
Model	A rule with adjustable numbers	price = a × size + b × rooms + c
Parameters (weights)	The adjustable numbers the model learns	a, b and c
Hyperparameters	Settings we choose before learning	The learning rate, the number of rounds
Training (fitting)	Finding good parameters from the data	model.fit(X, y)
Prediction (inference)	Using the trained model on new input	model.predict(new_house)
Loss (cost, error)	A number that says how wrong the model is	Average squared price error

Lesson: The words of machine learning

Table: step · code · what happens

Step	Code	What happens
Choose	model = LinearRegression()	Make an empty model with its settings (hyperparameters)
Learn	model.fit(X, y)	The model finds its parameters from the examples
Use	model.predict(X_new)	The model gives answers for new rows
Check	model.score(X_test, y_test)	R^2 for regression, accuracy for classification

Lesson: Your first models with scikit-learn

Table: area · what ml does there

Area	What ML does there
Phones	Face unlock, voice assistants, the keyboard that guesses your next word, photo search
Shopping and streaming	Recommends products, songs and videos you are likely to like
Banks	Flags card payments that look like fraud, scores loan applications
Health	Reads X rays and eye scans, predicts which patients need care first
Maps and transport	Predicts traffic and arrival times, plans delivery routes
Farming and weather	Predicts rain and crop yields, spots plant disease from photos
Factories	Predicts when a machine will break so it is fixed in time
Language	Translation, chatbots, summaries, search

Lesson: Where machine learning is used, and a short history

Table: year · milestone

Year	Milestone
1943	McCulloch and Pitts describe an artificial neuron
1958	Frank Rosenblatt publishes the perceptron, a neuron that learns its weights
1959	Arthur Samuel's checkers program improves by playing itself. He uses the words "machine learning"
1967	Cover and Hart study the nearest neighbour rule
1984	Breiman and others publish CART decision trees
1986	Rumelhart, Hinton and Williams make backpropagation popular for training neural networks
1995	Cortes and Vapnik publish support vector machines
2001	Breiman publishes random forests
2012	A deep network (AlexNet) wins the ImageNet image contest by a large margin
2016	AlphaGo beats a world champion at Go, using deep and reinforcement learning
2017	The transformer model is published. It powers today's language models
2022	Chatbots built on large language models reach millions of people

Lesson: Where machine learning is used, and a short history

Table: · traditional programming · machine learning · statistics

	Traditional programming	Machine learning	Statistics	Deep learning
Who writes the rule	A person	The computer, from examples	A person picks a model, data fits it	The computer, with many layers
Input to the process	Rules and data	Data and answers	Data and a model	Lots of data and answers
Main goal	Exact, explainable steps	Good predictions on new data	Understand and test relationships	Predictions on images, sound, text
Data needed	None	Hundreds to millions of rows	Often small samples	Usually very large
Best when	Rules are clear	Rules are hard to write	You must explain cause and certainty	Data is raw: pixels, audio, words

Lesson: Machine learning compared with similar ideas

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1491	Average Salary Excluding the Minimum and Maximum Salary	Easy	Robust average	Add everything, then take away the smallest and the largest before you divide by n − 2. Dropping extreme values is how a trimmed mean protects a model from outliers.
643	Maximum Average Subarray I	Easy	Sliding window (a moving average)	Keep the sum of the last k numbers. Add the new number and take away the one that leaves the window. This is how a moving average feature is computed fast.
1502	Can Make Arithmetic Progression From Sequence	Easy	Sort, then check one constant step	After sorting, every gap must be the same. A sequence like this fits a straight line exactly, the simplest pattern a model can learn.
896	Monotonic Array	Easy	One pass with two flags	Keep two flags, "can still be going up" and "can still be going down". Any drop clears the first, any rise clears the second. Checking a trend like this is a common data sanity check.
1984	Minimum Difference Between Highest and Lowest of K Scores	Easy	Sort, then slide a window of size k	After sorting, the best k scores sit next to each other. Check every window of k neighbours and keep the smallest spread.
2148	Count Elements With Strictly Smaller and Greater Elements	Easy	Find min and max, then count	An element counts when it is not the minimum and not the maximum. Finding the range of a feature is the first step of scaling it.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER Machine learning is not magic and not always the right tool. If a simple fixed rule works (for example "a password must have 8 characters"), write the rule. Use ML when the rule is hard to write but examples are easy to get.

Lesson: What machine learning is

COMMON MISTAKE A learning rate that is too big makes the guess jump past the best value and the loss can grow. One that is too small makes learning very slow. Choosing it well is part of the job.

Lesson: The learning loop: guess, check, adjust, repeat

IN THE EXAM A quick test: "Is there a right answer for each example in my data?" Yes means supervised. No means unsupervised. "Does the system learn by acting and getting rewards?" means reinforcement.

Lesson: The kinds of machine learning

IN THE EXAM Parameters are learned by the model. Hyperparameters are set by you. The learning rate is a hyperparameter, the slope of a line is a parameter. Interviewers love this question.

Lesson: The words of machine learning

COMMON MISTAKE Never measure a model only on its training data. The score will look great and mean nothing. Always keep a separate test set, and look at it only at the end.

Lesson: Learning, not memorising: generalisation

COMMON MISTAKE X must be two dimensional: a list of rows, even when there is only one feature. Write [[1], [2], [3]], not [1, 2, 3]. Passing a flat list is the most common first error in scikit-learn.

Lesson: Your first models with scikit-learn

REMEMBER AI, ML and deep learning are nested: every deep learning model is a machine learning model, and machine learning is one part of artificial intelligence. The Artificial Intelligence subject draws these as nested circles.

Lesson: Machine learning compared with similar ideas

1.2 The machine learning project: from a question to a working model

framing, the project stages, fair data splits, baselines, error analysis, leakage, deployment and monitoring.

KEY IDEAS

- **Frame first** Goal, output type, data known at prediction time, metric, success level, baseline. Ask if ML is needed at all.
- **Project loop** Frame, get data, explore and clean, split, baseline, train and tune, evaluate, deploy, monitor (CRISP-DM plus monitoring).
- **Three sets** Train learns, validation chooses, test scores once. Stratify rare classes, split time by time, keep groups together. Fix random_state.
- **Baseline** DummyRegressor (mean) and DummyClassifier (most frequent) set the bar every model must beat.
- **Leakage** Never train on information unavailable at prediction time. Fit preprocessing on training data only.
- **Uncertainty** SE of an accuracy = √(p(1 − p)/n). Range about p ± 2 SE. Four times the data halves the SE.

FORMULAS

Split sizes

$$n_{train} = n - n_{val} - n_{test}, \text{ with } n_{test} = \text{round}(n \times \text{test share})$$

For n = 1,000 with 20% test and 20% validation: 600, 200, 200.

Lesson: The maths of splitting and testing

Uncertainty of a test accuracy Exam favourite

$$SE = \sqrt{(p(1 - p) / n)}$$

p is the measured accuracy, n the number of test examples. Smaller n means a shakier score.

Lesson: The maths of splitting and testing

Rough 95% range Exam favourite

$$p \pm 2 \times SE$$

With p = 0.9 and n = 100: SE = 0.03, so the true accuracy is likely between 0.84 and 0.96.

Lesson: The maths of splitting and testing

Test size for a target precision

$$n \approx 4\, p(1 - p) / m^2$$

m is the half width you want. For p = 0.9 and m = 0.02, n ≈ 900.

Lesson: The maths of splitting and testing

Mean absolute error Exam favourite

$$MAE = (1/n)\, \Sigma\, |y_i - \hat{y}_i|$$

The average size of the mistakes, in the same unit as y.

Lesson: The maths of splitting and testing

TABLES

Table: stage · main question · typical output

Stage	Main question	Typical output
1. Frame	What decision, what output, what metric?	A one page problem statement
2. Get data	Which data exists and may we use it?	Raw tables, files, logs
3. Explore and clean	What is in the data, what is broken?	Clean table, charts, notes
4. Split	How do we test fairly?	Train, validation and test sets
5. Baseline	What does a simple rule score?	A number to beat
6. Train and tune	Which model and settings work best?	A trained model
7. Evaluate	How good is it on unseen data?	Test score and error analysis
8. Deploy	How do users get predictions?	An API, an app feature, a nightly job
9. Monitor	Is it still good next month?	Dashboards, alerts, retraining

Lesson: The stages of an ML project

Table: check · why · pandas

Check	Why	pandas
Shape	How many examples and features	df.shape
First rows	See real values and types	df.head()
Summary statistics	Spot impossible values (negative minutes)	df.describe()
Missing values	Holes must be filled or removed	df.isna().sum()
Group averages	First ideas for features (rain is slower)	df.groupby("raining")["minutes"].mean()

Lesson: Step 2 and 3: get the data and look at it

Table: data type · how to split · why

Data type	How to split	Why
Normal rows (independent)	Shuffle, then cut	Every part is a fair sample
Imbalanced classes	Stratified split	Each part keeps the rare class share
Time series (sales, prices)	By time: past for train, latest for test	In real use we always predict the future
Many rows per person or patient	Group split: all rows of one person in one part	Otherwise the model meets the same person in train and test

Lesson: Step 4: train, validation and test sets

Table: kind of leak · example · fix

Kind of leak	Example	Fix
Future information	late_fee_paid used to predict loan repayment	Use only facts known at decision time
Target in disguise	Hospital stay length used to predict if a patient was admitted	Remove features caused by the label
Preprocessing on all data	Scaling with the mean of train and test together	Fit the scaler on train only, use a Pipeline
Duplicates across sets	The same customer in train and test	Remove duplicates, use a group split
Time order broken	Shuffling stock prices, so the model sees the future	Split by time

Lesson: The silent killer: data leakage

Table: way to deploy · how it works · good for

Way to deploy	How it works	Good for
Batch	Run the model on all rows every night and store the answers	Churn lists, daily forecasts, email campaigns
Online (real time) API	The app sends one request and gets an answer in milliseconds	Delivery times, fraud checks, search
On device	The model runs inside the phone or camera	Face unlock, keyboard suggestions, offline apps
Streaming	The model reads events as they arrive	Sensor alarms, live recommendations

Lesson: Step 8 and 9: deploy and monitor

Table: · quick notebook experiment · crisp-dm project · mlops lifecycle

	Quick notebook experiment	CRISP-DM project	MLOps lifecycle
Goal	Try an idea fast	Deliver a working model for a business question	Keep many models healthy in production
Steps	Load, train, print a score	Six phases from business to deployment	CRISP-DM plus automation, monitoring and retraining
Testing	Often a single split	Train, validation and test	Automated tests for data, code and model
Repeatable	Rarely	Mostly, with notes	Yes: versioned data, code and models
Best for	Learning and first ideas	One off analysis and first models	Products used every day

Lesson: Ways of working compared

Table: · batch · online

	Batch	Online
When answers are made	In advance, on a schedule	At the moment of the request
Speed needed	Minutes or hours are fine	Milliseconds
Cost and complexity	Lower	Higher (servers always on)
Fresh data	As old as the last run	Uses the latest input

Lesson: Ways of working compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
905	Sort Array By Parity	Easy	Partition into two groups	Walk once and put even numbers in one list and odd numbers in another, then join them. Splitting rows into groups by a rule is what a data split does.
2161	Partition Array According to Given Pivot	Medium	Stable three way partition	Make three lists: smaller than the pivot, equal, and larger, keeping the original order inside each. Then join them.
2270	Number of Ways to Split Array	Medium	Prefix sums	Keep the sum of the left part as you move the cut. The right sum is the total minus the left sum. Count cuts where left is at least right.
2780	Minimum Index of a Valid Split	Medium	Count the dominant value on both sides	Find the value that is more than half of the list. Move the cut from left to right and check that it is more than half on both sides. This is the idea behind a stratified split: both parts keep the same majority.
915	Partition Array into Disjoint Intervals	Medium	Running max on the left, running min on the right	Build the smallest value of every suffix first. Then walk from the left keeping the largest value so far. The first cut where left max is at most right min is the answer.
384	Shuffle an Array	Medium	Fisher Yates shuffle	Go from the last position down. Swap each position with a random position at or before it. Every order is equally likely, which is what a fair train and test split needs.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER Ask "do we even need ML?" first. If a simple rule or a lookup table is good enough, ship that. Google's Rules of ML put it as rule number one: do not be afraid to launch a product without machine learning.

Lesson: Step 1: frame the problem before any code

REMEMBER In real projects most time goes to data: finding it, cleaning it and understanding it. Training the model is often the shortest stage.

Lesson: The stages of an ML project

COMMON MISTAKE Look at the data, but do the deep exploration on the training part only, after splitting. If you study the test set too, you start choosing features that fit it, and your final score becomes too optimistic.

Lesson: Step 2 and 3: get the data and look at it

IN THE EXAM Fix random_state (the seed) so the split is the same every time you run the program. Otherwise your scores change from run to run and you cannot compare models.

Lesson: Step 4: train, validation and test sets

COMMON MISTAKE A model that cannot beat the baseline usually means a bug, leakage the other way, or features that carry no signal. Check the data before trying a bigger model.

Lesson: Step 5: always start with a baseline

REMEMBER Change one thing at a time and keep notes. If you change the features, the model and the settings together, you never learn which change helped.

Lesson: Step 6 and 7: train, look at errors, improve

IN THE EXAM If a score looks too good to be true, it usually is. Hunt for leakage before you celebrate.

Lesson: The silent killer: data leakage

1.3 Python tools for ML: NumPy, pandas and scikit-learn

NumPy arrays and broadcasting, pandas tables and groupby, and the scikit-learn estimator API.

KEY IDEAS

- **NumPy** Arrays of one dtype, vectorised maths, masks (a[a > 0]), shape, reshape, .T, np.random.default_rng(seed).
- **Axis** axis=0 goes down rows (one value per column), axis=1 across columns (one per row). The named axis disappears.
- **Broadcasting** Compare shapes from the right. Sizes must be equal or 1. (n, d) minus (d,) centres every column.
- **pandas** DataFrame and Series. df["c"], masks, loc (labels, end included), iloc (positions). groupby with agg or transform, merge, isna, fillna.
- **scikit-learn** fit learns (attributes end in _), transform changes, predict answers. fit_transform on train, transform on test, Pipelines.

FORMULAS

Dot product Exam favourite

$$w \cdot x = w_1x_1 + w_2x_2 + \ldots + w_dx_d$$

np.dot(w, x) or w @ x. The core of linear models and neurons.

Lesson: NumPy maths for models

Predictions for many rows Exam favourite

$$\hat{y} = X\,w$$

X is n × d, w is d long, ŷ is n long. In NumPy: X @ w.

Lesson: NumPy maths for models

Shape rule for matrix product

$$(n \times d)\ @\ (d \times k) = (n \times k)$$

The inner sizes must match.

Lesson: NumPy maths for models

Mean along an axis Exam favourite

$$\text{mean}_j = (1/n)\ \Sigma_i\ X_{ij}$$

axis=0 gives one mean per column j.

Lesson: The maths behind the tools

Standard deviation Exam favourite

$$\text{std} = \sqrt{(\ (1/n)\ \Sigma\ (x_i - \text{mean})^2\ )}$$

NumPy uses 1/n by default (ddof=0). pandas uses 1/(n − 1) (ddof=1).

Lesson: The maths behind the tools

Standard scaling Exam favourite

$$z = (x - \text{mean}) / \text{std}$$

What StandardScaler does to each column.

Lesson: The maths behind the tools

Memory of an array

$$\text{bytes} = \text{number of values} \times \text{bytes per value}$$

A 1,000 × 1,000 float64 array uses 8,000,000 bytes, about 8 MB.

Lesson: The maths behind the tools

Broadcast result size

$$\text{size}_k = \max(a_k, b_k) \text{ when } a_k = b_k \text{ or one of them is } 1$$

Checked from the right end of the shapes.

Lesson: The maths behind the tools

TABLES

Table: library · main object · use it for

Library	Main object	Use it for
NumPy	ndarray (an n dimensional array)	Fast maths on vectors and matrices, random numbers
pandas	DataFrame (a table) and Series (one column)	Loading, cleaning, joining and summarising tables
scikit-learn	Estimator (a model) and transformer	Preprocessing, classic ML models, metrics, splits
matplotlib, seaborn	Figure and axes	Charts for exploring data and showing results
PyTorch, TensorFlow	Tensor	Neural networks on GPUs (Deep Learning section)

Lesson: The ML toolbox at a glance

Table: idea · meaning · example

Idea	Meaning	Example
shape	Size along each axis	(4, 3) means 4 rows and 3 columns
axis=0	Go down the rows, one answer per column	scores.mean(axis=0): average of each test
axis=1	Go across the columns, one answer per row	scores.mean(axis=1): average of each student
reshape	Same numbers, new shape	a.reshape(2, 6), a.reshape(-1, 1) for one column
.T	Swap rows and columns	(4, 3) becomes (3, 4)
Broadcasting	Stretch size 1 axes to match	(4, 3) minus (3,) subtracts a row from every row

Lesson: Shapes, axes and broadcasting

Table: task · pandas code

Task	pandas code
Read a CSV file	pd.read_csv("orders.csv")
Size and column types	df.shape, df.dtypes, df.info()
Summary numbers	df.describe()
Select columns	df["a"], df[["a", "b"]]
Filter rows	df[df["age"] > 25], df.query("age > 25")
New column	df["bmi"] = df["kg"] / df["m"] ** 2
Sort	df.sort_values("score", ascending=False)
Count values	df["city"].value_counts()

Lesson: pandas DataFrames

Table: task · pandas code · sql idea

Task	pandas code	SQL idea
Summary per group	df.groupby("city")["temp"].mean()	GROUP BY
Many summaries	df.groupby("city").agg(avg=("temp", "mean"))	GROUP BY with several aggregates
Group value on every row	df.groupby("city")["temp"].transform("mean")	Window function
Join two tables	a.merge(b, on="id", how="left")	LEFT JOIN
Find missing values	df.isna().sum()	IS NULL
Fill missing values	df["x"].fillna(df["x"].median())	COALESCE
Drop missing rows	df.dropna()	WHERE x IS NOT NULL

Lesson: groupby, merge and missing values

Table: method · who has it · what it does

Method	Who has it	What it does
fit(X, y)	Every estimator	Learn from training data. Learned values end with _
transform(X)	Transformers (scalers, encoders)	Change data using what fit learned
fit_transform(X)	Transformers	fit, then transform the same data (training data only)
predict(X)	Models	Give predictions for new rows
predict_proba(X)	Most classifiers	Give the probability of each class
score(X, y)	Models	Accuracy for classifiers, R^2 for regressors
get_params(), set_params()	Every estimator	Read or change hyperparameters

Lesson: The scikit-learn API: estimators and transformers

Table: · python list · numpy array · pandas dataframe

	Python list	NumPy array	pandas DataFrame
Holds	Anything, mixed types	Numbers of one type	Columns of different types
Speed for maths	Slow (Python loop)	Very fast (compiled)	Fast (NumPy inside)
Names for columns	No	No (just positions)	Yes
Missing values	Use None by hand	np.nan for floats	Built in: isna, fillna, dropna
Best for	Small, general data	Maths, models, images	Real world tables, cleaning, joining

Lesson: Lists, arrays and DataFrames compared

Table: · scikit-learn · pytorch or tensorflow

	scikit-learn	PyTorch or TensorFlow
Models	Classic ML: linear, trees, SVM, clustering	Neural networks of any shape
Hardware	CPU	CPU and GPU
Code needed	A few lines per model	You write the network and often the training loop
Best data	Tables up to millions of rows	Images, sound, text, very large data

Lesson: Lists, arrays and DataFrames compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
2877	Create a DataFrame from List	Easy	pandas: build a DataFrame	Pass the list of rows and the column names student_id and age to pd.DataFrame.
2878	Get the Size of a DataFrame	Easy	pandas: shape	df.shape is (rows, columns). Return it as a list.
2879	Display the First Three Rows	Easy	pandas: head	head(3) returns the first three rows.
2880	Select Data	Easy	pandas: filter rows, pick columns	Keep rows where student_id is 101, then keep only the name and age columns.
2881	Create a New Column	Easy	pandas: a computed column	A new column is made by assigning to df["bonus"]. The maths runs on the whole column at once.
2884	Modify Columns	Easy	pandas: change a column in place	Multiply the salary column by 2 and store it back.
2888	Reshape Data: Concatenate	Easy	pandas: stack tables	pd.concat puts the second table under the first.
2889	Reshape Data: Pivot	Easy	pandas: pivot long data into a wide table	pivot(index="month", columns="city", values="temperature") makes one row per month and one column per city.
2891	Method Chaining	Easy	pandas: chain filter, sort and select	Filter weight > 100, sort by weight from heavy to light, keep the name column. Writing it as one chain reads like a sentence.
1572	Matrix Diagonal Sum	Easy	2D indexing (NumPy style rows and columns)	Add mat[i][i] and mat[i][n − 1 − i] for every row. When n is odd, the centre was added twice, so take it away once.

Lesson: LeetCode practice

TIPS AND TRAPS

COMMON MISTAKE A NumPy array holds one type. np.array([1, 2.5]) turns everything into floats, and np.array([1, "a"]) turns everything into text. Check .dtype when results look strange.

Lesson: NumPy arrays

IN THE EXAM axis=0 "collapses" the rows. A tip to remember: the axis you name is the one that disappears from the shape. (4, 3).mean(axis=0) has shape (3,).

Lesson: Shapes, axes and broadcasting

REMEMBER Use np.random.default_rng(seed) for random numbers. The seed makes your results repeatable, which matters when you compare models.

Lesson: NumPy maths for models

IN THE EXAM loc uses labels and includes the end: df.loc[0:2] gives 3 rows. iloc uses positions and stops before the end: df.iloc[0:2] gives 2 rows.

Lesson: pandas DataFrames

COMMON MISTAKE transform keeps one value per original row, so you can put it back into the table as a feature. agg gives one row per group. Mixing them up is a common mistake.

Lesson: groupby, merge and missing values

IN THE EXAM Call fit_transform on the training set and only transform on the test set. Calling fit on the test set too is a form of data leakage.

Lesson: The scikit-learn API: estimators and transformers

2.1 Linear algebra for ML: vectors, distances and matrices

vectors and distances, matrices and products, the linear model, inverses, rank, projections and eigenvectors.

KEY IDEAS

- **Vectors** An example is a feature vector. Dot product $a \cdot b = \Sigma\, a_i b_i = \|a\|\,\|b\| \cos\theta$. 0 means orthogonal.
- **Distances** Euclidean $\sqrt{\Sigma (a_i - b_i)^2}$, Manhattan $\Sigma |a_i - b_i|$, cosine $a \cdot b / (\|a\|\,\|b\|)$. Scale features first.
- **Matrices** Matrix times vector: one dot product per row. $(n \times k)(k \times m) = n \times m$. $AB \neq BA$. $(AB)^T = B^T A^T$.
- **Linear model** $\hat{y} = X w + b$. Each weight is the change in prediction per unit of its feature.
- **Inverse** $\det [[a,b],[c,d]] = ad - bc$. Inverse exists only if $\det \neq 0$. Use np.linalg.solve. Dependent columns mean multicollinearity.
- **Eigen** $A v = \lambda v$. Sum of eigenvalues = trace, product = determinant. Base of PCA.

FORMULAS

Euclidean distance (L2) — Exam favourite

$$d(a, b) = \sqrt{\Sigma\, (a_i - b_i)^2}$$

The straight line distance. Used by k nearest neighbours and k means.

Lesson: Length, distance and similarity

Manhattan distance (L1)

$$d(a, b) = \Sigma\, |a_i - b_i|$$

The city block distance. Less hurt by one huge gap.

Lesson: Length, distance and similarity

Length (norm) of a vector — Exam favourite

$$\|v\|_2 = \sqrt{v_1^2 + \ldots + v_d^2},\ \|v\|_1 = \Sigma\, |v_i|$$

The distance from the origin. Ridge regression punishes L2, lasso punishes L1.

Lesson: Length, distance and similarity

Cosine similarity — Exam favourite

$$\cos\theta = a \cdot b / (\|a\|\,\|b\|)$$

1 for the same direction, 0 for a right angle, −1 for opposite. Used for text and embeddings.

Lesson: Length, distance and similarity

Linear model for all examples — Exam favourite

$$\hat{y} = X w + b$$

X is n × d, w has d weights, b is one number added to every prediction.

Lesson: A linear model is a matrix product

One prediction

$$\hat{y}_i = w \cdot x_i + b = w_1 x_{i1} + \ldots + w_d x_{id} + b$$

Each weight says how much the prediction changes when its feature grows by 1.

Lesson: A linear model is a matrix product

Bias trick

add a column of 1s to X, then $\hat{y} = X w$

b becomes just another weight, which makes the maths shorter.

Lesson: A linear model is a matrix product

2 × 2 determinant — Exam favourite

$$\det [[a, b], [c, d]] = a\, d - b\, c$$

How much the matrix scales areas. 0 means it squashes the plane to a line.

Lesson: Systems of equations, inverse, determinant and rank

2 × 2 inverse

$$[[a, b], [c, d]]^{-1} = (1 / (a\, d - b\, c))\, [[d, -b], [-c, a]]$$

Swap a and d, change the signs of b and c, divide by the determinant.

Lesson: Systems of equations, inverse, determinant and rank

Rank

rank(A) = number of linearly independent columns

Full rank means no column is a mix of the others.

Lesson: Systems of equations, inverse, determinant and rank

Projection

Exam favourite

$$\text{proj}_d(x) = ((x \cdot d) / (d \cdot d))\, d$$

The closest point to x on the line through d.

Lesson: Projections and eigenvectors: a first look

Eigen equation

Exam favourite

$$A\, v = \lambda\, v$$

v keeps its direction, its length is multiplied by λ.

Lesson: Projections and eigenvectors: a first look

Eigenvalues of a 2 × 2 matrix

$$\lambda^2 - (a + d)\, \lambda + (a\, d - b\, c) = 0$$

Sum of eigenvalues = a + d (the trace), product = the determinant.

Lesson: Projections and eigenvectors: a first look

Dot product and angle

Exam favourite

$$a \cdot b = \|a\|\, \|b\| \cos \theta$$

Dot product 0 means the vectors are at a right angle (orthogonal).

Lesson: The maths, all in one place

Shape of a product

$$(n \times k)(k \times m) = (n \times m)$$

The inner k must match. Each entry needs k multiplies.

Lesson: The maths, all in one place

Transpose of a product

$$(A\, B)^T = B^T\, A^T$$

The order flips.

Lesson: The maths, all in one place

Squared distance expanded

Exam favourite

$$\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2\, a \cdot b$$

Lets libraries compute all pairwise distances with one matrix product.

Lesson: The maths, all in one place

Cauchy Schwarz

$$|a \cdot b| \le \|a\|\, \|b\|$$

Why cosine similarity is always between −1 and 1.

Lesson: The maths, all in one place

TABLES

Table: name · what it is · why ml cares

Name	What it is	Why ML cares
Square matrix	Same number of rows and columns	Covariance matrices, systems of equations
Identity I	1 on the diagonal, 0 elsewhere	I v = v, the "do nothing" matrix
Transpose A^T	Rows become columns	$X^T X$ appears in linear regression
Symmetric	$A = A^T$	Covariance matrices are symmetric, which makes PCA work
Diagonal	Non zero numbers only on the diagonal	Scaling each feature separately
Inverse A^{-1}	$A\, A^{-1} = I$, exists only if $\det(A) \neq 0$	Solving equations, the normal equation

Lesson: Matrices: tables of numbers that transform vectors

Table: · euclidean (l2) · manhattan (l1) · cosine

	Euclidean (L2)	Manhattan (L1)	Cosine	Chebyshev (L∞)
Measures	Straight line	Sum of gaps along axes	Angle between directions	Largest single gap
Cares about length	Yes	Yes	No	Yes
Effect of one huge gap	Strong (squared)	Milder	Depends on direction	Only that gap counts
Good for	Dense numeric features of similar scale	Grids, high dimensions, outliers	Text, word counts, embeddings	King moves in chess, tolerances
scikit-learn name	euclidean	manhattan	cosine	chebyshev

Lesson: Which distance to use

Table: · plain loops · numpy

	Plain loops	NumPy
Readable for learning	Yes, every step visible	One line hides the steps
Speed	Slow	Very fast (BLAS libraries)
Numerical care	You handle it	solve, lstsq and eigh are carefully tuned
Use it when	Learning how it works	Real work

Lesson: Which distance to use

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1266	Minimum Time Visiting All Points	Easy	Chebyshev distance	Moving diagonally covers one step on both axes at once, so the time between two points is the larger of the two gaps, max(|dx|, |dy|). Add it for each pair of neighbours.
1779	Find Nearest Point That Has the Same X or Y Coordinate	Easy	Manhattan distance with a filter	Skip points that share neither x nor y. For the rest, compute the Manhattan distance and keep the smallest, earliest index on ties.
566	Reshape the Matrix	Easy	Reshape by flattening (NumPy reshape by hand)	If r × c does not equal the number of values, return the original. Otherwise read the values row by row and fill the new shape row by row.
1828	Queries on Number of Points Inside a Circle	Medium	Euclidean distance without the square root	A point is inside when $dx^2 + dy^2 \le r^2$. Comparing squared distances avoids the square root and gives the same answer.
1037	Valid Boomerang	Easy	Determinant (cross product) for collinearity	Three points lie on one line exactly when the 2 × 2 determinant of the two difference vectors is 0. If it is not 0, they form a boomerang.
1886	Determine Whether Matrix Can Be Obtained By Rotation	Easy	Rotation as transpose plus reverse	A quarter turn is the transpose with each row reversed. Try up to four turns and compare with the target each time.

Lesson: LeetCode practice

TIPS AND TRAPS

COMMON MISTAKE The order of components matters. [age, height] and [height, age] are different vectors. Every example must use the same feature order.

Lesson: Vectors: every example is a list of numbers

IN THE EXAM Distances only make sense when features are on similar scales. Height in centimetres (around 170) would swamp age in years (around 25). Scale features first (Foundations 3.2).

Lesson: Length, distance and similarity

IN THE EXAM Matrix multiplication is not commutative: A B is usually not B A, and B A may not even exist when the shapes do not match.

Lesson: Matrices: tables of numbers that transform vectors

COMMON MISTAKE Perfectly correlated features (one column is a multiple of another) make X^TX impossible to invert. This is called multicollinearity. Drop one of the features or use ridge regression.

Lesson: Systems of equations, inverse, determinant and rank

2.2 Calculus for ML: derivatives, gradients and gradient descent

derivatives, the chain rule, gradients, gradient descent, learning rates, SGD and mini-batches, and convexity.

KEY IDEAS

- **Derivative** Slope at a point, the limit of (f(x+h) − f(x))/h. Power rule $n\,x^{n-1}$. e^x to e^x, ln x to 1/x, σ' = σ(1 − σ).
- **Chain rule** d/dx f(g(x)) = f'(g(x)) g'(x). Slopes along a chain multiply. Base of backpropagation.
- **Gradient** Vector of partial derivatives, points uphill. Update $w \leftarrow w - \eta \nabla L$.
- **Learning rate** Too small is slow, too big diverges. On a x^2, need $\eta < 1/a$. Feature scaling helps.
- **Variants** Batch (all rows), SGD (1 row), mini-batch (32 to 512). Epoch = one pass. Steps per epoch = n / batch size.
- **Convexity** Convex losses (linear and logistic regression) have one minimum. Networks have local minima and saddles.

FORMULAS

Gradient Exam favourite

$$\nabla f(w) = [\partial f/\partial w_1, \partial f/\partial w_2, \ldots, \partial f/\partial w_d]$$

One partial derivative per parameter.

Lesson: Partial derivatives and the gradient

Gradient descent update Exam favourite

$$w \leftarrow w - \eta \nabla L(w)$$

η is the learning rate. Move against the gradient.

Lesson: Partial derivatives and the gradient

MSE gradient for a line Exam favourite

$$\partial L/\partial w = (2/n) \Sigma (\hat{y}_i - y_i) x_i,\ \partial L/\partial b = (2/n) \Sigma (\hat{y}_i - y_i)$$

Used to fit $\hat{y} = w x + b$.

Lesson: Partial derivatives and the gradient

Convex test (one variable) Exam favourite

$$f''(x) \geq 0 \text{ everywhere}$$

The second derivative (curvature) is never negative.

Lesson: Valleys, convexity and local minima

At a minimum

$$f'(x) = 0 \text{ and } f''(x) > 0$$

Flat and curving up.

Lesson: Valleys, convexity and local minima

Second order step (Newton)

$$x \leftarrow x - f'(x) / f''(x)$$

Uses curvature to pick the step size. Fast but costly with many parameters.

Lesson: Valleys, convexity and local minima

Derivative

Exam favourite

$$f'(x) = \lim_{h \to 0} (f(x + h) - f(x)) / h$$

The slope at a single point.

Lesson: The maths, all in one place

Chain rule

Exam favourite

$$d/dx\ f(g(x)) = f'(g(x))\ g'(x)$$

Slopes along a chain multiply.

Lesson: The maths, all in one place

Gradient descent

Exam favourite

$$w_{t+1} = w_t - \eta\ \nabla L(w_t)$$

One step for every parameter at once.

Lesson: The maths, all in one place

Steps on f(x) = a x²

$$x_{t+1} = (1 - 2\ a\ \eta)\ x_t$$

Converges when $|1 - 2a\eta| < 1$, so $\eta < 1/a$.

Lesson: The maths, all in one place

Central difference

$$f'(x) \approx (f(x + h) - f(x - h)) / 2h$$

Error shrinks like h^2. Used for gradient checks.

Lesson: The maths, all in one place

TABLES

Table: rule · formula · example

Rule	Formula	Example
Constant	d/dx c = 0	d/dx 7 = 0
Power	d/dx x^n = n x^{n-1}	d/dx x^3 = 3x^2
Constant multiple	d/dx c f(x) = c f'(x)	d/dx 5x^2 = 10x
Sum	(f + g)' = f' + g'	d/dx (x^2 + 4x) = 2x + 4
Product	(f g)' = f' g + f g'	d/dx x^2 · x = 2x · x + x^2 = 3x^2
Exponential	d/dx e^x = e^x	Its own slope
Logarithm	d/dx ln x = 1/x	Used in log loss
Sigmoid	d/dx σ(x) = σ(x)(1 − σ(x))	Largest at x = 0, value 0.25
ReLU	d/dx max(0, x) = 1 if x > 0, else 0	Used in neural networks

Lesson: The derivative rules you need

Table: learning rate · what happens

Learning rate	What happens
Too small	Safe but very slow. Training takes forever
About right	Loss drops quickly and settles
A little too big	Bounces from side to side, still settles
Far too big	Each step overshoots more and the loss explodes (diverges)

Lesson: Gradient descent: walking downhill

Table: · batch gd · stochastic gd (sgd) · mini-batch gd

	Batch GD	Stochastic GD (SGD)	Mini-batch GD
Rows per step	All n	1	A small group, often 32 to 512
Steps per pass over the data (epoch)	1	n	n / batch size
Path to the minimum	Smooth	Very noisy	A little noisy
Memory	Needs all data at once	Tiny	Small
Used for	Small datasets	Online learning, streams	Almost all deep learning

Lesson: Batch, stochastic and mini-batch gradient descent

Table: · closed form (solve equations) · gradient descent · newton method

	Closed form (solve equations)	Gradient descent	Newton method	Search without gradients
Needs	A formula for the answer	The gradient	Gradient and curvature (Hessian)	Only the loss value
Speed per step	One big computation	Cheap	Expensive (d × d matrix)	Cheap but many tries
Steps needed	One	Many	Few	Very many
Works for	Linear regression	Almost every model	Small, smooth problems	Hyperparameters, black boxes
Example	Normal equation	Training a neural network	Logistic regression solvers (lbfgs is a cheaper cousin)	Grid or random search

Lesson: Optimisers compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
852	Peak Index in a Mountain Array	Medium	Follow the slope (binary search)	Look at the middle. If the next value is bigger, the slope goes up, so the peak is to the right. Otherwise it is here or to the left. This is gradient ascent on a list.
441	Arranging Coins	Easy	Binary search on a growing function	k full rows need k(k + 1)/2 coins, which grows with k. Binary search for the largest k that fits in n.
633	Sum of Square Numbers	Medium	Two pointers on a monotone sum	Start with a = 0 and b = the integer square root of c. If $a^2 + b^2$ is too small, raise a. If too big, lower b.
1283	Find the Smallest Divisor Given a Threshold	Medium	Binary search on the answer	A bigger divisor makes the sum of rounded up quotients smaller. Binary search for the smallest divisor whose sum is at most the threshold.
1011	Capacity To Ship Packages Within D Days	Medium	Binary search on the answer	The capacity is at least the heaviest package and at most the total. For a guess, count the days needed by filling each day greedily. Fewer days with more capacity: binary search.
2529	Maximum Count of Positive Integer and Negative Integer	Easy	Binary search for where the sign changes	The list is sorted, so find the first index that is at least 0 and the first index that is above 0. The counts follow from those two positions.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER — The central difference (f(x + h) − f(x − h)) / 2h is much more accurate than the one sided version. Use it to check hand written gradients ("gradient checking").

Lesson: The derivative rules you need

IN THE EXAM An epoch is one full pass over the training data. With 1,000 rows and a batch size of 100, one epoch has 10 update steps.

Lesson: Batch, stochastic and mini-batch gradient descent

COMMON MISTAKE Gradient descent finds a point where the gradient is near 0. For a convex loss this is the best answer. For a non convex loss it may be a local minimum or a saddle point.

Lesson: Valleys, convexity and local minima

2.3 Probability and statistics for ML

probability rules, Bayes rule, distributions, mean and variance, the central limit theorem, maximum likelihood, cross entropy and hypothesis tests.

KEY IDEAS

- **Rules** Complement 1 − P(A). Or: P(A) + P(B) − P(A and B). And: P(A) P(B | A). Conditional P(A | B) = P(A and B) / P(B).
- **Bayes rule** P(A | B) = P(B | A) P(A) / P(B), with P(B) from total probability. Rare events: false alarms dominate.
- **Distributions** Bernoulli p, binomial np and np(1 − p), Poisson λ and λ, normal μ and σ^2. 68, 95, 99.7 rule.
- **Describe** Mean, median, Var = $E[X^2] - E[X]^2$, sample std uses n − 1. Correlation in [−1, 1], not causation.
- **CLT** Averages become normal. Standard error σ / √n. 95% interval: estimate ± 1.96 SE.
- **Likelihood and losses** MLE maximises Σ log P(x | θ). Normal noise gives MSE, Bernoulli gives log loss. Gaussian prior gives L2. Cross entropy −Σ p log q.

FORMULAS

Bayes rule Exam favourite

$$P(A \mid B) = P(B \mid A) \times P(A) / P(B)$$

Posterior = likelihood × prior / evidence.

Lesson: Bayes rule: updating a belief with new evidence

Total probability Exam favourite

$$P(B) = P(B \mid A)\,P(A) + P(B \mid \text{not } A)\,P(\text{not } A)$$

Adds up every way B can happen.

Lesson: Bayes rule: updating a belief with new evidence

Binomial Exam favourite

$$P(X = k) = C(n, k)\, p^k\, (1 - p)^{n-k}$$

C(n, k) = n! / (k! (n − k)!) ways to pick which tries are yes.

Lesson: The distributions you will meet everywhere

Normal density Exam favourite

$$f(x) = (1 / (\sigma\sqrt{(2\pi)}))\, e^{-(x - \mu)^2 / (2\sigma^2)}$$

The bell curve.

Lesson: The distributions you will meet everywhere

Poisson

$$P(X = k) = \lambda^k\, e^{-\lambda} / k!$$

Mean and variance are both λ.

Lesson: The distributions you will meet everywhere

68, 95, 99.7 rule

$$P(|X - \mu| < k\sigma) \approx 0.68, 0.95, 0.997 \text{ for } k = 1, 2, 3$$

The program above measures 0.684, 0.954, 0.997.

Lesson: The distributions you will meet everywhere

Mean (expected value) Exam favourite

$$E[X] = \Sigma\, x\, P(x), \text{ sample mean } \bar{x} = (1/n)\, \Sigma\, x_i$$

The centre.

Lesson: Mean, variance and correlation

Variance Exam favourite

$$\text{Var}(X) = E[(X - \mu)^2] = E[X^2] - (E[X])^2$$

Average squared distance from the mean.

Lesson: Mean, variance and correlation

Correlation

Exam favourite

$$r = \mathrm{Cov}(X, Y) / (\sigma_X \sigma_Y)$$

Between −1 and 1. 0 means no straight line relation.

Lesson: Mean, variance and correlation

Linearity of expectation

$$E[aX + bY] = a\,E[X] + b\,E[Y]$$

Always true, even when X and Y depend on each other.

Lesson: Mean, variance and correlation

Variance of a sum

$$\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\,\mathrm{Cov}(X, Y)$$

The covariance term vanishes when X and Y are independent.

Lesson: Mean, variance and correlation

Standard error of the mean

Exam favourite

$$SE = \sigma / \sqrt{n}$$

To halve the error bar, you need four times as much data.

Lesson: The law of large numbers and the central limit theorem

Likelihood

Exam favourite

$$L(\theta) = \Pi_i\, P(x_i \mid \theta)$$

Product over independent data points.

Lesson: Maximum likelihood: where loss functions come from

Log likelihood

Exam favourite

$$\log L(\theta) = \Sigma_i \log P(x_i \mid \theta)$$

Sums are easier than products and do not underflow.

Lesson: Maximum likelihood: where loss functions come from

Coin MLE

Exam favourite

$$\hat{p} = \text{heads} / n$$

Set the derivative of h log p + t log(1 − p) to 0.

Lesson: Maximum likelihood: where loss functions come from

Normal noise gives MSE

$$-\log L = (1 / (2\sigma^2))\, \Sigma\, (y_i - \hat{y}_i)^2 + \text{constant}$$

Maximising likelihood is minimising squared error.

Lesson: Maximum likelihood: where loss functions come from

Bernoulli gives log loss

$$-\log L = -\Sigma\, [y \log p + (1 - y) \log(1 - p)]$$

The loss of logistic regression.

Lesson: Maximum likelihood: where loss functions come from

Entropy

Exam favourite

$$H(p) = -\Sigma\, p_i \log_2 p_i$$

In bits. A fair coin has 1 bit, a sure thing has 0.

Lesson: Entropy, cross entropy and KL divergence

Cross entropy

Exam favourite

$$H(p, q) = -\Sigma\, p_i \log q_i$$

p is the truth, q the prediction. This is the classification loss.

Lesson: Entropy, cross entropy and KL divergence

KL divergence

Exam favourite

$$KL(p \,||\, q) = \Sigma\, p_i \log(p_i / q_i) = H(p, q) - H(p)$$

Always 0 or more, and 0 only when p = q. Not symmetric.

Lesson: Entropy, cross entropy and KL divergence

z score

Exam favourite

$$z = (\text{measured} - \text{expected under null}) / \text{standard error}$$

How many standard errors away the result is.

Lesson: Hypothesis tests and confidence intervals

95% confidence interval

Exam favourite

$$\text{estimate} \pm 1.96 \times \text{standard error}$$

1.96 is where the normal curve leaves 2.5% in each tail.

Lesson: Hypothesis tests and confidence intervals

Conditional probability

Exam favourite

$$P(A \mid B) = P(A \text{ and } B) / P(B)$$

Only look at the cases where B happened.

Lesson: The maths, all in one place

Bayes rule

Exam favourite

$$P(A \mid B) = P(B \mid A)\, P(A) / P(B)$$

Posterior from prior and likelihood.

Lesson: The maths, all in one place

Expectation and variance Exam favourite

$$E[X] = \Sigma\ x\ P(x),\ Var(X) = E[X^2] - E[X]^2$$

Centre and spread.

Lesson: The maths, all in one place

Standard error Exam favourite

$$SE = \sigma / \sqrt{n}$$

From the central limit theorem.

Lesson: The maths, all in one place

Log likelihood Exam favourite

$$\log L(\theta) = \Sigma \log P(x_i \mid \theta)$$

Maximise it, or minimise its negative.

Lesson: The maths, all in one place

Cross entropy Exam favourite

$$H(p, q) = -\Sigma\ p \log q$$

The classification loss.

Lesson: The maths, all in one place

TABLES

Table: rule · formula · example with one die

Rule	Formula	Example with one die
Complement	P(not A) = 1 − P(A)	P(not 6) = 1 − 1/6 = 5/6
Sum rule (or)	P(A or B) = P(A) + P(B) − P(A and B)	P(even or at least 5) = 3/6 + 2/6 − 1/6 = 4/6
Product rule (and)	P(A and B) = P(A) × P(B | A)	Two sixes in a row = 1/6 × 1/6 = 1/36
Independent events	P(A and B) = P(A) × P(B)	Dice rolls do not affect each other
Conditional	P(A | B) = P(A and B) / P(B)	P(6 | even) = (1/6) / (3/6) = 1/3

Lesson: Probability: counting how likely things are

Table: distribution · what it counts · parameters · mean

Distribution	What it counts	Parameters	Mean	Variance	ML example
Bernoulli	One yes or no	p	p	p(1 − p)	Is this email spam
Binomial	Number of yes in n tries	n, p	np	np(1 − p)	Clicks out of n ads shown
Poisson	Events in a fixed time	λ	λ	λ	Calls per hour, website visits
Uniform	Any value in a range, all equal	a, b	(a + b)/2	$(b - a)^2/12$	Random initial weights
Normal (Gaussian)	Measurements around a centre	μ, σ	μ	σ^2	Heights, errors of a regression
Categorical	One of k classes	p_1 to p_k			Softmax output of a classifier

Lesson: The distributions you will meet everywhere

Table: term · plain meaning

Term	Plain meaning
Null hypothesis	The boring explanation: there is no real difference
p value	How surprising the data would be if the null were true
Significance level α	The cut off you chose before the test, often 0.05
Type I error	Saying there is a difference when there is none (false alarm)
Type II error	Missing a difference that is really there
Power	The chance of finding a real difference. More data gives more power

Lesson: Hypothesis tests and confidence intervals

Table: · frequentist · bayesian

	Frequentist	Bayesian
What probability means	How often something happens over many repeats	How strongly you believe something
Parameters	Fixed but unknown numbers	Random variables with a distribution
Uses a prior	No	Yes, updated by the data
Typical answer	A point estimate (MLE) and a confidence interval	A full posterior distribution and a credible interval
With little data	Can overfit or be unstable	The prior keeps estimates sensible
ML examples	Most of scikit-learn, p values, A/B tests	Naive Bayes, Gaussian processes, Bayesian optimisation for tuning

Lesson: Frequentist and Bayesian thinking compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1227	Airplane Seat Assignment Probability	Medium	Probability by cases	Try small n by hand. With one person the answer is 1. With more, the first person picks their own seat or the last person's seat with the same chance, and every other choice just passes the same problem to someone else, so the answer is 0.5.
528	Random Pick with Weight	Medium	Prefix sums and binary search	Turn the weights into running totals. Pick a random number from 1 to the total and binary search for the first running total that reaches it. Index i is chosen with chance w[i] / total.
1093	Statistics from a Large Sample	Medium	Mean, median and mode from counts	count[v] says how many times value v appears. Walk the counts once for the minimum, maximum, total, and the most common value. For the median, walk again until you pass the middle position (or the two middle positions).
295	Find Median from Data Stream	Hard	Two heaps	Keep the smaller half in a max heap and the larger half in a min heap, with sizes that differ by at most one. The median is the top of the bigger heap, or the average of both tops.
837	New 21 Game	Medium	Probability with dynamic programming	prob[x] is the chance of ever having exactly x points. Each draw adds 1 to maxPts with equal chance, so prob[x] is the sum of prob over the last maxPts totals that are still drawing (below k), divided by maxPts. Keep that sum in a sliding window.
688	Knight Probability in Chessboard	Medium	Spread the probability step by step	Keep a board of chances. Each move, every square sends 1/8 of its chance to each of its 8 knight moves that stay on the board. After k moves, add up the board.
808	Soup Servings	Medium	Expected value with memoised recursion	Count in units of 25 ml. Each of the 4 serving choices has chance 0.25. The chance that A runs out first, plus half the chance both run out together, is the average over the 4 choices. For big n the answer is so close to 1 that you can return 1.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER

P(A | B) is read "the probability of A given B". It means: only look at the cases where B happened, and ask how often A happened among them.

Lesson: Probability: counting how likely things are

IN THE EXAM

Ignoring the prior is called the **base rate fallacy**. It is a common exam trap and a real problem in fraud and disease detection, where the event you look for is rare.

Lesson: Bayes rule: updating a belief with new evidence

REMEMBER A **PMF** (probability mass function) gives probabilities for whole numbers like counts. A **PDF** (probability density function) is for continuous values. For a PDF, a single exact value has probability 0, and you get probabilities from areas under the curve, which is what the **CDF** (cumulative distribution function) adds up.

Lesson: The distributions you will meet everywhere

COMMON MISTAKE The sample standard deviation divides by n − 1 instead of n (ddof=1 in numpy, the default in pandas). This small change corrects for the fact that the sample mean is closer to the data than the true mean is. The program shows 12.9 with n and 13.79 with n − 1.

Lesson: Mean, variance and correlation

IN THE EXAM Correlation is not causation. Ice cream sales and drownings both rise in summer, but one does not cause the other. Correlation also only measures straight line relations: $y = x^2$ on symmetric x has correlation 0.

Lesson: Mean, variance and correlation

REMEMBER A model scores 90% accuracy on 400 test examples. The standard error is $\sqrt{(0.9 \times 0.1 / 400)} = 0.015$, so the real accuracy is likely within about 90% ± 3%. Two models that differ by 1% on this test set may not really differ at all.

Lesson: The law of large numbers and the central limit theorem

IN THE EXAM The program shows KL(p || q) = 0.5108 but KL(q || p) = 0.3681. KL divergence is not a true distance because swapping the two distributions changes the answer.

Lesson: Entropy, cross entropy and KL divergence

COMMON MISTAKE A p value is not the chance that the null hypothesis is true. If you test 20 useless features at the 0.05 level, about one will look significant by luck. Correct for many tests (for example the Bonferroni correction divides α by the number of tests).

Lesson: Hypothesis tests and confidence intervals

STEP 3 Foundations · Working with data

3.1 Data cleaning: missing values, types, outliers and joins

inspecting data, missing values and imputation, types and text, outliers, joins and reshaping, and a validated cleaning function.

KEY IDEAS

- **Inspect** shape, dtypes, isna().sum(), duplicated().sum(), unique(), describe(). Write down problems before fixing.
- **Missing values** dropna or fillna. Median for numbers, mode for categories, plus a missing flag. Fit the imputer on training data only.
- **MCAR, MAR, MNAR** Pure chance, depends on other columns, depends on the missing value. Dropping is only unbiased under MCAR.
- **Types and text** str.strip, str.lower, replace, to_numeric and to_datetime with errors="coerce", astype, drop_duplicates.
- **Outliers** IQR fences Q1 − 1.5 IQR and Q3 + 1.5 IQR (robust). |z| > 3 can be masked. Fix, remove, clip or keep.
- **Combine** merge on a key: inner, left, right, outer. Check row counts. melt makes long data, pivot makes wide data.

FORMULAS

IQR fences

Exam favourite

$$\text{low} = Q1 - 1.5 \times IQR,\ \text{high} = Q3 + 1.5 \times IQR,\ IQR = Q3 - Q1$$

Tukey's box plot rule. Use 3 instead of 1.5 for far outliers.

Lesson: Outliers: find them, then decide

z score

Exam favourite

$$z = (x - \text{mean}) / \text{std}$$

Flag $|z| > 3$. Easily fooled by the outliers themselves.

Lesson: Outliers: find them, then decide

Robust z score

$$z = 0.6745\ (x - \text{median}) / \text{MAD}$$

MAD is the median of $|x - \text{median}|$. Flag $|z| > 3.5$.

Lesson: Outliers: find them, then decide

Quantile position (pandas linear method)

Exam favourite

$$\text{position} = (n - 1) \times q, \text{ then interpolate between the two nearest sorted values}$$

Q1 is q = 0.25, the median q = 0.5, Q3 q = 0.75.

Lesson: The maths, all in one place

IQR fences

Exam favourite

$$Q1 - 1.5\ IQR \text{ and } Q3 + 1.5\ IQR$$

Values outside are outliers.

Lesson: The maths, all in one place

Mean after mean filling

Exam favourite

$$\text{mean stays the same, variance shrinks by the factor } (n_{\text{known}} - 1)/(n - 1) \text{ or so}$$

Every filled value sits exactly at the mean, adding no spread.

Lesson: The maths, all in one place

z score

$$z = (x - \bar{x}) / s$$

Distance from the mean in standard deviations.

Lesson: The maths, all in one place

TABLES

Table: what the output shows · the problem · the fix (next lessons)

What the output shows	The problem	The fix (next lessons)
price has type str, not a number	One value is the word "unknown"	pd.to_numeric(errors="coerce")
size_m2 has 1 missing, rooms has 2	Holes in the data	Drop or fill (impute)
1 duplicated row	House 5 appears twice	drop_duplicates()
Six different city spellings	Spaces and upper and lower case	str.strip().str.title()
size_m2 of 2000 for a flat	Almost surely a typing mistake	Outlier check, then fix or remove

Lesson: Load the data and look before you touch it

Table: why the value is missing · name · example · what to do

Why the value is missing	Name	Example	What to do
Pure chance	MCAR (missing completely at random)	A sensor glitch loses some readings	Dropping or simple filling is fine
Depends on other columns you have	MAR (missing at random)	Older people skip the "app usage" question	Fill using the other columns (KNN or iterative imputer)
Depends on the missing value itself	MNAR (missing not at random)	People with high income hide their income	Add a missing flag. Any fill is biased

Lesson: Missing values: drop or fill

Table: problem · pandas tool · example

Problem	pandas tool	Example
Extra spaces	str.strip()	" Delhi " to "Delhi"
Mixed case	str.lower(), str.title()	"DELHI" to "delhi"
Synonyms or typos	replace(dict)	{{"bombay": "mumbai"}}
Numbers as text	pd.to_numeric(errors="coerce")	"unknown" becomes NaN
Dates as text	pd.to_datetime(errors="coerce")	"2024-02-30" becomes NaT
Repeated rows	drop_duplicates(subset=...)	Keep the first copy
Wrong integer type	astype(int)	After all holes are filled

Lesson: Fix types, text, dates and duplicates

Table: join · keeps · in the program

Join	Keeps	In the program
inner	Only keys in both tables	3 rows: customer 4 has no name and Ravi has no order
left	Every left row	4 rows: Ravi appears with NaN order
right	Every right row	Would keep the order from customer 4
outer	Every key from both	Ravi and customer 4 both appear

Lesson: Combine and reshape tables

Table: · drop rows · mean or median fill · knn imputer

	Drop rows	Mean or median fill	KNN imputer	Iterative imputer	Model handles NaN
Idea	Remove rows with holes	One number for every hole	Average of the k most similar rows	Predict each column from the others, repeat	Trees learn which way NaN goes
Keeps rows	No	Yes	Yes	Yes	Yes
Uses other columns	No	No	Yes	Yes	Yes
Speed	Fast	Fast	Slow on big data (needs distances)	Slowest	Fast
Best when	Few holes, lots of data	MCAR, quick baseline	Similar rows have similar values	Columns are strongly related	HistGradientBoosting, XGBoost, LightGBM

Lesson: Smarter filling compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
2882	Drop Duplicate Rows	Easy	pandas: drop_duplicates on one column	Keep the first row for each email. drop_duplicates with subset set to the email column does exactly that.
2883	Drop Missing Data	Easy	pandas: dropna on one column	Remove rows where the name is missing. dropna with subset set to the name column keeps rows with holes elsewhere.
2885	Rename Columns	Easy	pandas: rename	Give rename a dictionary from old names to new names.
2886	Change Data Type	Easy	pandas: astype	The grade column holds floats like 73.0. astype(int) turns them into whole numbers.
2887	Fill Missing Data	Easy	pandas: fillna with a constant	A missing quantity here means none in stock, so fill it with 0.
2890	Reshape Data: Melt	Easy	pandas: melt wide to long	Keep product as the id column. Every quarter column becomes rows with a quarter name and a sales value.
1667	Fix Names in a Table	Easy	pandas: string cleaning	str.capitalize makes the first letter upper case and the rest lower case. Then sort by user_id.
1517	Find Users With Valid E-Mails	Easy	pandas: str.match with a pattern	The name must start with a letter and use only letters, digits, underscore, dot or dash, followed by exactly @leetcode.com.
196	Delete Duplicate Emails	Easy	pandas: sort, then drop duplicates in place	Sort by id so the smallest id comes first, then drop later copies of each email. LeetCode checks the same table, so change it in place.
183	Customers Who Never Order	Easy	pandas: anti join with isin	Keep customers whose id is not in the customerId column of orders. The tilde sign means "not".

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER pandas 3.0 stores text in a new string type, shown as `str`, instead of the old `object`. Older tutorials show object for text columns. Both work the same way for cleaning.

Lesson: Load the data and look before you touch it

COMMON MISTAKE Fitting the imputer on the whole dataset before splitting lets test rows influence the fill value. That is data leakage (Foundations 1.2). Put the imputer inside a Pipeline so it is fitted on the training part only.

Lesson: Missing values: drop or fill

REMEMBER errors="coerce" hides problems quietly. After using it, count the new NaN values and look at the original text of those rows, so you know what you turned into missing values.

Lesson: Fix types, text, dates and duplicates

IN THE EXAM The z score rule missed the 150 in the program: z = 2.97, just under 3. The outlier itself raised the mean and the standard deviation, which hid it. This is called **masking**. The IQR rule uses quartiles, which one extreme value cannot move, so it is more robust.

Lesson: Outliers: find them, then decide

REMEMBER Asha has two orders, so she appears twice after the join. If you wanted one row per customer, group the orders first (for example total amount per customer) and then merge. Use merge(..., validate="one_to_one") to make pandas check this for you.

Lesson: Combine and reshape tables

REMEMBER In the program, price depends on size. Mean filling ignores that and puts the same price on a small and a large flat, so its average error is 23.75. KNN looks at the 3 flats with the most similar size and is off by only 4.09. Scale the columns first in real data, because KNN uses distances.

Lesson: Smarter filling compared

3.2 Features: encoding, scaling, engineering and selection

encoding categories, scaling numbers, engineering new features, text features, feature selection and ColumnTransformer pipelines.

KEY IDEAS

- **Encoding** One hot for unordered (handle_unknown="ignore"), ordinal for a real order, target encoding with cross fitting for many categories.
- **Scaling** Standard (x − μ)/σ, min max to 0 to 1, robust (x − median)/IQR, log1p for skew. Trees need none.
- **No leakage** Fit scalers, encoders, imputers and selectors on training data only, inside a Pipeline and ColumnTransformer.
- **New features** Date parts, ratios, bins (pd.cut), logs, group aggregates, sin and cos for cycles, polynomial and interaction terms.
- **Text** CountVectorizer counts words. TF-IDF = tf × idf, idf high for rare words.
- **Selection** Filter (F test, mutual information), wrapper (RFE), embedded (L1, tree importance). Choose k with cross validation.

FORMULAS

TF-IDF (scikit-learn smooth version)

Exam favourite

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times (\ln((1 + N) / (1 + \text{df}(t))) + 1),$$

then each row is scaled to length 1

N is the number of texts, df(t) the number of texts containing t.

Lesson: Features from text

Standardisation Exam favourite

$$z = (x - \mu) / \sigma$$

Mean 0, standard deviation 1.

Lesson: The maths, all in one place

Min max Exam favourite

$$x' = (x - \min) / (\max - \min)$$

Range 0 to 1.

Lesson: The maths, all in one place

Robust Exam favourite

$$x' = (x - \text{median}) / \text{IQR}$$

Ignores outliers when choosing the scale.

Lesson: The maths, all in one place

Number of polynomial columns Exam favourite

$$C(d + p, p) - 1$$

d features, degree p, without the constant column.

Lesson: The maths, all in one place

Mutual information

$$I(X, Y) = \Sigma\, p(x, y) \log(p(x, y) / (p(x)\, p(y)))$$

0 when X and Y are independent. Catches non linear links.

Lesson: The maths, all in one place

Smoothed target encoding

$$\text{enc}(c) = (n_c \times \text{mean}_c + m \times \text{mean}_{\text{all}}) / (n_c + m)$$

Rare categories move towards the overall mean.

Lesson: The maths, all in one place

TABLES

Table: scaler · formula · outliers · use it for

Scaler	Formula	Outliers	Use it for
StandardScaler	(x − mean) / std	Pull the mean and inflate the std	The default for linear models, SVM, PCA, neural networks
MinMaxScaler	(x − min) / (max − min)	One outlier squeezes every other value	Pixel values, inputs that must be in 0 to 1
RobustScaler	(x − median) / IQR	Barely affected	Data with outliers you want to keep
Log transform	log(1 + x)	Shrinks big values a lot	Skewed data: prices, incomes, counts
No scaling			Decision trees, random forests, gradient boosting

Lesson: Scaling numbers

Table: kind · example · pandas or sklearn

Kind	Example	pandas or sklearn
Date parts	month, weekday, is_weekend, hour	.dt.month, .dt.dayofweek
Ratios	price per m^2, debt to income	a / b
Binning	age groups new, mid, old	pd.cut, KBinsDiscretizer
Log or power	log of price	np.log1p, PowerTransformer
Aggregates	average spend per customer	groupby().transform("mean")
Cyclical	hour 23 is close to hour 0	sin and cos of 2π × hour / 24

Lesson: Creating new features

Table: · one hot · ordinal · target encoding

	One hot	Ordinal	Target encoding	Hashing	Learned embedding
Columns made	k	1	1 (per class)	A fixed number you choose	A small fixed number
Many categories (1,000+)	Too wide	Fine but fake order	Good	Good	Good
Leakage risk	None	None	High unless cross fitted	None	Low
Unseen categories	All zeros	Needs a rule	Overall mean	Hashed like any value	Unknown token
Best with	Linear models, few categories	Trees, real order	High cardinality, boosting	Huge, streaming data	Neural networks (Deep Learning)

Lesson: Encodings compared, including target encoding

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1873	Calculate Special Bonus	Easy	pandas: a feature from a rule	The bonus is the full salary when the id is odd and the name does not start with M, otherwise 0. Build the column with a boolean mask, then sort by id.
1757	Recyclable and Low Fat Products	Easy	pandas: combine two masks	Keep rows where both flags are "Y". Use & between the two conditions and wrap each in brackets.
595	Big Countries	Easy	pandas: an "or" mask	A country is big if its area is at least 3,000,000 or its population is at least 25,000,000. Use | for "or".
178	Rank Scores	Medium	pandas: dense rank as a feature	rank(method="dense", ascending=False) gives equal scores the same rank with no gaps. Sort by score from high to low.
2356	Number of Unique Subjects Taught by Each Teacher	Easy	pandas: groupby with nunique	Group by teacher and count the different subject ids. reset_index turns the result back into a table.
1741	Find Total Time Spent by Each Employee	Easy	pandas: a computed column, then groupby sum	Make a column out_time minus in_time, then add it up for each day and employee.
1484	Group Sold Products By The Date	Easy	pandas: groupby with two aggregates	For each date, count the different products and join their sorted names with commas.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER pd.get_dummies is handy for a quick look, but use OneHotEncoder in real projects. It remembers the categories it saw during training, so the test data always gets exactly the same columns in the same order.

Lesson: Encoding categories as numbers

IN THE EXAM Trees split on one feature at a time with thresholds like "income > 52". Multiplying a feature by 1,000 moves the threshold but not the split, so tree models do not need scaling.

Lesson: Scaling numbers

COMMON MISTAKE Only build features from information you would have at prediction time. "Number of support calls in the month after cancelling" predicts cancelling perfectly, and is useless (and leaky) for a real prediction.

Lesson: Creating new features

COMMON MISTAKE The number of columns grows fast. With d features and degree p there are C(d + p, p) − 1 columns: 10 features at degree 3 gives 285 columns. Use regularisation (Supervised 1.2) to stop the extra columns from overfitting.

Lesson: Polynomial and interaction features

REMEMBER In this program the full 30 features give the best accuracy (0.981). Selection does not always raise the score. Here its value is a much simpler model: 5 features keep 0.949. Different methods also choose different features when features are correlated, as the radius, perimeter and area columns are.

Lesson: Feature selection: keep what helps

COMMON MISTAKE Selecting features on the whole dataset and then cross validating is leakage: the test folds helped choose the features, so the score is too optimistic. Put the selector inside the Pipeline, as the k loop in the program does.

Lesson: Feature selection: keep what helps

IN THE EXAM

Target encoding replaces a category by the average target for that category. Using a row's own target to encode it leaks the answer. scikit-learn's TargetEncoder avoids this with cross fitting: each row is encoded with means from the other folds, and rare categories are pulled towards the overall mean. That is why its numbers in the program differ a little from the plain shop means.

Lesson: Encodings compared, including target encoding

STEP 1 Supervised Learning · Regression

1.1 Linear regression: fit the best straight line

least squares, the normal equation, regression metrics, gradient descent, residual checks, robust regression and statistical inference.

KEY IDEAS

- **Model and loss** $\hat{y} = b_0 + \Sigma b_j x_j$. Minimise $\Sigma(y - \hat{y})^2$. Slope = cov(x, y)/var(x), line through $(\bar{x}, \bar{y})$.
- **Solving** Normal equation $w = (X^T X)^{-1} X^T y$ (use solve), or gradient descent with scaled features.
- **Metrics** MAE, MSE, RMSE, $R^2 = 1 - SSE/SST$, adjusted R^2. Always on test data.
- **Assumptions** Linear, independent errors, equal spread, normal errors for p values. Check residuals for curves and funnels.
- **Outliers** Squared loss is pulled by outliers. Huber or absolute loss is robust. Median minimises absolute error.
- **Inference** statsmodels OLS gives standard errors, p values and confidence intervals. Correlated features make weights unstable.

FORMULAS

Simple linear regression — Exam favourite

$$b_1 = \Sigma(x_i - \bar{x})(y_i - \bar{y}) / \Sigma(x_i - \bar{x})^2,\ b_0 = \bar{y} - b_1\bar{x}$$

The slope is cov(x, y) / var(x).

Lesson: Linear regression: predict a number with a straight line

Least squares loss — Exam favourite

$$L(b) = \Sigma\ (y_i - \hat{y}_i)^2$$

Squaring makes every error positive and punishes big errors more.

Lesson: Linear regression: predict a number with a straight line

Normal equation — Exam favourite

$$w = (X^T X)^{-1} X^T y$$

Exact answer. Costs about $O(nd^2 + d^3)$, so fine up to a few thousand features.

Lesson: Many features and the normal equation

Huber loss — Exam favourite

$$L_\delta(e) = e^2/2 \text{ if } |e| \le \delta, \text{ otherwise } \delta|e| - \delta^2/2$$

Quadratic near 0, linear far away.

Lesson: Outliers and robust regression

Absolute error loss

$$L(e) = |e|$$

The best constant under it is the median, not the mean.

Lesson: Outliers and robust regression

Adjusted R^2 — Exam favourite

$$R^2_{adj} = 1 - (1 - R^2)(n - 1)/(n - d - 1)$$

Plain R^2 never falls when you add a feature. The adjusted version does if the feature does not help enough.

Lesson: Reading weights like a statistician

Model Exam favourite

$$\hat{y} = Xw \text{ (X has a first column of 1s)}$$

A weighted sum of the features.

Lesson: The maths, all in one place

Loss Exam favourite

$$L(w) = ||y - Xw||^2$$

Sum of squared errors.

Lesson: The maths, all in one place

Gradient Exam favourite

$$\nabla L = -2X^T(y - Xw)$$

Set to 0 for the minimum.

Lesson: The maths, all in one place

R^2 Exam favourite

$$R^2 = 1 - SSE/SST,\ SST = \Sigma(y - \bar{y})^2$$

Share of variation explained.

Lesson: The maths, all in one place

Simple regression slope

$$b_1 = r \times s_y / s_x$$

r is the correlation of x and y.

Lesson: The maths, all in one place

TABLES

Table: metric · formula · unit · notes

Metric	Formula	Unit	Notes
MAE	$(1/n)\ \Sigma\ \|y - \hat{y}\|$	Same as y	Easy to explain. Treats all errors the same
MSE	$(1/n)\ \Sigma\ (y - \hat{y})^2$	y squared	The training loss. Hard to read
RMSE	√MSE	Same as y	Punishes big errors more than MAE
R^2	1 − SSE / SST	No unit	Share of variation explained. Compare models on the same data
MAPE	$(1/n)\ \Sigma\ \|y - \hat{y}\| / \|y\|$	Percent	Breaks when y is near 0

Lesson: Fit, predict and measure on real data

Table: · normal equation (linearregression) · gradient descent (sgdregressor)

	Normal equation (LinearRegression)	Gradient descent (SGDRegressor)
Answer	Exact, in one step	Close, after many steps
Needs scaling	No	Yes
Learning rate to choose	No	Yes
Many features (100,000+)	Slow: d^3 work	Fine
Data larger than memory	No	Yes, with partial_fit
Use when	Most tabular problems	Huge or streaming data

Lesson: Gradient descent for linear regression

Table: what you see in the residuals · meaning · fix

What you see in the residuals	Meaning	Fix
A curve (U shape)	The relation is not linear	Add x^2, a log, or use a tree model
A funnel (spread grows)	Heteroscedasticity	Log transform y, weighted least squares, robust errors
A pattern over time	Errors are not independent	Time series models
A few huge residuals	Outliers or data errors	Check the rows, use Huber regression

Lesson: Assumptions and residual checks

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1232	Check If It Is a Straight Line	Easy	Same slope everywhere (cross multiply)	Compare the slope from the first point to every other point with the first slope. Cross multiply instead of dividing so a vertical line does not divide by zero.
2280	Minimum Lines to Represent a Line Chart	Medium	Sort, then count slope changes	Sort the points by day. A new line starts whenever the slope between neighbours changes. Compare slopes by cross multiplying to avoid float errors.
149	Max Points on a Line	Hard	Count slopes from each point	Fix one point. Every other point gives a slope, stored as a reduced fraction (dy, dx) with gcd so equal slopes match exactly. The most common slope plus the fixed point is a line.
462	Minimum Moves to Equal Array Elements II	Medium	The median minimises absolute error	Moving every number to one value c costs the sum of |x − c|. That sum is smallest at the median, the same reason MAE is minimised by the median while MSE is minimised by the mean.
2448	Minimum Cost to Make Array Equal	Hard	Weighted median of a convex cost	The total cost is a convex function of the target value. Its minimum is at the weighted median: sort by value and walk until the running cost weight reaches half of the total weight.
1515	Best Position for a Service Centre	Hard	Gradient descent on a convex loss	The sum of distances is convex, so gradient descent works. Step against the gradient and halve the step size whenever a step does not help, until the step is tiny. (The Weiszfeld update is a faster version.)

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER

Read a weight as "holding the other features fixed". Above, the rooms weight is 3.7 lakh per extra room for a flat of the same size and age. If two features are strongly correlated (size and rooms), their separate weights become unstable and hard to read. This is called multicollinearity, and ridge regression (next chapter) fixes it.

Lesson: Many features and the normal equation

REMEMBER

An R^2 of 0.36 on the diabetes data sounds low, but medical data is noisy. Judge a score against a simple baseline (always the mean, $R^2 = 0$) and against what is possible in that field, not against 1.

Lesson: Fit, predict and measure on real data

IN THE EXAM

A feature can be statistically significant and still useless for prediction, or very useful for prediction while its p value is unclear because it is correlated with another feature. Prediction (machine learning) and inference (statistics) answer different questions.

Lesson: Reading weights like a statistician

1.2 Polynomial regression and regularisation: ridge, lasso, elastic net

polynomial features, underfitting and overfitting, ridge, lasso, elastic net, and choosing alpha with cross validation.

KEY IDEAS

- **Fit diagnosis** Underfit: both errors high (bias). Overfit: train low, test high (variance). Choose complexity on validation data.
- **Ridge** Penalty $\alpha\Sigma w^2$. $w = (X^TX + \alpha I)^{-1}X^Ty$. Shrinks, never zeros. Fixes multicollinearity.
- **Lasso** Penalty $\alpha\Sigma|w|$. Soft threshold sets weak weights to 0, so it selects features. Coordinate descent.
- **Elastic net** Mix with l1_ratio. Keeps correlated groups together. Safe default.
- **Practice** Scale first (Pipeline). Tune α with RidgeCV, LassoCV, ElasticNetCV on a log grid. Extend the grid if the best is at the edge.

- **Theory** L1 diamond vs L2 ball. Ridge = Gaussian prior, lasso = Laplace prior. Ridge shrinks along SVD directions by $d^2/(d^2 + \alpha)$.

FORMULAS

Soft threshold (one lasso update) Exam favourite

$$w_j = \text{sign}(\rho)\ \max(|\rho| - \alpha, 0) / z_j$$

ρ is how much feature j explains the remaining error. If $|\rho| \le \alpha$ the weight becomes 0.

Lesson: Lasso (L1): weights that become exactly zero

Ridge Exam favourite

$$\min \|y - Xw\|^2 + \alpha\|w\|_2{}^2,\ w = (X^TX + \alpha I)^{-1}X^Ty$$

Always solvable because $X^TX + \alpha I$ can be inverted for $\alpha > 0$.

Lesson: The maths, all in one place

Lasso Exam favourite

$$\min (1/2n)\|y - Xw\|^2 + \alpha\|w\|_1$$

This is the exact scaling scikit-learn uses.

Lesson: The maths, all in one place

Elastic net Exam favourite

$$\min (1/2n)\|y - Xw\|^2 + \alpha r\|w\|_1 + (\alpha(1 - r)/2)\|w\|_2{}^2$$

r is l1_ratio.

Lesson: The maths, all in one place

Ridge on one scaled feature

$$w_{ridge} = w_{ols} \times s / (s + \alpha)$$

$s = \Sigma x^2$. Shrinks by a factor, never to 0.

Lesson: The maths, all in one place

Lasso on one scaled feature

$$w_{lasso} = \text{sign}(w_{ols})\ \max(|w_{ols}| - \alpha', 0)$$

Moves by a fixed amount and stops at 0.

Lesson: The maths, all in one place

TABLES

Table: · underfitting · good fit · overfitting

	Underfitting	Good fit	Overfitting
Training error	High	Low	Very low
Test error	High	Low	High
Also called	High bias	Balanced	High variance
Fixes	More features, a more flexible model, less penalty	Keep it	More data, fewer features, simpler model, more penalty

Lesson: Curves, underfitting and overfitting

Table: · least squares · ridge (l2) · lasso (l1)

	Least squares	Ridge (L2)	Lasso (L1)	Elastic net
Penalty	None	$\alpha\Sigma w^2$	$\alpha\Sigma\|w\|$	Both
Weights exactly 0	No	No	Yes	Yes
Correlated features	Unstable weights	Shares weight evenly	Picks one, drops others	Keeps the group
Closed form	Yes	Yes	No (coordinate descent)	No
More features than rows	Fails (no unique answer)	Works	Keeps at most n features	Works
Best for	Few, clean features	Many useful features	Few useful among many	Correlated groups, safe default

Lesson: Penalties compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
2333	Minimum Sum of Squared Difference	Medium	Shrink the largest values first (like an L2 penalty)	Only the gaps |a − b| matter, and you have k1 + k2 unit decreases. A squared sum falls fastest when you cut the biggest gaps, so bring every gap above some level down to that level (binary search the level), then spend what is left one by one.
1685	Sum of Absolute Differences in a Sorted Array	Medium	L1 distances with prefix sums	For position i, numbers on the left are smaller and numbers on the right are larger. With prefix sums, the left part is i × x minus their sum and the right part is their sum minus count × x.
2602	Minimum Operations to Make All Array Elements Equal	Medium	L1 cost per query with prefix sums and binary search	Sort once and build prefix sums. For a query q, binary search how many numbers are below q. The cost is (count below × q − their sum) plus (sum above − count above × q).
1838	Frequency of the Most Frequent Element	Medium	A budget constraint with a sliding window	Sort. For a window ending at nums[r], raising every number to nums[r] costs nums[r] × size − window sum. Shrink the window from the left while the cost is over the budget k. Like a penalty budget, k limits how far the values may move.
1802	Maximum Value at a Given Index in a Bounded Array	Medium	Binary search under a sum budget	The cheapest array with value v at index falls by 1 per step on both sides and never goes below 1. Its sum grows with v, so binary search the largest v whose minimum sum fits in maxSum.
1049	Last Stone Weight II	Medium	Pick signs to make a sum as close to 0 as possible	Smashing stones is the same as giving each stone a + or − sign. Split them into two groups with sums as equal as possible: find all reachable subset sums up to half the total.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER — In the program, size and rooms are almost the same feature. Least squares gives them 2.55 and 0.44, an arbitrary split. Ridge with α = 1 gives 1.49 and 1.45: a fair share, and much more stable if the data changes a little.

Lesson: Ridge regression (L2): keep weights small

IN THE EXAM — In the program the true model uses 3 of 10 features. At α = 0.5, lasso keeps exactly those 3 and sets the other 7 to 0. Ridge with α = 10 keeps all 10 with small weights.

Lesson: Lasso (L1): weights that become exactly zero

REMEMBER — The three features a, b and c are almost the same. Lasso gives them 0.91, 0.24 and 1.74 (an uneven, arbitrary split), while elastic net gives 0.94, 0.88 and 1.07, closer to an even share of the true weight 3.

Lesson: Elastic net: the best of both

REMEMBER — With 65 polynomial features on 309 training rows, plain least squares overfits (test R^2 0.225). Ridge and lasso both lift it to about 0.37, and lasso does it with only 14 of the 65 features.

Lesson: Choosing alpha with cross validation

STEP 2 Supervised Learning · Classification

2.1 Logistic regression: probabilities for classification

the sigmoid, log loss training, thresholds, softmax for many classes, odds ratios and the penalty C.

KEY IDEAS

- **Model** $P(y = 1 \mid x) = \sigma(w \cdot x + b)$, $\sigma(z) = 1/(1 + e^{-z})$. Score = log odds. Boundary $w \cdot x + b = 0$.
- **Training** Minimise log loss (convex). Gradient $(1/n)\Sigma(p - y)x$. No closed form, use lbfgs or gradient descent.
- **Threshold** Default 0.5. Lower it for higher recall, raise it for higher precision. Choose from costs on validation data.
- **Multi class** Softmax $p_j = e^{z_j}/\Sigma e^{z}$, loss $-\ln p_{true}$. Or one versus rest.
- **Interpretation** Odds ratio e^{w} per unit. Scale features for comparable weights.
- **Penalty** $C = 1/\alpha$, small C = strong L2 penalty. Needed for separable data. Polynomial features give curved boundaries.

FORMULAS

Sigmoid Exam favourite

$$\sigma(z) = 1 / (1 + e^{-z})$$

Maps any number to 0 to 1. $\sigma(0) = 0.5$.

Lesson: From a score to a probability

Logistic model Exam favourite

$$P(y = 1 \mid x) = \sigma(w \cdot x + b)$$

A linear score inside a sigmoid.

Lesson: From a score to a probability

Log odds (logit) Exam favourite

$$\ln(p / (1 - p)) = w \cdot x + b$$

Linear in the features.

Lesson: From a score to a probability

Log loss Exam favourite

$$L = -(1/n) \Sigma [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$$

Convex in w and b.

Lesson: Training with log loss

Gradient Exam favourite

$$\partial L/\partial w = (1/n) \Sigma (p_i - y_i) x_i$$

Prediction minus label, times the input.

Lesson: Training with log loss

Softmax Exam favourite

$$p_j = e^{z_j} / \Sigma_m e^{z_m}$$

Subtract the largest score first so e^{z} never overflows. The answer does not change.

Lesson: More than two classes: softmax

Model Exam favourite

$$p = \sigma(w \cdot x + b), \sigma(z) = 1 / (1 + e^{-z})$$

Probability of class 1.

Lesson: The maths, all in one place

Sigmoid derivative Exam favourite

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Used to derive the gradient.

Lesson: The maths, all in one place

Log loss and gradient Exam favourite

$$L = -(1/n)\Sigma[y \ln p + (1 - y) \ln(1 - p)], \nabla_w L = (1/n) X^T(p - y)$$

The derivative of the sigmoid cancels nicely.

Lesson: The maths, all in one place

Softmax and its loss Exam favourite

$$p_j = e^{z_j} / \Sigma e^{z_m}, L = -\ln p_{true}$$

Gradient for class j: $p_j - 1[j = true]$.

Lesson: The maths, all in one place

Odds ratio

$$OR = e^{w}$$

Multiplier on the odds per unit of the feature.

Lesson: The maths, all in one place

TABLES

Table: · linear regression on 0 and 1 · logistic regression · naive bayes

	Linear regression on 0 and 1	Logistic regression	Naive Bayes	Tree models
Output	Any number	A probability	A probability	Class or probability
Boundary	Straight	Straight (curved with extra features)	Depends on the distributions	Boxes, very flexible
Loss	Squared error	Log loss (convex)	Counts and likelihoods	Impurity
Calibrated probabilities	No	Usually good	Often too extreme	Often need calibration
Explainable	Yes	Yes: odds ratios	Yes	Single tree yes, forests less
Needs scaling	Helpful	Yes for the solver and penalty	No	No

Lesson: Logistic regression compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1822	Sign of the Product of an Array	Easy	Decide the sign without the big number	A 0 makes the product 0. Otherwise only the count of negative numbers matters: odd gives −1, even gives 1. A classifier also only needs the sign of its score.
1608	Special Array With X Elements Greater Than or Equal X	Easy	Search for a threshold	Try every threshold x from 0 to n and count how many numbers are at least x. Exactly x of them means x is the answer.
2389	Longest Subsequence With Limited Sum	Easy	Prefix sums and a threshold per query	Take the smallest numbers first. Sort, build prefix sums, and for each query binary search how many prefix sums stay at or under it.
1385	Find the Distance Value Between Two Arrays	Easy	Nearest neighbour check with a threshold	Sort arr2. For each x in arr1, binary search the closest values in arr2. Count x when even the closest one is more than d away.
2300	Successful Pairs of Spells and Potions	Medium	Binary search for the decision boundary	Sort the potions. For a spell s, a potion works when s × potion ≥ success, so find the first potion at least ceil(success / s). Everything to its right succeeds, like points past a decision boundary.
1300	Sum of Mutated Array Closest to Target	Medium	Choose a threshold that minimises an error	Capping every number at v gives a sum that grows with v. Binary search the largest v whose capped sum is at most the target, then compare v and v + 1 and keep the one with the smaller error (smaller v on a tie).

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER The program learns w = 2.59 and b = −7.11, so the 50% point is 7.11 / 2.59 = 2.75 hours. Students who study more than 2.75 hours are predicted to pass.

Lesson: Training with log loss

COMMON MISTAKE LogisticRegression adds an L2 penalty by default (C = 1.0). If you see a ConvergenceWarning, scale the features or raise max_iter. Do not just ignore it.

Lesson: Logistic regression in scikit-learn

IN THE EXAM Here 1 means malignant. Moving the threshold from 0.5 down to 0.2 cuts the missed cancers from 2 to 1, and the price is a precision drop from 1.0 to 0.852 (more healthy people sent for extra tests).

Lesson: Choosing the threshold

REMEMBER One versus rest (OvR) is the other way to do multi class: train k separate yes or no models and pick the most confident. Softmax trains them together and gives probabilities that add up to 1, so it is usually preferred.

Lesson: More than two classes: softmax

COMMON MISTAKE Odds are not probabilities. Tripling the odds from 0.1 to 0.3 moves the probability from 9% to 23%, not to 27%. Near 50% the effect on probability is largest.

Lesson: Reading the weights: odds ratios

IN THE EXAM In scikit-learn, C is the inverse of the penalty strength. C = 0.001 shrinks the weights a lot (underfits, 0.891), C = 1 is best here (0.981), and C = 100 hardly penalises (0.963, slightly overfit). Ridge uses α, where a bigger number means a stronger penalty, so do not mix them up.

Lesson: Curved boundaries and the penalty C

2.2 k nearest neighbours: learn from similar examples

the KNN vote, choosing k, distance measures, scaling, KNN regression, the curse of dimensionality and fast similarity search.

KEY IDEAS

- **Algorithm** Store data. For a query: distances to all, sort, k nearest, majority vote (or average for numbers). No training, slow prediction O(nd).
- **Choosing k** Small k overfits, large k underfits. k = 1 gives 100% training accuracy. Cross validate, odd k for two classes.
- **Distance** Euclidean, Manhattan, Minkowski p, cosine for text. Scale features first, always.
- **Variants** weights="distance" (1/d). KNeighborsRegressor. NearestNeighbors for similarity search.
- **High dimensions** Distances concentrate, nearest is barely nearer than farthest. Select features or use PCA first.
- **Speed and theory** k-d tree and ball tree for low d, approximate search (HNSW) for embeddings. 1-NN error at most twice Bayes error.

FORMULAS

Minkowski distance Exam favourite

$$d(a, b) = (\Sigma\ |a_i - b_i|^p)^{1/p}$$

p = 1 Manhattan, p = 2 Euclidean, p → ∞ Chebyshev (largest difference).

Lesson: Distance measures and scaling

Cosine similarity Exam favourite

$$\cos(a, b) = a \cdot b\ /\ (||a||\ ||b||)$$

1 means the same direction. Cosine distance = 1 − cos.

Lesson: Distance measures and scaling

Weighted KNN prediction Exam favourite

$$\hat{y} = \Sigma\ (1/d_i)\ y_i\ /\ \Sigma\ (1/d_i) \text{ over the k nearest}$$

If some $d_i = 0$, that point's value is used directly.

Lesson: KNN for numbers and weighted votes

KNN classification Exam favourite

$$\hat{y} = \text{most common class among the k nearest } N_k(x)$$

Majority vote.

Lesson: The maths, all in one place

KNN regression Exam favourite

$$\hat{y} = (1/k)\ \Sigma_{i \in N_k(x)}\ y_i$$

Local average.

Lesson: The maths, all in one place

Euclidean distance Exam favourite

$$d = \sqrt{\Sigma(a_i - b_i)^2}$$

The default.

Lesson: The maths, all in one place

Cost of brute force prediction Exam favourite

$$O(n \times d) \text{ per query}$$

n stored examples, d features.

Lesson: The maths, all in one place

Cover and Hart bound

$$R^* \le R_{1NN} \le 2R^*(1 - R^*) \text{ as } n \to \infty$$

With endless data, 1-NN errs at most twice the best possible error R* (two classes).

Lesson: The maths, all in one place

TABLES

Table: · knn · logistic regression · decision tree

	KNN	Logistic regression	Decision tree
Training time	None (just store)	Fast	Fast
Prediction time	Slow: compare with all n examples	Very fast	Very fast
Memory	Keeps the whole training set	One weight per feature	The tree
Boundary	Any shape	Straight	Boxes
Needs scaling	Yes, essential	Yes	No
Many features	Suffers badly (curse of dimensionality)	Fine with a penalty	Fine
Explanation	Shows similar examples	Weights and odds ratios	Rules

Lesson: KNN compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
973	K Closest Points to Origin	Medium	Nearest neighbours with a heap	Keep a max heap of the k closest points seen so far (store negative squared distances). Push each point and pop when the heap grows past k. Squared distance is enough, no square root needed.
658	Find K Closest Elements	Medium	Binary search for the start of the window	The answer is a window of k neighbours in the sorted list. Binary search its left edge: if x is closer to arr[mid + k] than to arr[mid], move right.
1637	Widest Vertical Area Between Two Points Containing No Points	Easy	Sort along one axis, then look at neighbours	Only x matters. Sort the x values and take the largest gap between neighbouring values. Sorting first is also how a k-d tree splits points along one axis.
1030	Matrix Cells in Distance Order	Easy	Sort by Manhattan distance	List every cell and sort by $\|r - r0\| + \|c - c0\|$. This is exactly how KNN orders candidates before taking the first k.
1584	Min Cost to Connect All Points	Medium	Manhattan distances and a minimum spanning tree	Grow a tree from point 0. Keep, for every point outside the tree, its cheapest Manhattan distance to the tree, and add the cheapest point each round (Prim's algorithm, $O(n^2)$).
719	Find K-th Smallest Pair Distance	Hard	Binary search on the distance, count with two pointers	Sort. For a guess distance g, count pairs with difference at most g using a sliding window. The answer is the smallest g with at least k such pairs.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM In the program, k = 1 scores 1.0 on training data but only 0.86 in cross validation. The best is around k = 7 (0.913), and k = 101 underfits (0.82). Training accuracy is useless for choosing k.

Lesson: Choosing k

COMMON MISTAKE Foundations 3.2 showed scaling raise KNN accuracy on the wine data from 0.691 to 0.949. Scaling is not optional for KNN.

Lesson: Distance measures and scaling

2.3 Naive Bayes: classify by counting clues

Bayes rule as a classifier, text with multinomial naive Bayes, smoothing, logs, Gaussian and Bernoulli versions, and its limits.

KEY IDEAS

- **Rule** $P(c \mid x) \propto P(c)\Pi P(x_j \mid c)$. Features independent given the class. Pick argmax of $\log P(c) + \Sigma \log P(x_j \mid c)$.
- **Smoothing** $P(w \mid c) = (n_{w,c} + \alpha)/(n_c + \alpha V)$. Prevents zeros. Use logs to avoid underflow.
- **Variants** MultinomialNB counts (text), BernoulliNB 0 or 1 (absence counts), GaussianNB numbers (mean, variance), ComplementNB imbalanced text.
- **Strengths** One pass of counting, little data, many features, no scaling. Great text baseline.
- **Weaknesses** Overconfident probabilities, no feature interactions, correlated features double counted. Calibrate if probabilities matter.
- **Theory** Generative. Linear boundary for text. Ng and Jordan: faster to its (higher) best error than logistic regression.

FORMULAS

Naive Bayes rule Exam favourite

$$P(c \mid x_1, \ldots, x_d) \propto P(c)\ \Pi_j\ P(x_j \mid c)$$

Prior times one likelihood per feature.

Lesson: Bayes rule as a classifier

Prediction Exam favourite

$$\hat{y} = \operatorname{argmax}_c\ [\log P(c) + \Sigma_j \log P(x_j \mid c)]$$

Logs turn the product into a sum.

Lesson: Bayes rule as a classifier

Laplace (add alpha) smoothing Exam favourite

$$P(w \mid c) = (n_{w,c} + \alpha)\ /\ (n_c + \alpha V)$$

Every word gets a small chance, so no probability is ever 0.

Lesson: Smoothing and log probabilities

Gaussian likelihood Exam favourite

$$p(x_j \mid c) = (1\ /\ \sqrt{(2\pi\sigma_{jc}{}^2)})\ \exp(-(x_j - \mu_{jc})^2\ /\ (2\sigma_{jc}{}^2))$$

One mean and one variance per feature and class.

Lesson: Gaussian naive Bayes for numbers

Bayes rule Exam favourite

$$P(c \mid x) = P(x \mid c)\ P(c)\ /\ P(x)$$

P(x) is the same for every class, so it can be ignored when comparing.

Lesson: The maths, all in one place

Naive assumption Exam favourite

$$P(x_1, \ldots, x_d \mid c) = \Pi_j\ P(x_j \mid c)$$

Features independent given the class.

Lesson: The maths, all in one place

Multinomial likelihood Exam favourite

$$P(w \mid c) = (n_{w,c} + \alpha)\ /\ (n_c + \alpha V)$$

Smoothed word share.

Lesson: The maths, all in one place

Bernoulli likelihood

$$P(x \mid c) = \Pi_j\ p_{jc}{}^{x_j}\ (1 - p_{jc})^{1\ -\ x_j}$$

Absent features count through (1 – p).

Lesson: The maths, all in one place

Log score Exam favourite

$$\text{score}(c) = \log P(c) + \Sigma \log P(x_j \mid c)$$

Pick the largest. No underflow.

Lesson: The maths, all in one place

TABLES

Table: model · features · p(x j | c) · typical use

Model	Features	$P(x_j \mid c)$	Typical use
MultinomialNB	Counts (how many times)	Share of counts	Text with word counts or TF-IDF
BernoulliNB	0 or 1 (present or not)	Chance the feature is present, and absent words count too	Short texts, binary flags
GaussianNB	Real numbers	Normal density	Measurements
CategoricalNB	Categories	Share of each category	Tabular categorical data
ComplementNB	Counts	Learned from the other classes	Text with imbalanced classes

Lesson: Which naive Bayes to use

Table: · naive bayes · logistic regression · knn

	Naive Bayes	Logistic regression	KNN
Type	Generative (models P(x | y))	Discriminative (models P(y | x))	Instance based
Training	One pass of counting	Iterative optimisation	None
Little data	Good	Can overfit	Poor
Many features (text)	Excellent	Good with a penalty	Poor
Feature interactions	No	Only if you add them	Yes
Probabilities	Overconfident	Well calibrated	Rough (share of votes)

Lesson: Naive Bayes compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
242	Valid Anagram	Easy	Compare two count tables	Two strings are anagrams when every letter appears the same number of times. Build a count table for each and compare them, exactly like comparing word counts per class.
383	Ransom Note	Easy	Counts as a budget	Count the letters in the magazine. Each letter of the note uses one count. If a count would go below zero, the note cannot be made.
819	Most Common Word	Easy	Tokenise, then count	Turn punctuation into spaces, lower the case, split into words, skip banned words, and count. This is the same tokenising step CountVectorizer does before naive Bayes.
884	Uncommon Words from Two Sentences	Easy	Count words across both texts	A word is uncommon when it appears exactly once in the two sentences together. Count all words of both and keep the ones with count 1.
1160	Find Words That Can Be Formed by Characters	Easy	Compare counts per word	Count the available characters once. A word is good when, for every letter, it needs no more than are available. Add up the lengths of the good words.
692	Top K Frequent Words	Medium	Count, then sort by count and word	Count every word. Sort by count from high to low, and alphabetically when counts tie. Keep the first k. This is how you find the most telling words of a class.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER

The hand made filter gave 0.871 for "free money now" and scikit-learn gives 0.882. The difference is the vocabulary: CountVectorizer ignores one letter words like "a" by default, so it has 17 words instead of 18, which changes the smoothing slightly. Always check what the tokeniser keeps.

Lesson: Text classification with multinomial naive Bayes

IN THE EXAM

BernoulliNB also uses the words that are missing: a message without "meeting" is a little more likely to be spam. MultinomialNB ignores missing words entirely.

Lesson: Which naive Bayes to use

REMEMBER

On this numeric dataset logistic regression is better at every training size. Naive Bayes stays close at 30 rows and needs no iterations at all. Ng and Jordan (2002) showed that naive Bayes reaches its best error with fewer examples, while logistic regression reaches a lower best error with more examples, so naive Bayes shines when data is tiny and its assumptions roughly hold, as with text.

Lesson: Strengths and weaknesses

2.4 Support vector machines: the widest street

The widest margin classifier, soft margins and hinge loss, kernels, tuning C and gamma, training, and SVR.

KEY IDEAS

- **Margin** Boundary $w \cdot x + b = 0$, edges ± 1, width $2/|w|$. Hard margin: min $\frac{1}{2}|w|^2$ with $y(w \cdot x + b) \geq 1$.
- **Soft margin** min $\frac{1}{2}|w|^2 + C\Sigma\max(0, 1 - y\, f(x))$. Small C: wide street, strong regularisation.
- **Support vectors** Points on or inside the margin ($\alpha > 0$). Others can move without changing the model.
- **Kernels** Linear $x \cdot z$, poly $(\gamma x \cdot z + r)^d$, RBF $\exp(-\gamma|x - z|^2)$. Kernel trick: dot products only.
- **Practice** Scale first. Grid search C and γ on a log scale. Big γ overfits. SVC up to about 10^4 to 10^5 rows, else LinearSVC or SGD.
- **Extras** SVR uses an ε tube. No built in probabilities (Platt scaling). Dual solved by SMO in libsvm.

FORMULAS

Decision rule

Exam favourite

$$\hat{y} = \text{sign}(w \cdot x + b)$$

Which side of the line the point is on.

Lesson: The widest street

Margin width

Exam favourite

$$\text{width} = 2 / |w|$$

A smaller |w| means a wider street.

Lesson: The widest street

Soft margin objective

Exam favourite

$$\min_{w,b} \tfrac{1}{2}|w|^2 + C \sum_i \max(0, 1 - y_i(w \cdot x_i + b))$$

Wide street plus a hinge penalty for every violation.

Lesson: Soft margin, hinge loss and C

Epsilon insensitive loss

Exam favourite

$$L = \max(0, |y - f(x)| - \varepsilon)$$

Errors inside the tube cost nothing.

Lesson: Support vector regression

Hard margin primal

Exam favourite

$$\min \tfrac{1}{2}|w|^2 \text{ subject to } y_i(w \cdot x_i + b) \geq 1$$

Widest street with every point outside it.

Lesson: The maths, all in one place

Soft margin primal

Exam favourite

$$\min \tfrac{1}{2}|w|^2 + C\Sigma\xi_i,\ y_i(w \cdot x_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0$$

Slack ξ is the hinge loss.

Lesson: The maths, all in one place

Dual

$$\max \Sigma\alpha_i - \tfrac{1}{2}\Sigma\alpha_i\alpha_j y_i y_j K(x_i, x_j),\ 0 \leq \alpha_i \leq C,\ \Sigma\alpha_i y_i = 0$$

Only dot products appear.

Lesson: The maths, all in one place

Kernel prediction

Exam favourite

$$f(x) = \Sigma_{i \in SV}\ \alpha_i y_i K(x_i, x) + b$$

A weighted vote of the support vectors.

Lesson: The maths, all in one place

RBF kernel

Exam favourite

$$K(x, z) = \exp(-\gamma|x - z|^2)$$

1 for identical points, near 0 for far ones.

Lesson: The maths, all in one place

TABLES

Table: kernel · k(x, z) · boundary · settings

Kernel	K(x, z)	Boundary	Settings
Linear	$x \cdot z$	Straight line or flat plane	C
Polynomial	$(\gamma\ x \cdot z + r)^d$	Curves of degree d	C, degree d, γ, r (coef0)
RBF (Gaussian)	$\exp(-\gamma\|x - z\|^2)$	Any smooth shape	C, γ
Sigmoid	$\tanh(\gamma\ x \cdot z + r)$	Like a small neural network, not always valid	C, γ, r

Lesson: Kernels: curved boundaries

Table: · svm · logistic regression · knn

	SVM	Logistic regression	KNN
Loss	Hinge (zero beyond the margin)	Log loss (never exactly zero)	None
Which points matter	Only support vectors	All points, a little each	The k nearest at prediction time
Probabilities	Not built in (Platt scaling adds them)	Built in and well calibrated	Share of votes
Curved boundaries	Kernels	Only with added features	Naturally
Large data (millions of rows)	Only the linear version	Yes	Slow at prediction
Needs scaling	Yes	Yes with a penalty	Yes

Lesson: SVMs compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
164	Maximum Gap	Medium	Sort, then find the widest gap	The widest gap between neighbouring values appears only after sorting. Sort, then compare each pair of neighbours. An SVM in one dimension puts its boundary in the middle of exactly such a gap.
2078	Two Furthest Houses With Different Colors	Easy	The answer uses an end	The furthest pair with different colours always includes the first or the last house. Scan from each end for the furthest house with a different colour.
836	Rectangle Overlap	Easy	Separated on one axis means no overlap	Two boxes do not overlap exactly when one is completely to the left, right, above or below the other. That is the separating axis idea: if a straight line splits them, they are separable.
593	Valid Square	Medium	Count squared distances	Compute the 6 squared distances between the 4 points. A square has exactly two different values, 4 equal sides and 2 equal diagonals, and none of them is zero. Squared distances are what the RBF kernel uses.
2249	Count Lattice Points Inside a Circle	Medium	Check each grid point against each circle	Look at every integer point in the box that holds all circles. A point counts when its squared distance to some centre is at most r squared. A circle boundary is what the x squared feature lets a linear SVM draw.
812	Largest Triangle Area	Easy	Try every triple with the cross product	Try every choice of three points and compute the area with the cross product formula, half the absolute value of (x2 – x1)(y3 – y1) – (x3 – x1)(y2 – y1). The sign of the same cross product tells which side of a line a point lies on, the core test of any linear classifier.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER

Three of the six points have w·x + b equal to exactly +1 or −1: those are the support vectors. The other three are further away (±1.4) and could move freely without changing the line.

Lesson: The widest street

IN THE EXAM

As C grows the street shrinks from 8.73 to 3.43 and the number of support vectors drops from 162 to 83, because fewer points are allowed inside the street. Test accuracy improves a little and then stops changing. Small C means strong regularisation. Note that C works the opposite way to the ridge penalty λ: C is roughly 1 / λ.

Lesson: Soft margin, hinge loss and C

REMEMBER The linear SVM is no better than guessing on the two rings (0.557). The degree 2 polynomial kernel scores 1.0 because $(x \cdot z)^2$ contains the features x_1^2 and x_2^2, which is exactly what describes a circle. Adding $x_1^2 + x_2^2$ by hand and using a linear SVM also scores 1.0.

Lesson: Kernels: curved boundaries

REMEMBER Small γ (0.01) gives a nearly straight boundary. γ = 1 is best here (test 0.917). At γ = 100 every point gets its own little bubble: training accuracy 0.983 but test only 0.775, classic overfitting, and the number of support vectors jumps to 170.

Lesson: Tuning C and gamma, and scaling

IN THE EXAM Scaling alone lifts accuracy from 0.912 to 0.974. The grid search picks C = 100 with γ = 0.001: a wide reach with a high penalty. Both knobs interact, so search them together, never one at a time.

Lesson: Tuning C and gamma, and scaling

REMEMBER Twenty lines of NumPy reach 97.5% accuracy, and the mean hinge loss falls from 0.257 to 0.086 over 30 passes. The learned w = (0.86, 0.87) points from class −1 towards class +1, as expected.

Lesson: How an SVM is trained

REMEMBER With ε = 0.01 nearly every point (57 of 60) is a support vector. With ε = 0.3 only 5 are, and the mean error rises from 0.077 to 0.12. Pick ε close to the noise level you are willing to ignore.

Lesson: Support vector regression

IN THE EXAM SVC(probability=True) fits an extra logistic curve to the SVM scores with internal cross validation (Platt scaling). It is slower and its probabilities can disagree with predict. Use decision_function for ranking.

Lesson: SVMs compared

STEP 3 Supervised Learning · Trees and ensembles

3.1 Decision trees: a chain of yes or no questions

Questions chosen by Gini and entropy, overfitting and pruning, regression trees, feature importance, and instability.

KEY IDEAS

- **Model** A chain of yes or no questions. Leaf = majority class (classification) or mean (regression). No scaling needed.
- **Impurity** Gini $1 - \Sigma p^2$, entropy $-\Sigma p \log_2 p$. Split minimises $(n_L/n)I(L) + (n_R/n)I(R)$. IG = drop in entropy.
- **Overfitting** Full trees memorise. Use max_depth, min_samples_leaf, max_leaf_nodes, or ccp_alpha from cost_complexity_pruning_path with CV.
- **Regression** Step function, minimises squared error, cannot extrapolate.
- **Importance** feature_importances_ (impurity, biased to many valued features). permutation_importance on test data is safer.
- **Weakness** High variance, axis parallel cuts, greedy. Fixed by ensembles: forests (3.2) and boosting (3.3).

FORMULAS

Gini impurity Exam favourite

$$G = 1 - \Sigma_k \, p_k^2$$

Chance two random picks have different classes.

Lesson: Choosing a split: Gini and entropy

Entropy Exam favourite

$$H = -\Sigma_k \, p_k \log_2 p_k$$

Bits needed to say the class.

Lesson: Choosing a split: Gini and entropy

Information gain Exam favourite

$$IG = H(\text{parent}) - \Sigma_{\text{child}} (n_{\text{child}}/n)\, H(\text{child})$$

How much a split reduces impurity.

Lesson: Choosing a split: Gini and entropy

Cost complexity Exam favourite

$$R_\alpha(T) = R(T) + \alpha\,|T|$$

Training error plus a price α per leaf. Larger α gives a smaller tree.

Lesson: Overfitting, depth and pruning

Leaf prediction (regression) Exam favourite

$$\hat{y}_{\text{leaf}} = \text{mean of } y_i \text{ in the leaf}$$

Splits minimise $\Sigma(y - \hat{y}_{\text{leaf}})^2$.

Lesson: Regression trees

Weighted impurity of a split Exam favourite

$$I_{\text{split}} = (n_L/n)\, I(L) + (n_R/n)\, I(R)$$

Pick the split with the smallest value.

Lesson: The maths, all in one place

Regression split

$$\min \Sigma_{i\in L}(y_i - \bar{y}_L)^2 + \Sigma_{i\in R}(y_i - \bar{y}_R)^2$$

Squared error around each side's mean.

Lesson: The maths, all in one place

Cost complexity Exam favourite

$$R_\alpha(T) = R(T) + \alpha|T|$$

Prune leaves until the price outweighs the gain.

Lesson: The maths, all in one place

TABLES

Table: strength · weakness

Strength	Weakness
Easy to read and explain	High variance, unstable
No scaling, handles mixed feature types	Axis parallel cuts, staircase for slanted boundaries
Captures interactions and thresholds	Cannot extrapolate beyond the training targets
Fast to train and predict	Greedy, so not the best possible tree
Handles missing values (scikit-learn 1.3 and later)	Biased impurity importance

Lesson: Why one tree is not enough

Table: · decision tree · logistic regression · knn

	Decision tree	Logistic regression	KNN
Boundary	Axis parallel boxes	One straight line	Any shape
Feature scaling	Not needed	Needed with a penalty	Needed
Interactions	Found automatically	Only if added	Automatic
Explaining a prediction	Follow the path of questions	Read the weights	Show the neighbours
Variance	High	Low	High for small k
Extrapolation	No	Yes	No

Lesson: Trees compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
104	Maximum Depth of Binary Tree	Easy	Depth is one more than the deeper child	An empty tree has depth 0. Otherwise the depth is 1 plus the larger depth of the two children. This is what max_depth limits in a decision tree.
112	Path Sum	Easy	Subtract as you go down	Walk from the root, subtracting each value from the target. At a leaf, check whether what is left equals the leaf value. A decision tree prediction is also one root to leaf path.
102	Binary Tree Level Order Traversal	Medium	A queue, one level at a time	Keep a queue of the nodes on the current level. Record their values, then replace the queue with all their children. This breadth first order is how the animation grows the tree.
700	Search in a Binary Search Tree	Easy	Go left or right by one comparison	At each node compare the value: smaller goes left, larger goes right. Each comparison is one yes or no question, exactly like a decision tree prediction.
108	Convert Sorted Array to Binary Search Tree	Easy	Split at the middle	Make the middle value the root, then build the left half and the right half the same way. Splitting into two balanced halves is what a good first question does.
257	Binary Tree Paths	Easy	Carry the path down	Walk down carrying the path so far. At a leaf, save it. Every root to leaf path of a decision tree is one readable rule, such as "petal width > 0.8 and petal width ≤ 1.75 means versicolor".

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER

Two questions about one feature already classify 96% of the iris flowers. The tree never looked at sepal length or petal length because petal width was always the best question. You can print any tree with export_text or draw it with plot_tree.

Lesson: A model made of yes or no questions

IN THE EXAM

The weather example is the classic one from Quinlan: splitting 14 days by outlook gives an information gain of 0.247 bits, because overcast days are all "play". In practice Gini and entropy choose the same split most of the time, and Gini is a little faster since it needs no logarithm.

Lesson: Choosing a split: Gini and entropy

REMEMBER

The full tree has 16 leaves and 0.906 test accuracy. Cost complexity pruning picks $\alpha = 0.0088$, keeps only 6 leaves and scores 0.918: a smaller tree that is also more accurate on new data.

Lesson: Overfitting, depth and pruning

IN THE EXAM

With depth 1 the tree knows only two numbers (0.59 and −0.7), so 0.5, 1.5 and 3.0 all get the same prediction. Depth 5 has 23 steps and follows the sine curve closely, including some of its noise. A tree can never predict outside the range of the training targets, so it cannot extrapolate a trend.

Lesson: Regression trees

REMEMBER

One feature (worst concave points) carries 73% of the impurity importance. A column of pure random noise still gets 0.009 impurity importance because the full tree used it to split a few training points, while its permutation importance on test data is about 0 (−0.005). Strobl and others (2007) documented this bias.

Lesson: Feature importance

IN THE EXAM

Five bootstrap samples of the same data give three different root features and between 12 and 19 leaves. Each tree is a reasonable model, but they disagree. That disagreement is variance, and averaging removes it.

Lesson: Why one tree is not enough

3.2 Random forests: many trees vote

Bagging and bootstrap samples, why averaging works, tuning trees and features, OOB scores, regression forests and extra trees.

KEY IDEAS

- **Idea** Average many trees: vote or mean probability for classes, mean for numbers. Reduces variance, keeps low bias.
- **Bagging** Bootstrap sample of n rows with replacement. About 63.2% distinct rows, 36.8% out of bag (OOB score).
- **Randomness** max_features at each split: sqrt(p) for classification, all for regression. Decorrelates trees.
- **Variance** $\rho\sigma^2 + (1 - \rho)\sigma^2/B$. Lower ρ beats more B.
- **Settings** n_estimators 200 to 500 (more never overfits), max_features, min_samples_leaf, oob_score=True, n_jobs=-1.
- **Variants** BaggingClassifier for any model, ExtraTrees (random cuts, no bootstrap). Compare with a linear baseline.

FORMULAS

Chance a row is left out — Exam favourite

$$P(\text{out of bag}) = (1 - 1/n)^n \to 1/e \approx 0.368$$

So about 63.2% of the rows are in each sample.

Lesson: Bootstrap samples and bagging

Variance of an average — Exam favourite

$$\text{Var} = \rho\sigma^2 + (1 - \rho)\sigma^2 / B$$

More trees shrink the second term, less correlation shrinks the first.

Lesson: Why averaging works, and why trees must differ

Bagged prediction (regression) — Exam favourite

$$\hat{f}(x) = (1/B)\ \Sigma_b\ \hat{f}_b(x)$$

Average of B trees.

Lesson: The maths, all in one place

Bagged prediction (classification) — Exam favourite

$$\hat{y} = \text{majority vote of } \hat{y}_b, \text{ or argmax of the mean class probabilities}$$

scikit-learn averages probabilities.

Lesson: The maths, all in one place

Out of bag share — Exam favourite

$$(1 - 1/n)^n \to e^{-1} \approx 0.368$$

Rows a tree never saw.

Lesson: The maths, all in one place

Variance of the forest — Exam favourite

$$\rho\sigma^2 + (1 - \rho)\sigma^2/B$$

Decorrelate the trees.

Lesson: The maths, all in one place

Default max_features

$$\text{classification } \sqrt{p}, \text{ regression } p \text{ (all)}$$

p is the number of features.

Lesson: The maths, all in one place

TABLES

Table: · one tree · random forest · gradient boosting

	One tree	Random forest	Gradient boosting
How trees are built	One deep tree	Many deep trees, in parallel, independent	Many shallow trees, in sequence, each fixing the last
Main error reduced	None	Variance	Bias (and variance with care)
Tuning needed	Depth or pruning	Little (works well with defaults)	More (learning rate, depth, number of trees)
Overfits with more trees	Not applicable	No	Yes, without early stopping
Interpretability	High	Lower (importances)	Lower (importances, SHAP)
Typical accuracy on tables	Fair	Very good	Often the best

Lesson: Forests compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
169	Majority Element	Easy	One pass vote counting	Boyer and Moore voting: keep a candidate and a count. A matching value adds 1, a different value takes 1 away, and at 0 the next value becomes the candidate. The majority always survives, just as a forest's majority survives the noisy trees.
229	Majority Element II	Medium	Count, then keep values above n/3	Count every value and keep those that appear more than n/3 times. At most two values can do that. (Boyer and Moore with two candidates does it in constant memory.)
1287	Element Appearing More Than 25% In Sorted Array	Easy	Compare with the value a quarter further on	In a sorted array a value that fills more than a quarter must equal the value n/4 positions later. Check each position against that one.
347	Top K Frequent Elements	Medium	Count, then sort by count	Count each value and return the k values with the largest counts. Sorting by count is the simple way. Buckets by count make it linear.
2404	Most Frequent Even Element	Easy	Count only the even values	Count the even values. Return the one with the largest count, the smallest value when counts tie, or −1 when there are no even values.
1394	Find Lucky Integer in an Array	Easy	A value whose count equals itself	Count every value. A lucky integer appears exactly as many times as its value. Return the largest one, or −1.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER

One tree scores 0.917. Bagging 200 trees lifts that to 0.96, and the random forest also reaches 0.96. On this dataset the extra feature randomness adds little over bagging, but on data with a few dominant features it usually helps more, as the max_features lesson shows.

Lesson: Many trees are wiser than one

IN THE EXAM

This formula is the heart of random forests and appears in The Elements of Statistical Learning, equation 15.1. Lower correlation between trees matters more than more trees.

Lesson: Why averaging works, and why trees must differ

REMEMBER

Test accuracy rises from 0.889 with one tree to 0.953 with 100 trees, then stays flat at 300. Training accuracy is 1.0 from 20 trees on, but that is not overfitting: each tree memorises its own sample and the vote still generalises.

Lesson: Number of trees, out of bag score and max_features

IN THE EXAM

With one random feature per split the trees agree least (0.864) but each is weak. With all features (None) the forest is just bagging: the trees agree more (0.919) and test accuracy drops to 0.924. sqrt, here 5 of 30 features, balances the two and scores best (0.953).

Lesson: Number of trees, out of bag score and max_features

REMEMBER

On the diabetes data (442 rows, 10 features, mostly linear effects) one tree has a negative R^2, worse than predicting the mean. The forest lifts it to 0.442 and extra trees to 0.479, yet plain linear regression still scores 0.482. Always compare with a simple baseline.

Lesson: Forests for regression, and extra trees

3.3 Boosting: small trees that fix each other

Fitting residuals, AdaBoost weights, gradient boosting for any loss, learning rate and early stopping, and the modern libraries.

KEY IDEAS

- **Idea** Add shallow trees in sequence. Each fits the current mistakes. $F = F_0 + \eta \Sigma h_m$. Reduces bias.
- **AdaBoost** $\alpha = \frac{1}{2}\ln((1-\varepsilon)/\varepsilon)$. Wrong points $\times e^{\alpha}$, right $\times e^{-\alpha}$. Exponential loss, noise sensitive.
- **Gradient** Fit trees to $-\partial L/\partial F$: $y - F$ (squared), $y - p$ (log loss), $\text{sign}(y - F)$ (absolute).
- **Tuning** Small learning rate (0.05 to 0.1), many trees with early stopping, depth 3 to 6, subsample 0.8, L2 penalty.
- **Libraries** XGBoost (2nd order, $w^* = -G/(H + \lambda)$), LightGBM (histograms, leaf wise), CatBoost (categories), sklearn HistGB.
- **Versus** Forest: parallel deep trees, variance, never overfits with more trees. Boosting: sequential shallow trees, bias, overfits.

FORMULAS

Say of a weak learner — Exam favourite

$$\alpha_m = \tfrac{1}{2}\ln((1-\varepsilon_m)/\varepsilon_m)$$

Zero when $\varepsilon = 0.5$ (guessing), large when ε is small.

Lesson: AdaBoost: reweight the hard examples

Weight update — Exam favourite

$$w_i \leftarrow w_i\, e^{-\alpha_m y_i h_m(x_i)}, \text{ then normalise}$$

Wrong points grow, right points shrink.

Lesson: AdaBoost: reweight the hard examples

Gradient boosting update — Exam favourite

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x),\ h_m \text{ fitted to } -\partial L/\partial F_{m-1}$$

Gradient descent where each step is a tree.

Lesson: Gradient boosting: descent in function space

Additive model — Exam favourite

$$F_M(x) = F_0 + \eta \sum_{m=1}^{M} h_m(x)$$

A sum of small trees.

Lesson: The maths, all in one place

Pseudo residuals — Exam favourite

$$r_{im} = -\partial L(y_i, F)/\partial F \text{ at } F = F_{m-1}(x_i)$$

Squared error gives $y - F$.

Lesson: The maths, all in one place

AdaBoost say — Exam favourite

$$\alpha = \tfrac{1}{2}\ln((1-\varepsilon)/\varepsilon)$$

From minimising the exponential loss.

Lesson: The maths, all in one place

XGBoost leaf weight

Exam favourite

$$w^* = -G / (H + \lambda)$$

G and H are the sums of first and second derivatives in the leaf.

Lesson: The maths, all in one place

XGBoost split gain

$$\text{Gain} = \tfrac{1}{2}[G_L^2/(H_L + \lambda) + G_R^2/(H_R + \lambda) - G^2/(H + \lambda)] - \gamma$$

Split only if the gain is positive.

Lesson: The maths, all in one place

TABLES

Table: loss · task · pseudo residual −∂l/∂f

Loss	Task	Pseudo residual −∂L/∂F
Squared error $\frac{1}{2}(y - F)^2$	Regression	$y - F$
Absolute error $\lvert y - F \rvert$	Robust regression	$\text{sign}(y - F)$
Log loss	Classification	$y - p$, with $p = \text{sigmoid}(F)$
Exponential e^{-yF}	Classification (AdaBoost)	$y\, e^{-yF}$

Lesson: Gradient boosting: descent in function space

Table: library · signature idea · tree growth · categorical features

Library	Signature idea	Tree growth	Categorical features
XGBoost	Second order Taylor expansion, regularised leaf weights	Level by level (depth wise) by default	Supported (enable_categorical)
LightGBM	Histogram bins, GOSS sampling, feature bundling	Leaf wise (best leaf first)	Native
CatBoost	Ordered boosting, symmetric trees	Symmetric (same split per level)	Native, target statistics
sklearn HistGB	Histogram bins, LightGBM style	Leaf wise	Native (categorical_features)

Lesson: XGBoost, LightGBM, CatBoost and HistGradientBoosting

Table: · random forest (bagging) · gradient boosting

	Random forest (bagging)	Gradient boosting
Trees built	Independently, in parallel	In sequence, each fixing the last
Tree size	Deep (low bias, high variance)	Shallow (high bias, low variance)
Reduces mostly	Variance	Bias
More trees	Never overfits	Overfits without early stopping
Tuning effort	Low	Medium to high
Sensitive to noisy labels	Less	More (it chases hard points)

Lesson: Boosting compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1480	Running Sum of 1d Array	Easy	Keep adding to a total	Each answer is the previous answer plus the next number. A boosted model is built the same way: each round adds one more tree to the running total.
1732	Find the Highest Altitude	Easy	Running total and its maximum	Start at altitude 0, add each gain, and remember the highest altitude seen, including the start.
1413	Minimum Value to Get Positive Step by Step Sum	Easy	Fix the lowest point of the running sum	Track the lowest running sum. The start value must lift that lowest point to at least 1, so the answer is 1 minus the lowest running sum, but never less than 1.
724	Find Pivot Index	Easy	Left sum against right sum	Compute the total once. Walk left to right keeping the left sum. The right sum is total minus left sum minus the current value. The first index where they match is the answer.
238	Product of Array Except Self	Medium	Prefix products times suffix products	For each position multiply the product of everything before it by the product of everything after it. Two passes, no division.
134	Gas Station	Medium	Restart after the running total goes negative	If the total gas is less than the total cost, no start works. Otherwise walk once, keeping a running tank. When it goes below zero, no station so far can be the start, so restart from the next one. Like boosting, every failure tells you where to look next.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER

Ten lines of code are a complete gradient boosting regressor. The error falls from 0.503 (just the average) to 0.019 after 30 stumps, each of which asks a single question about x.

Lesson: Small trees that fix each other's mistakes

IN THE EXAM

The labels + + − − + + − − cannot be split by any single cut. After round 1 the two points it got wrong (x = 5 and 6) carry 0.25 each instead of 0.125, so round 2 picks a stump that gets them right. Three stumps together classify all 8 points. AdaBoost is gradient boosting with the exponential loss $e^{-yF(x)}$.

Lesson: AdaBoost: reweight the hard examples

IN THE EXAM

With learning rate 1.0 the best test error comes after just 1 tree and then gets much worse (6,489 after 500 trees). With 0.1 the best is after 15 trees. With 0.01 it is after 180 trees, the lowest error of the three (3,180), and still close to it after 500. Smaller steps are slower but safer.

Lesson: Learning rate, number of trees and early stopping

REMEMBER

HistGradientBoostingClassifier was allowed 1,000 trees but stopped after 59, when the validation score had not improved for 20 rounds. Early stopping turns the number of trees from a guess into something measured.

Lesson: Learning rate, number of trees and early stopping

REMEMBER

All three reach about 97% on this small dataset, so the choice rarely matters for accuracy. It matters for speed, memory, categorical handling and deployment. HistGradientBoosting needs no extra install and is a good default inside scikit-learn.

Lesson: XGBoost, LightGBM, CatBoost and HistGradientBoosting

IN THE EXAM

On this dataset scaled logistic regression is best (0.981) and AdaBoost close behind (0.977). The breast cancer features are mostly linearly related to the label, so a linear model is hard to beat. Boosting shines on larger tables with nonlinear effects and interactions. Always include a simple baseline.

Lesson: Boosting compared

STEP 4 Supervised Learning · Evaluate and improve

4.1 Metrics: measuring a model properly

Confusion matrix, precision and recall, the accuracy trap, thresholds, ROC and PR curves, regression and probability metrics.

KEY IDEAS

- **Confusion** TP, FP (false alarm), FN (miss), TN. sklearn rows = true, columns = predicted.
- **Rates** Precision TP/(TP+FP), recall TP/(TP+FN), specificity TN/(TN+FP), F1 = 2PR/(P+R), F_β.
- **Imbalance** Accuracy misleads. Use recall, precision, balanced accuracy, PR AUC. Compare with DummyClassifier.
- **Threshold** Choose from costs with predict_proba. Lower threshold = more recall, less precision.
- **Curves** ROC: TPR vs FPR, AUC = ranking probability. PR curve for rare positives (baseline = positive share).
- **Regression and probabilities** MAE robust, RMSE punishes big errors, R^2 vs mean. Log loss and Brier for probabilities.

FORMULAS

Precision Exam favourite

$$\text{precision} = TP / (TP + FP)$$

Of the predicted positives, the share that are real.

Lesson: The confusion matrix

Recall (sensitivity, TPR) Exam favourite

$$\text{recall} = TP / (TP + FN)$$

Of the real positives, the share we found.

Lesson: The confusion matrix

F1 score Exam favourite

$$F1 = 2PR / (P + R)$$

Harmonic mean, low if either is low.

Lesson: The confusion matrix

F beta score Exam favourite

$$F_\beta = (1 + \beta^2)\, PR / (\beta^2 P + R)$$

$\beta > 1$ weights recall more, $\beta < 1$ weights precision more.

Lesson: Thresholds and the precision recall trade off

False positive rate Exam favourite

$$FPR = FP / (FP + TN) = 1 - \text{specificity}$$

Share of negatives wrongly flagged.

Lesson: ROC curves and AUC

MAE Exam favourite

$$MAE = (1/n)\, \Sigma |y_i - \hat{y}_i|$$

Average absolute error.

Lesson: Metrics for regression

RMSE Exam favourite

$$RMSE = \sqrt{((1/n)\, \Sigma (y_i - \hat{y}_i)^2)}$$

Big errors count more.

Lesson: Metrics for regression

R^2 Exam favourite

$$R^2 = 1 - SSE / SST$$

Share of variance explained. Can be negative.

Lesson: Metrics for regression

Log loss Exam favourite

$$L = -(1/n)\, \Sigma [y \ln p + (1 - y) \ln(1 - p)]$$

Confident mistakes cost a lot.

Lesson: Metrics for probabilities

Accuracy Exam favourite

$$(TP + TN) / (TP + FP + FN + TN)$$

Share correct.

Lesson: The maths, all in one place

Precision, recall Exam favourite

$$P = TP/(TP + FP),\ R = TP/(TP + FN)$$

Alarms that are real, positives found.

Lesson: The maths, all in one place

Specificity

$$TN / (TN + FP)$$

Negatives correctly rejected.

Lesson: The maths, all in one place

F_β Exam favourite

$$(1 + \beta^2)PR / (\beta^2P + R)$$

F1 when β = 1.

Lesson: The maths, all in one place

AUC as ranking Exam favourite

$$AUC = P(\text{score of random positive} > \text{score of random negative})$$

Ties count one half.

Lesson: The maths, all in one place

Balanced accuracy

$$(TPR + TNR) / 2$$

0.5 for a constant prediction.

Lesson: The maths, all in one place

TABLES

Table: situation · use · avoid

Situation	Use	Avoid
Balanced classes, equal costs	Accuracy, F1	Nothing special
Rare positives you must find	Recall, PR curve, average precision	Accuracy, ROC AUC alone
False alarms are expensive	Precision, precision at k	Recall alone
Compare rankers before picking a threshold	ROC AUC	Accuracy at 0.5
Probabilities feed a decision	Log loss, Brier, calibration plot	Accuracy
Many classes	Macro F1, per class report, confusion matrix	Micro averages alone

Lesson: Which metric when

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1512	Number of Good Pairs	Easy	Count pairs with a running tally	Each new value forms a good pair with every earlier equal value. Keep a count per value and add it before increasing it. AUC is also a count of pairs.
1365	How Many Numbers Are Smaller Than the Current Number	Easy	Sort once, read the first position	After sorting, the first position of a value equals how many values are smaller. That rank is exactly what AUC compares between positives and negatives.
1051	Height Checker	Easy	Compare with the sorted order	Sort a copy and count the positions where the original differs. It is an error count between what is and what should be, like comparing predictions with labels.
2006	Count Number of Pairs With Absolute Difference K	Easy	Look back for x minus k and x plus k	Keep counts of values seen so far. Each new value pairs with every earlier value equal to x − k or x + k.
977	Squares of a Sorted Array	Easy	Two pointers from both ends	The largest square is at one of the two ends. Compare the ends, place the bigger square at the back of the answer, and move that pointer inward. Squared errors are why RMSE punishes large misses.
1200	Minimum Absolute Difference	Easy	Sort, then look at neighbours	After sorting, the smallest absolute difference is between neighbours. Find it, then collect every neighbouring pair with that difference.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

Predicting "never fraud" scores 99.1% accuracy and catches none of the 89 frauds: recall 0 and balanced accuracy 0.5, the same as guessing. Always look at the class you care about.

Lesson: Why accuracy can lie

REMEMBER

classification_report gives precision, recall and F1 for every class. The macro average treats each class equally, the weighted average weights by the number of examples (support). For a rare class, the macro average shows problems that the weighted average hides.

Lesson: Why accuracy can lie

REMEMBER

At the default 0.5 the model misses 4 of 64 cancers. At 0.1 it misses none, at the cost of precision falling from 0.984 to 0.653, so about one in three alarms is a healthy patient sent for more tests. Here that trade is usually worth it.

Lesson: Thresholds and the precision recall trade off

IN THE EXAM

AUC equals the share of (positive, negative) pairs where the positive gets the higher score. Here 12 of 16 pairs are ordered correctly, so AUC = 0.75, exactly what scikit-learn reports. This ranking view is the Mann and Whitney U statistic.

Lesson: ROC curves and AUC

REMEMBER

As positives become rare (50%, 5%, 1%), ROC AUC stays between 0.69 and 0.79, while average precision collapses from 0.82 to 0.12. With 1% positives a random model would have average precision 0.01, so 0.12 is better than random but means most alarms are false. Saito and Rehmsmeier (2015) recommend PR curves for such data.

Lesson: Precision recall curves for rare positives

IN THE EXAM

Four predictions are off by 10 and one by 100. MAE is 28, but RMSE is 45.6, pulled up by the single big miss because it is squared. If that miss matters, RMSE tells you. If it is a rare outlier you can live with, MAE is fairer.

Lesson: Metrics for regression

REMEMBER

The careful model is right every time with modest confidence. The bold model says 99% for everything and is wrong once: its log loss is almost three times worse (1.159 against 0.434), because that one confident mistake costs −ln(0.01) = 4.6 on its own.

Lesson: Metrics for probabilities

4.2 Validation: honest estimates of performance

Three way splits, k fold cross validation, the right splitter for groups and time, leakage, and learning curves.

KEY IDEAS

- **Splits** Train fits, validation chooses, test reports once. Stratify classes. Retrain on train + validation at the end.
- **Cross validation** k folds, each validates once. cross_val_score(model, X, y, cv=5). Report mean ± std. Shuffle sorted rows.
- **Splitters** StratifiedKFold (classes), GroupKFold (groups=), TimeSeriesSplit (time), LeaveOneOut (tiny data).
- **Leakage** Preprocessing on all data, target leaks, group and time leaks, duplicates. Fix: Pipeline, right splitter, drop future features.
- **Learning curves** High bias: curves meet low, get a better model. High variance: gap closing, get more data or regularise.
- **Selection bias** Best of many CV scores is optimistic. Use a locked test set or nested CV.

FORMULAS

Cross validation estimate Exam favourite

$$CV = (1/k)\ \Sigma_{i=1}^{k}\ \text{score(model trained without fold i, fold i)}$$

Report the mean and the standard deviation.

Lesson: K fold cross validation

K fold estimate Exam favourite

$$CV_k = (1/k)\ \Sigma_i\ E_i$$

E_i is the score on fold i.

Lesson: The maths, all in one place

Standard error of the mean Exam favourite

$$SE = s / \sqrt{k}$$

Rough, since folds share training rows.

Lesson: The maths, all in one place

Fold size Exam favourite

$$n / k \text{ rows validate, } n(k-1)/k \text{ train}$$

5 fold: 80% train, 20% validate.

Lesson: The maths, all in one place

Fits for a grid search Exam favourite

$$k \times (\text{number of settings}) + 1 \text{ refit}$$

Grows fast with every new parameter.

Lesson: The maths, all in one place

TABLES

Table: leak · example · fix

Leak	Example	Fix
Preprocessing on all data	StandardScaler or SelectKBest fit before the split	Put it in a Pipeline
Target leakage	A "refund issued" column when predicting fraud	Drop features not known at prediction time
Group leakage	Same patient in train and test	GroupKFold
Time leakage	Using next month's values	TimeSeriesSplit, features from the past only
Duplicates	The same row in train and test	Remove duplicates before splitting

Lesson: Data leakage: the silent killer

Table: method · good for · watch out

Method	Good for	Watch out
Single holdout split	Big data, quick checks	Noisy on small data
K fold (k = 5 or 10)	Most tabular problems	Shuffle sorted data, k times the cost
Stratified k fold	Classification, rare classes	Default for classifiers
Group k fold	Repeated rows per person or device	Needs a groups array
Time series split	Forecasting, anything ordered in time	Fewer, smaller early training sets
Leave one out	Very small data	n fits, high variance estimate
Nested cross validation	Honest score after tuning	$k_{outer} \times k_{inner} \times$ grid fits

Lesson: Validation methods compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1013	Partition Array Into Three Parts With Equal Sum	Easy	Cut whenever a third of the total is reached	The total must split into three equal thirds. Walk through, adding to a running sum, and cut each time it reaches one third. Like making folds, you split the data into equal parts in order.
189	Rotate Array	Medium	Slice the last k to the front	Rotating right by k moves the last k values to the front. Take k modulo the length first. Rotating which fold is held out is the heart of cross validation.
1652	Defuse the Bomb	Easy	A window that wraps around	For k > 0 sum the next k values, for k < 0 the previous |k|, wrapping around the end with the modulo operator, and 0 for k = 0.
1343	Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold	Medium	Slide a fixed window	Keep the sum of the current window of k values. Slide it one step by adding the new value and removing the oldest. Compare the sum with k times the threshold.
2090	K Radius Subarray Averages	Medium	A centred sliding window	Each answer is the average of the 2k + 1 values centred on that position, or −1 if the window does not fit. Slide the window sum as in the last problem and use integer division.
1876	Substrings of Size Three with Distinct Characters	Easy	Check every window of three	Look at every window of three letters and count those whose three letters are all different.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM Three different values of k tie at 0.982 on just 114 validation rows, and the rule "take the first best" picks k = 1, the most fragile choice. A single small validation set is too noisy to choose reliably. Cross validation, next, uses every row for validation once.

Lesson: Train, validation and test sets

REMEMBER The same decision tree scores anywhere from 0.904 to 0.956 depending only on the random split. Reporting one of those numbers could flatter or punish the model by 5 points. Ten fold cross validation gives 0.917 with a standard deviation of 0.033, an honest picture of both the level and the uncertainty.

Lesson: K fold cross validation

IN THE EXAM Each patient's label is random, so there is nothing real to learn. With plain KFold the forest scores 0.88 because it recognises the patients from their other rows. GroupKFold reveals the truth: 0.535, close to guessing. This mistake appears in published medical studies.

Lesson: Stratified, grouped and time ordered splits

IN THE EXAM The data is pure noise with random labels, so the honest answer is about 0.5. Choosing the 20 best of 5,000 features on all rows, and only then cross validating, reports 0.85. Doing the same selection inside a pipeline gives 0.48. This exact error is the "wrong way" to do cross validation in The Elements of Statistical Learning, section 7.10.2.

Lesson: Data leakage: the silent killer

REMEMBER Naive Bayes stops improving at about 0.81 on validation and its training score falls to 0.87: the curves meet at a low level, a high bias model, so more digits will not help. The RBF SVM still gains from 0.961 to 0.972 as the data doubles, with training at 1.0: more data keeps paying off.

Lesson: Learning curves: will more data help?

4.3 Tuning and calibration

Validation curves, grid and random search, halving and Bayesian search, nested CV, calibration, and class imbalance.

KEY IDEAS

- **Hyperparameters** Chosen before fit (C, gamma, max_depth, learning rate). validation_curve shows under and overfitting.
- **Search** GridSearchCV (all combinations), RandomizedSearchCV (n_iter, loguniform). Keys like svc__C. best_estimator_ is refitted.
- **Smarter** Successive halving and Hyperband, Bayesian optimisation (Optuna). Log scales for C, gamma, alpha, learning rate.
- **Honest scores** best_score_ is optimistic. Nested CV: cross_val_score(GridSearchCV(...), cv=outer). Or a locked test set.
- **Calibration** CalibratedClassifierCV: sigmoid (Platt, small data) or isotonic (more data). calibration_curve, Brier, ECE. Ranking unchanged.
- **Imbalance** class_weight="balanced" ($n / (K\, n_c)$), TunedThresholdClassifierCV for the chosen metric. Judge by recall, precision, PR AUC.

FORMULAS

Successive halving

Exam favourite

$$\text{n candidates} \to n/\eta \to n/\eta^2 \ldots, \text{budget} \times \eta \text{ each round}$$

With η = 3, 81 candidates need 5 rounds to reach 1.

Lesson: Smarter searches: halving and Bayesian optimisation

Platt scaling

Exam favourite

$$p' = 1 / (1 + \exp(-(A\,s + B)))$$

s is the model score, A and B fitted on held out data.

Lesson: Calibrating probabilities

Expected calibration error

Exam favourite

$$\text{ECE} = \Sigma_b\, (n_b/n)\, |\text{mean predicted}_b - \text{observed}_b|$$

Weighted gap between the curve and the diagonal.

Lesson: Calibrating probabilities

Grid size

Exam favourite

$$|\text{grid}| = \Pi_j\, (\text{number of values for hyperparameter } j)$$

Grows exponentially.

Lesson: The maths, all in one place

Random search coverage

Exam favourite

$$P(\text{at least one try in the top q share}) = 1 - (1 - q)^n$$

60 tries: 95% chance of the top 5%.

Lesson: The maths, all in one place

Balanced class weight

Exam favourite

$$w_c = n / (K \times n_c)$$

K classes, n_c rows in class c.

Lesson: The maths, all in one place

Platt scaling

Exam favourite

$$p' = \text{sigmoid}(A\,s + B)$$

Two numbers fitted by log loss.

Lesson: The maths, all in one place

Nested CV cost

$$k_{\text{outer}} \times k_{\text{inner}} \times |\text{grid}| \text{ fits}$$

Plus refits.

Lesson: The maths, all in one place

TABLES

Table: method · idea · best when

Method	Idea	Best when
Grid search	Every combination	1 or 2 hyperparameters, few values
Random search	Random combinations	Several hyperparameters, a fixed budget
Successive halving, Hyperband	Small budget for many, big budget for the best	Expensive models where early results predict final ones
Bayesian optimisation (Optuna)	A model of score versus settings picks the next try	Each fit is expensive, up to about 20 hyperparameters

Lesson: Smarter searches: halving and Bayesian optimisation

Table: goal · tool · remember

Goal	Tool	Remember
See over and underfitting for one knob	validation_curve	Plot train and validation together
Search a small space	GridSearchCV	Cost = settings × folds
Search a big space on a budget	RandomizedSearchCV, loguniform	n_iter sets the cost
Expensive models	HalvingRandomSearchCV, Optuna	Early results must predict final ones
Honest score after tuning	Nested cross validation	Report it, then refit once
Honest probabilities	CalibratedClassifierCV	Sigmoid for small data, isotonic for large
Rare classes	class_weight, TunedThresholdClassifierCV	Pick the metric first

Lesson: Tuning tools compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
875	Koko Eating Bananas	Medium	Binary search on the setting	The eating speed is a setting you tune. Faster always works if slower works, so binary search the smallest speed that finishes within h hours. Hours at speed k is the sum of the ceilings of pile / k.
69	Sqrt(x)	Easy	Binary search for the largest square below x	Search the largest whole number whose square is at most x. Each guess halves the range, like halving the candidates in successive halving.
35	Search Insert Position	Easy	First position not smaller than the target	Binary search for the first index whose value is at least the target. That is where it is or where it would go.
162	Find Peak Element	Medium	Walk uphill with binary search	Compare the middle with its right neighbour. If the right is higher, a peak lies to the right, otherwise at or to the left. Finding the top of a validation curve is the same search.
1482	Minimum Number of Days to Make m Bouquets	Medium	Binary search the day, check greedily	If m bouquets are possible by day d, they are possible on any later day. Binary search the first such day. To check a day, count runs of k adjacent bloomed flowers.
2187	Minimum Time to Complete Trips	Medium	Binary search the time	In time t, a bus with trip time b makes t // b trips. More time never means fewer trips, so binary search the least t that reaches the total.

Lesson: LeetCode practice

TIPS AND TRAPS

REMEMBER

Training accuracy climbs to 1.0 as the tree deepens, but validation accuracy peaks at depth 2 (0.928) and then slips. To the left of the peak the tree underfits, to the right it overfits. The validation curve shows exactly where to stop.

Lesson: Parameters and hyperparameters

IN THE EXAM

The grid tried 12 settings (60 fits) and found 0.979. Random search with the same number of settings over wide log ranges found C = 10.3 and gamma = 0.015, scoring 0.981, values a coarse grid could never hit. After the search, best_estimator_ is already refitted on all the data you passed.

Lesson: Grid search and random search

IN THE EXAM

On 150 rows with 30 settings, the best inner score is 0.953 but the nested estimate is 0.933. Two points of the inner score were the luck of picking the best of 30. With a locked test set you would see the same drop.

Lesson: Nested cross validation

REMEMBER

Naive Bayes says 0.97 for its most confident bin, yet only 83% of those are positive, and says 0.03 where 24% are positive. After isotonic calibration the predicted and observed values match within about 0.04 and the Brier score falls from 0.203 to 0.168. This is the fix promised in the naive Bayes chapter.

Lesson: Calibrating probabilities

IN THE EXAM

With 5% positives the default logistic regression finds only half of them (recall 0.5). Balanced weights find 92%, but precision drops to 0.28: more catches, many more false alarms. Whether that trade is right depends on the costs, which is why the threshold should be tuned for the metric you care about.

Lesson: Imbalanced classes: weights and thresholds

REMEMBER Tuning the threshold for F1 moves it from 0.5 to 0.25 and lifts F1 a little, from 0.625 to 0.632. With a different metric, such as a cost per missed case, the best threshold would be different again.

Lesson: Imbalanced classes: weights and thresholds

STEP 1 Unsupervised Learning · Clustering

1.1 k-means: grouping points around centres

Assign and update, inertia and local minima, k-means++, choosing k, scaling, and when k-means fails.

KEY IDEAS

- **Algorithm** Choose k centres. Repeat: assign each point to the nearest centre, move each centre to its points' mean. Minimises inertia $\Sigma|x - \mu|^2$.
- **Local minima** Always converges, not always to the best. k-means++ starts ($P \propto D^2$) and n_init > 1.
- **Choosing k** Elbow of inertia vs k, silhouette (b − a)/max(a, b) in [−1, 1], plus the purpose of the clusters.
- **Practice** Scale features. KMeans(n_clusters, n_init, random_state). predict for new points. MiniBatchKMeans for big data.
- **Limits** Round, similar sized clusters. Straight boundaries. Outliers pull centres. Switch to DBSCAN, hierarchical or GMM.
- **Theory** Coordinate descent, NP hard globally, hard assignment limit of EM for spherical Gaussians.

FORMULAS

k-means objective (inertia) Exam favourite

$$J = \Sigma_i |x_i - \mu_{c(i)}|^2$$

Sum of squared distances to the assigned centre.

Lesson: Finding groups without labels

Update step Exam favourite

$$\mu_k = (1/|C_k|) \Sigma_{i \in C_k} x_i$$

The mean minimises the squared distances.

Lesson: Finding groups without labels

Silhouette of a point Exam favourite

$$s = (b - a) / \max(a, b)$$

a = mean distance inside its cluster, b = to the nearest other cluster.

Lesson: Choosing k: elbow and silhouette

Objective Exam favourite

$$J(c, \mu) = \Sigma_i |x_i - \mu_{c(i)}|^2$$

Minimised over assignments c and centres μ.

Lesson: The maths, all in one place

Assign step Exam favourite

$$c(i) = \text{argmin}_k |x_i - \mu_k|^2$$

Best c for fixed μ.

Lesson: The maths, all in one place

Update step Exam favourite

$$\mu_k = \text{mean of the points with } c(i) = k$$

Best μ for fixed c.

Lesson: The maths, all in one place

k-means++ seeding Exam favourite

$$P(\text{choose } x) = D(x)^2 / \Sigma\, D(x')^2$$

D(x) = distance to the nearest chosen centre.

Lesson: The maths, all in one place

Silhouette

$$s = (b - a)/\max(a, b)$$

Average over points.

Lesson: The maths, all in one place

TABLES

Table: assumption · breaks when · try instead

Assumption	Breaks when	Try instead
Round clusters	Long or curved shapes	DBSCAN, spectral clustering
Similar sizes and spreads	One tight and one wide cluster	Gaussian mixture
No outliers	Extreme points drag centres	k-medoids, DBSCAN
Known k	No idea how many groups	DBSCAN, hierarchical with a dendrogram
Numeric features, scaled	Categories, different units	k-modes, scaling, other distances

Lesson: When k-means fails

Table: · k-means · hierarchical · dbscan

	k-means	Hierarchical	DBSCAN	Gaussian mixture
Needs k	Yes	No (cut the tree later)	No	Yes
Cluster shape	Round	Depends on linkage	Any	Elliptical
Handles noise	No	No	Yes (marks it)	Partly
Soft assignment	No	No	No	Yes (probabilities)
Speed on large data	Very fast	Slow (n^2 memory)	Fast with an index	Moderate

Lesson: k-means compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
200	Number of Islands	Medium	Flood fill each new piece of land	Scan the grid. Each time you meet unvisited land, count an island and visit all land connected to it. Each island is a cluster of touching cells.
695	Max Area of Island	Medium	Flood fill and count the cells	Visit each island as in the last problem, counting its cells, and keep the largest count. The size of each cluster is often the first thing you check after clustering.
547	Number of Provinces	Medium	Union find over the connections	Every city starts as its own group. For each connection, merge the two groups. The number of groups left is the answer. Single linkage clustering merges groups the same way.
1029	Two City Scheduling	Medium	Sort by how much city A saves	Send everyone to city B, then move the half for whom going to A instead saves the most. Sort by cost A minus cost B and send the first half to A. It is an assignment of points to two groups at the lowest total cost.
2033	Minimum Operations to Make a Uni-Value Grid	Medium	Move everything to the median	All values must share the same remainder when divided by x, or it is impossible. Then the cheapest common value is the median, because the median minimises the total absolute distance, just as the mean minimises the squared distance in k-means.
1478	Allocate Mailboxes	Hard	One dimensional k medians by dynamic programming	Sort the houses. The best single mailbox for a run of houses sits at their median. Then dp[j][i] is the least distance for the first i houses with j mailboxes: try every last run. This is exact clustering in one dimension, something k-means can only approximate.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM The start is deliberately bad: all three centres begin in the bottom left group. k-means still converges in three rounds, but to a poor answer with inertia 40.5, because the two groups on the right share one centre. It found a local minimum, not the best clustering. Good starting centres matter.

Lesson: Finding groups without labels

REMEMBER Age holds the real groups (30 against 50) and income is pure noise, but income is measured in tens of thousands. Unscaled, the distance is almost all income, so k-means splits by income and agrees with the true groups no better than chance (ARI 0.007). After StandardScaler it finds the groups perfectly (ARI 1.0).

Lesson: k-means in scikit-learn

IN THE EXAM The data was made with 4 groups, but two of them are close together. The elbow is around k = 3 or 4 and the silhouette prefers k = 3 (0.696 against 0.634). Neither is wrong: at a coarse level there are 3 groups. The final choice depends on what the clusters are for.

Lesson: Choosing k: elbow and silhouette

REMEMBER With purely random starts, the six runs end anywhere from 329 to 1461. With k-means++ every run ends between 329 and 351, and keeping the best of 10 gives 329. That is why the default is k-means++ with several starts.

Lesson: Starting centres and k-means++

IN THE EXAM On the two moons k-means cuts straight across both shapes (ARI 0.24). On a tight blob next to a wide one, it moves the boundary into the wide blob and steals its edge points (ARI 0.61), because it always splits halfway between centres.

Lesson: When k-means fails

1.2 Hierarchical clustering and DBSCAN

Dendrograms and linkages, DBSCAN core, border and noise points, choosing eps, and HDBSCAN for different densities.

KEY IDEAS

- **Agglomerative** Start with n clusters, merge the closest two n − 1 times. Dendrogram of merge heights, cut at the biggest jump or at the k you need.
- **Linkages** Single min (chains), complete max (compact), average mean, Ward rise in squared error (round, default, Euclidean only).
- **DBSCAN** Core: at least minPts within eps (self included). Clusters grow through core points. Border joins, rest is noise −1. No k, any shape.
- **Settings** minPts about 2 × features. eps at the knee of the sorted k distance plot. Scale first.
- **HDBSCAN and OPTICS** Handle different densities. HDBSCAN needs only min_cluster_size. OPTICS reachability plot valleys.
- **Costs and theory** Agglomerative n^2 memory. Single linkage = MST. DBSCAN about n log n with an index, n^2 worst case.

FORMULAS

Ward merge cost Exam favourite

$$\Delta(A, B) = (|A||B| / (|A| + |B|)) \, |\mu_A - \mu_B|^2$$

The rise in total squared error when A and B merge. It is the k-means objective, grown one merge at a time.

Lesson: Linkage: how far apart are two clusters?

Single linkage Exam favourite

$$d(A, B) = \min_{a \in A,\, b \in B} d(a, b)$$

Closest pair.

Lesson: The maths, all in one place

Complete linkage Exam favourite

$$d(A, B) = \max_{a \in A,\, b \in B} d(a, b)$$

Farthest pair.

Lesson: The maths, all in one place

Average linkage

$$d(A, B) = (1 / |A||B|) \sum_{a,\, b} d(a, b)$$

Mean of all pairs.

Lesson: The maths, all in one place

Ward merge cost Exam favourite

$$\Delta(A, B) = |A||B| / (|A| + |B|) \cdot |\mu_A - \mu_B|^2$$

Rise in total squared error.

Lesson: The maths, all in one place

eps neighbourhood Exam favourite

$$N_\varepsilon(p) = \{\{\, q : d(p, q) \le \varepsilon \,\}\}$$

Includes p itself.

Lesson: The maths, all in one place

Core point Exam favourite

$$|N_\varepsilon(p)| \ge \text{minPts}$$

Dense enough to grow a cluster.

Lesson: The maths, all in one place

Mutual reachability (HDBSCAN)

$$d_m(a, b) = \max(\text{core}_k(a), \text{core}_k(b), d(a, b))$$

Pushes sparse points apart.

Lesson: The maths, all in one place

TABLES

Table: linkage · distance between clusters a and b · good at · weak at

Linkage	Distance between clusters A and B	Good at	Weak at
Single	min over pairs d(a, b)	Long chains, curved shapes	Chaining through noise, unequal splits
Complete	max over pairs d(a, b)	Compact groups	Sensitive to outliers, splits big groups
Average	mean over pairs d(a, b)	A balance of the two	Still prefers round shapes
Ward	rise in within cluster squared error	Round groups of similar size	Curved shapes, needs Euclidean distance

Lesson: Linkage: how far apart are two clusters?

Table: · k-means · agglomerative (ward) · agglomerative (single)

	k-means	Agglomerative (Ward)	Agglomerative (single)	DBSCAN	HDBSCAN
Needs k	Yes	No (cut later)	No (cut later)	No	No
Main settings	k	linkage, cut	cut	eps, minPts	min_cluster_size
Cluster shape	Round	Round	Any, chains	Any	Any
Marks noise	No	No	No	Yes	Yes
Different densities	Poor	Fair	Poor	Poor	Good
Cost for n points	About n × k per round	n^2 memory, up to n^3 time	n^2 (SLINK)	About n log n with an index	About n log n

Lesson: Hierarchical and density methods compared

Table: question · answer

Question	Answer
Is k-means a special case of agglomerative clustering?	No. Ward uses the same squared error objective, but merges greedily and never moves a point back. The two can give different answers.
Is single linkage related to DBSCAN?	Yes. Single linkage cut at height eps gives the connected groups of DBSCAN with minPts = 1 or 2, and HDBSCAN is single linkage on a density aware distance.
Can DBSCAN predict a new point?	Not directly. scikit-learn DBSCAN has no predict. Assign a new point to the cluster of its nearest core point within eps, or noise otherwise.

Lesson: Hierarchical and density methods compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1971	Find if Path Exists in Graph	Easy	Union find or a flood fill from the source	Join the two ends of every edge with union find, then check whether source and destination have the same root. Being in the same connected group is how DBSCAN decides that two core points share a cluster.
990	Satisfiability of Equality Equations	Medium	Merge all equalities first, then test the inequalities	Put every pair joined by == into the same group with union find. Then each != must link two different groups, or the equations contradict each other.
684	Redundant Connection	Medium	The first edge that joins two nodes already in one group	Add the edges in order with union find. An edge whose two ends already share a root closes a cycle, so it is the answer. Kruskal's minimum spanning tree, and so single linkage clustering, skip exactly these edges.
1319	Number of Operations to Make Network Connected	Medium	Count the groups, then count the spare cables	n computers need at least n − 1 cables, otherwise the answer is −1. With enough cables, joining g separate groups takes g − 1 moves.
721	Accounts Merge	Medium	Union the emails inside each account	Every email is a node. Join all emails of one account. Emails that end up with the same root belong to one person. This is clustering by shared links, just as DBSCAN joins clusters through shared core points.
827	Making A Large Island	Hard	Label every island with its size, then try each water cell	Flood fill each island and store its size under a new id. For each water cell, add 1 to the sizes of the distinct islands around it. One changed cell can bridge two groups, which is exactly how single linkage chains clusters together.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

Six points need exactly five merges. The first three merges join close pairs at distance 1 or 3. Then the jumps grow: 4 and finally 8. The big jump from 4 to 8 says the data has two natural groups at the top level, and the gap from 1 to 3 says three groups also make sense.

Lesson: Hierarchical clustering: a tree of merges

REMEMBER

SciPy writes the tree as a table Z with one row per merge. Original points are numbered 0 to 5, and the cluster made by row r gets the number 6 + r. fcluster cuts the tree, either at a height or to get a chosen number of clusters.

Lesson: Hierarchical clustering: a tree of merges

IN THE EXAM

Single linkage follows the two moons perfectly (ARI 1.0) because each moon is a chain of close points. On the blobs it fails completely: it leaves 296 points in one cluster and makes two tiny clusters from 4 stray points. Ward does the opposite: worst on the moons, best on the blobs (0.808). There is no best linkage, only a best linkage for the shape of your data.

Lesson: Linkage: how far apart are two clusters?

IN THE EXAM

Points 0 to 4 form one dense group and points 5 to 8 another, so all nine are core points and make two clusters. Points 9 and 10 have nobody within 0.8, so they are noise. Nobody asked for 2 clusters: the density alone decided it.

Lesson: DBSCAN: clusters are dense regions

REMEMBER

DBSCAN recovers both moons perfectly (ARI 1.0 on the real points) and labels 12 of the 15 random points as noise. The other 3 landed close to a moon and joined it as border points. k-means on the same data cuts straight across the moons (ARI 0.241).

Lesson: DBSCAN: clusters are dense regions

IN THE EXAM

Half the points have their 5th neighbour within 0.079 and 99% within 0.179, then the largest is 0.285. That knee suggests eps around 0.2, which gives exactly 2 clusters and no noise. At 0.1 the moons break into 8 pieces with 16 noise points, at 0.05 almost everything is noise, and from 0.4 the two moons merge into one cluster.

Lesson: Choosing eps and minPts

IN THE EXAM

With one tight, one wide and one medium group, eps 0.3 suits the tight group but shatters the wide one into pieces (8 clusters, 161 noise points). eps 1.5 suits the wide group but merges two groups (2 clusters). eps 1.0 happens to work here, but you only know that because the true labels are known. HDBSCAN finds the 3 groups (ARI 0.952) without any eps, and OPTICS also finds 3 (ARI 0.94).

Lesson: Clusters of different density: HDBSCAN and OPTICS

1.3 Gaussian mixture models and EM

Mixtures of Gaussians, the E and M steps, soft clusters, covariance types, BIC, and density based anomaly scores.

KEY IDEAS

- **Model** $p(x) = \Sigma\ \pi_k\ N(x \mid \mu_k, \Sigma_k)$. Weights add to 1. Each component has its own mean and covariance.
- **EM** E step: r_{ik} by Bayes rule. M step: $N_k = \Sigma\ r$, $\pi = N_k/n$, weighted means and covariances. Likelihood never falls, local maxima.
- **Practice** GaussianMixture(n_components, covariance_type, n_init). predict_proba for soft labels. Scale features.
- **Choosing** BIC = p ln n − 2 ln L (lower is better) for K and covariance_type. AIC penalises less.
- **Uses** Soft clustering, density estimation, anomaly scores with score_samples, sampling, BayesianGaussianMixture for automatic K.
- **Theory** ELBO and Jensen, k-means as the hard spherical limit, collapse to a point prevented by reg_covar, label switching.

FORMULAS

Mixture density — Exam favourite

$$p(x) = \Sigma_k\ \pi_k\ N(x \mid \mu_k, \sigma_k^2)$$

Weights π_k add up to 1.

Lesson: Mixtures of bell curves and the EM algorithm

Responsibility (E step) — Exam favourite

$$r_{ik} = \pi_k\ N(x_i \mid \mu_k, \sigma_k^2)\ /\ \Sigma_j\ \pi_j\ N(x_i \mid \mu_j, \sigma_j^2)$$

Bayes rule for which curve made x_i.

Lesson: Mixtures of bell curves and the EM algorithm

Log likelihood — Exam favourite

$$\ln L = \Sigma_i\ \ln\ \Sigma_k\ \pi_k\ N(x_i \mid \mu_k, \Sigma_k)$$

The sum inside the log makes direct maximisation hard.

Lesson: The maths, all in one place

E step — Exam favourite

$$r_{ik} = \pi_k\ N(x_i \mid \mu_k, \Sigma_k)\ /\ \Sigma_j\ \pi_j\ N(x_i \mid \mu_j, \Sigma_j)$$

Responsibilities.

Lesson: The maths, all in one place

M step: weights and means

Exam favourite

$$N_k = \Sigma_i\, r_{ik},\ \pi_k = N_k / n,\ \mu_k = (1/N_k)\, \Sigma_i\, r_{ik}\, x_i$$

Weighted counts and means.

Lesson: The maths, all in one place

M step: covariances

Exam favourite

$$\Sigma_k = (1/N_k)\, \Sigma_i\, r_{ik}\, (x_i - \mu_k)(x_i - \mu_k)^T$$

Weighted spread.

Lesson: The maths, all in one place

1D normal density

$$N(x \mid \mu, \sigma^2) = \exp(-(x - \mu)^2 / 2\sigma^2) / (\sigma \sqrt{(2\pi)})$$

The bell curve.

Lesson: The maths, all in one place

BIC and AIC

Exam favourite

$$BIC = p \ln n - 2 \ln L,\ AIC = 2p - 2 \ln L$$

Lower is better.

Lesson: The maths, all in one place

TABLES

Table: covariance_type · shape of each cluster · parameters per component in d dimensions

covariance_type	Shape of each cluster	Parameters per component in d dimensions
spherical	Round, own size	d means + 1 variance
diag	Ellipse along the axes	d means + d variances
tied	Same tilted ellipse for all	d means (plus one shared d(d + 1)/2)
full	Own tilted ellipse	d means + d(d + 1)/2 covariances

Lesson: Covariance types and choosing K

Table: · k-means · gaussian mixture

	k-means	Gaussian mixture
Assignment	Hard: one cluster per point	Soft: a probability for every cluster
Cluster shape	Round, same size	Ellipses of any size and tilt (full covariance)
What it optimises	Sum of squared distances (inertia)	Log likelihood of the data
Algorithm	Lloyd: assign, update	EM: E step, M step
Output	Centres and labels	Means, covariances, weights and a density
Choosing K	Elbow, silhouette	BIC, AIC, Bayesian prior
Speed	Faster	Slower, more parameters
Can fail by	Wrong shapes, local minima	Local maxima, a component collapsing onto one point

Lesson: GMM compared with k-means and the others

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1854	Maximum Population Year	Easy	Add 1 at each birth, subtract 1 at each death, and sweep	Mark +1 at each birth year and −1 at each death year, then take running totals. The year with the highest total is the densest point on the time line, the same idea as finding where a density is highest.
2848	Points That Intersect With Cars	Easy	Mark every covered point in a set	Put every integer from start to end of each car into a set and count the set. Overlaps count once, just as a mixture covers a point whichever component it came from.
2028	Find Missing Observations	Medium	Work out the missing total from the mean	The full mean fixes the total sum, so the missing n rolls must add up to mean × (m + n) minus the known sum. If that total is outside n to 6n, it is impossible. Otherwise share it out as evenly as possible. Our answer puts the extra 1s first, and LeetCode accepts any valid answer.
1109	Corporate Flight Bookings	Medium	Difference array, then running sums	Add the seats at the first flight of each booking and subtract them after the last flight. A running sum then gives every flight's total in one pass, the same sweep used to build a histogram of overlapping ranges.
1094	Car Pooling	Medium	Sweep the stops and track the load	Add passengers where they get on and remove them where they get off, then sweep the stops in order. If the load ever goes above the capacity, the answer is no.
4	Median of Two Sorted Arrays	Hard	Binary search for the split of the shorter array	Cut both arrays so that the left parts hold half of all the values and every left value is at most every right value. Binary search the cut in the shorter array. The median then comes from the values next to the cut. It finds the middle of two combined groups without merging them.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

The first guess puts both means near the middle. Round by round the curves separate, and by round 9 they sit at 154.8 and 176.2, which are exactly the means of the shorter five and the taller five. The log likelihood rises every round, from −39.65 to −34.52, and then stays put. A height of 165 cm sits between the groups, so it gets a probability of 0.575 for the taller group instead of a hard label.

Lesson: Mixtures of bell curves and the EM algorithm

IN THE EXAM

The two clusters are long and flat, one above the other. k-means splits them left and right (ARI about 0), because that gives the smaller squared distances. A GMM with one start falls into the same trap. With 5 starts it finds the higher log likelihood (−3.787 against −4.045) and the right answer (ARI 0.941). A point halfway between the clusters gets about 50% for each, and a point inside the lower cluster gets 100%.

Lesson: Gaussian mixtures in scikit-learn

REMEMBER

One cluster is tilted and one is tall. Full covariance needs the most parameters (11) but fits so much better that its BIC is the lowest by far (3835 against 4062 for tied). Spherical is the cheapest at 7 parameters but cannot follow either shape, so its BIC is the worst.

Lesson: Covariance types and choosing K

IN THE EXAM

The data was made from 4 blobs. BIC falls steeply up to 4 components (4169) and then rises again, so it picks 4. AIC also has its lowest value at 4 here, but its rise after that is much flatter, because its penalty per parameter is smaller.

Lesson: Covariance types and choosing K

REMEMBER

The threshold is the lowest 1% of the training log densities. A day with normal temperature and power use scores about −6.3. A day at 70 degrees scores −13.2 and a day with 320 units of power scores −25.5, both far below the threshold, so both are flagged. The model can also generate realistic new days.

Lesson: Densities, anomalies, new samples and automatic K

IN THE EXAM

make_blobs made 3 blobs, but two of them overlap so much that they look like one long cluster. Of the 8 components allowed, only 2 keep a weight above 1%, and the others shrink to almost nothing. The prior chose K by itself and judged two overlapping blobs to be one group, which shows that automatic K still needs a check.

Lesson: Densities, anomalies, new samples and automatic K

IN THE EXAM

k-means is the limit of EM for a GMM whose components all share one round covariance $\sigma^2 I$, with σ shrinking to 0. Then each responsibility becomes 0 or 1, the E step becomes "nearest centre", and the M step becomes "mean of the points".

Lesson: GMM compared with k-means and the others

STEP 2 Unsupervised Learning · Dimensionality reduction

2.1 Principal component analysis (PCA)

Directions of most variance, eigenvalues and the SVD, choosing k, scaling, reconstruction, and kernel PCA.

KEY IDEAS

- **Idea** Orthogonal directions of decreasing variance. Eigenvectors of the covariance matrix, eigenvalues are their variances.
- **Steps** Centre (and scale if units differ), covariance or SVD, sort by eigenvalue, keep top k, project $Z = X_c V_k$.
- **Choosing k** explained_variance_ratio_, scree elbow, PCA(0.95), or cross validation of the next model.
- **Facts** $\lambda = s^2/(n-1)$. Reconstruction error = sum of dropped eigenvalues. Max variance = min squared distance.
- **Pitfalls** Unscaled mixed units, fitting before the split, ignoring labels, sign flips, only straight directions.
- **Variants** Kernel PCA for curves, randomized and IncrementalPCA for big data, TruncatedSVD for sparse text.

FORMULAS

Covariance matrix Exam favourite

$$C = X_c^T X_c / (n-1)$$

X_c is the centred data, n rows.

Lesson: The idea: keep the directions where data spreads most

Eigen equation Exam favourite

$$C\,v = \lambda\,v$$

v is a component direction, λ the variance along it.

Lesson: The idea: keep the directions where data spreads most

Projection Exam favourite

$$Z = X_c V_k$$

V_k holds the top k eigenvectors as columns.

Lesson: The idea: keep the directions where data spreads most

First component Exam favourite

$$v_1 = \text{argmax}_{|v|=1}\, v^T C\, v$$

The unit direction with the largest variance.

Lesson: The maths, all in one place

Solution Exam favourite

$$C\,v_1 = \lambda_1 v_1,\ \text{variance} = \lambda_1$$

The top eigenvector and its eigenvalue.

Lesson: The maths, all in one place

Explained variance ratio Exam favourite

$$\lambda_j / \Sigma_i \lambda_i$$

Share of the total variance.

Lesson: The maths, all in one place

SVD route Exam favourite

$$X_c = U\,S\,V^T,\ \lambda_j = s_j^2 / (n-1)$$

Components are the rows of V^T.

Lesson: The maths, all in one place

Reconstruction

$$\hat{X} = Z\,V_k^T + \text{mean}$$

Map back to the original features.

Lesson: The maths, all in one place

Reconstruction error Exam favourite

$$(1/(n-1))\,|X_c - X_c\,V_k\,V_k^T|^2 = \Sigma_{j>k}\,\lambda_j$$

The dropped variance.

Lesson: The maths, all in one place

TABLES

Table: · pca · feature selection · lda

	PCA	Feature selection	LDA	t-SNE and UMAP	Autoencoder
Uses labels	No	Sometimes	Yes	No	No
New features	Linear mixes of all features	A subset of the originals	Linear mixes	Nonlinear map	Nonlinear mixes
Keeps	Most variance	Most useful features	Best class separation	Local neighbours	What rebuilds the input best
Transform new data	Yes	Yes	Yes	No for t-SNE, yes for UMAP	Yes
Interpretable	Partly (loadings)	Yes	Partly	No	No
Speed	Fast	Varies	Fast	Slow on big data	Needs training

Lesson: PCA compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
867	Transpose Matrix	Easy	Swap rows and columns	Row i of the answer is column i of the input. The transpose appears in every PCA formula, starting with the covariance matrix $X^T X$.
48	Rotate Image	Medium	Transpose, then reverse every row	A quarter turn clockwise is a transpose followed by flipping each row. PCA is also a rotation: it turns the axes to line up with the directions of most variance. LeetCode changes the matrix in place, and ours also returns it so it can be checked.
2482	Difference Between Ones and Zeros in Row and Column	Medium	Count once per row and once per column	Precompute the number of ones in each row and each column. Then each cell is 2 × ones in its row + 2 × ones in its column − rows − columns. Row and column totals are also the first step of centring data.
1329	Sort the Matrix Diagonally	Medium	Group cells by i minus j	Cells on the same diagonal share the value i − j. Collect each diagonal, sort it, and write it back in order. The diagonal of a covariance matrix holds the variances, and an eigen decomposition turns the whole matrix diagonal.
1314	Matrix Block Sum	Medium	Two dimensional prefix sums	Build P where P[i][j] is the sum of the top left i by j block. Any rectangle sum is then four lookups. The same trick computes many means and covariances over windows quickly.
1074	Number of Submatrices That Sum to Target	Hard	Fix two rows, then count subarrays with a hash map	For every pair of top and bottom rows, add the columns between them into one array. Then count subarrays of that array with the target sum using running sums and a dictionary. It reduces a two dimensional problem to many one dimensional ones, much as PCA reduces dimensions.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

Hours and scores rise together, so almost all the variation lies along one line. The first component keeps 99.97% of the variance, so each student can be described by one number instead of two with almost no loss. The direction (0.16, 0.99) is mostly along the score axis, because scores vary in tens of points and hours only in single hours. The scaling lesson comes back to this.

Lesson: The idea: keep the directions where data spreads most

REMEMBER

The SVD gives the same variances as the eigenvalues of the covariance matrix (133.328 and 0.038). Rebuilt from one number each, the points move only slightly onto the line: 4 hours becomes 3.7 and 6 hours becomes 5.9, while the scores stay almost exactly as they were.

Lesson: PCA in scikit-learn and the SVD

IN THE EXAM

Each digit image has 64 pixel values. The first 2 components keep only 28.5% of the variance, which is enough for a rough picture but not for detail. 21 components keep 90% and 29 keep 95%, so more than half of the 64 numbers are close to redundant.

Lesson: How many components to keep

REMEMBER

The reconstruction error falls quickly at first and then slowly. With 20 of the 64 numbers the average squared pixel error is about 2, on a pixel scale of 0 to 16, and the digits are easy to read again. The error with k components equals the sum of the variances that were dropped.

Lesson: How many components to keep

IN THE EXAM

In the wine data, proline goes up to 1680 while hue stays below 1.71. Unscaled, the first component explains 99.8% of the variance, but almost all of that is proline, which tells you nothing new. After scaling, the first component explains 36.2% and is led by flavanoids and total phenols, a real combination of related features.

Lesson: Scaling, pipelines and pitfalls

REMEMBER

With the full 64 features, logistic regression scores 0.92. With 30 components it scores 0.907 using half the features, and with 10 it still scores 0.84. PCA here trades a little accuracy for a smaller, faster model. It does not always help accuracy, because the label is not used to choose the directions.

Lesson: Scaling, pipelines and pitfalls

IN THE EXAM

Two circles, one inside the other, cannot be separated by any straight direction, so one PCA component gives 45% accuracy, no better than guessing. Kernel PCA with gamma 2 turns them into one number that separates them perfectly. With gamma 15 the kernel is too narrow and accuracy drops to 71%, so gamma needs tuning.

Lesson: Kernel PCA and PCA at scale

IN THE EXAM

A linear autoencoder with a squared error loss and k hidden units learns the same subspace as PCA with k components, though not necessarily the same axes. So PCA is the simplest autoencoder.

Lesson: PCA compared

2.2 Manifold learning: t-SNE, UMAP, MDS and Isomap

Isomap and geodesic distance, MDS from distances, t-SNE and perplexity, reading maps safely, and UMAP.

KEY IDEAS

- **Manifolds** Low dimensional surface in high dimensions. PCA flattens, manifold methods unroll. Isomap: k neighbour graph, shortest paths, classical MDS.
- **MDS** Map from a distance table. Classical MDS: $B = -\frac{1}{2} J D^{(2)} J$, top eigenvectors. Metric MDS minimises stress.
- **t-SNE** Gaussian $p_{j|i}$ with perplexity 2^H, Student t q_{ij}, minimise KL(P | Q), early exaggeration, init="pca".
- **Reading maps** Trust neighbours. Not sizes, not gaps. Try several perplexities and seeds. No transform. Not for model input.
- **UMAP** Fuzzy neighbour graph, cross entropy, n_neighbors and min_dist, fast, transforms new points, umap-learn package.
- **Costs** Exact t-SNE $O(n^2)$, Barnes-Hut $O(n \log n)$. Floyd and Warshall $O(n^3)$. PCA to 50 first speeds things up.

FORMULAS

Stress Exam favourite

$$\text{stress} = \Sigma_{i<j} (d_{ij} - |z_i - z_j|)^2$$

Given distances against map distances.

Lesson: Multidimensional scaling: a map from distances

High dimensional affinity Exam favourite

$$p_{j|i} = \exp(-|x_i - x_j|^2 / 2\sigma_i^2) / \Sigma_{k \neq i} \exp(-|x_i - x_k|^2 / 2\sigma_i^2)$$

$p_{ij} = (p_{j|i} + p_{i|j}) / 2n$.

Lesson: t-SNE: maps that keep neighbours together

Map affinity (Student t) Exam favourite

$$q_{ij} = (1 + |y_i - y_j|^2)^{-1} / \Sigma_{k \neq l} (1 + |y_k - y_l|^2)^{-1}$$

Heavy tails let far points sit far apart.

Lesson: t-SNE: maps that keep neighbours together

Perplexity Exam favourite

$$\text{Perp}(P_i) = 2^{H(P_i)}$$

H is the entropy in bits.

Lesson: t-SNE: maps that keep neighbours together

Classical MDS

Exam favourite

$$B = -\tfrac{1}{2}\, J\, D^{(2)}\, J,\ J = I - (1/n)\, 1\, 1^{T},\ Z = V_k\, \Lambda_k^{1/2}$$

Double centre the squared distances, then take the top eigenvectors.

Lesson: The maths, all in one place

Isomap

Exam favourite

$$D_{geo} = \text{shortest paths in the k neighbour graph, then classical MDS on } D_{geo}$$

Geodesic distances.

Lesson: The maths, all in one place

t-SNE cost

Exam favourite

$$KL(P \mid Q) = \Sigma_{i \neq j}\, p_{ij}\, \log(p_{ij} / q_{ij})$$

Big penalty for neighbours placed far apart.

Lesson: The maths, all in one place

t-SNE gradient

Exam favourite

$$\partial C / \partial y_i = 4\, \Sigma_j\, (p_{ij} - q_{ij})(y_i - y_j)(1 + |y_i - y_j|^2)^{-1}$$

Attraction minus repulsion.

Lesson: The maths, all in one place

Stress (metric MDS)

$$\Sigma_{i<j}\, (d_{ij} - |z_i - z_j|)^2$$

Lower is better.

Lesson: The maths, all in one place

TABLES

Table: you see in the map · can you trust it?

You see in the map	Can you trust it?
Points close together	Usually yes: they are close in the original space
Separate islands	Often, but check at a few perplexities and seeds
Big or small clusters	No: t-SNE evens out densities
Distance between clusters	Only roughly, if at all
Clusters in pure noise	Possible at low perplexity, so check

Lesson: Reading t-SNE maps without being fooled

Table: method · keeps · new points · scales to

Method	Keeps	New points	Scales to	Use it for
PCA	Global variance, linear	Yes	Millions	Compression, first look, preprocessing
Classical MDS	All pairwise distances	No	Thousands	Maps from a distance table
Isomap	Geodesic distances	Yes (approximate)	Thousands	Unrolling one smooth manifold
LLE, spectral	Local neighbourhoods	Partly	Thousands	Smooth manifolds, clustering prep
t-SNE	Local neighbours	No	Tens of thousands (Barnes-Hut)	Pictures of clusters
UMAP	Local and some global	Yes	Millions	Pictures of large data, some preprocessing

Lesson: UMAP and choosing a method

Table: · pca · isomap · t-sne

	PCA	Isomap	t-SNE	UMAP
Linear	Yes	No	No	No
What it keeps	Variance	Path distances	Neighbours	Neighbour graph
Deterministic	Yes	Yes	No (random start)	No (random start)
Main setting	n_components	n_neighbors	perplexity	n_neighbors, min_dist
Distances in output meaningful	Yes	Roughly	Only locally	Only locally
Good as model input	Yes	Sometimes	No	Sometimes

Lesson: Manifold learning compared with PCA

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1791	Find Center of Star Graph	Easy	The centre appears in the first two edges	Every edge touches the centre, so the node shared by the first two edges is the answer. Reading structure from a list of edges is the first step of every neighbour graph method.
743	Network Delay Time	Medium	Dijkstra from the source, answer is the largest distance	Run Dijkstra's algorithm with a heap from node k. If some node is never reached the answer is −1, otherwise it is the largest shortest path distance. Isomap runs this same search from every point.
1334	Find the City With the Smallest Number of Neighbors at a Threshold Distance	Medium	All pairs shortest paths with Floyd and Warshall	Fill a distance table, then for every middle city m relax d[i][j] with d[i][m] + d[m][j]. Count reachable cities within the threshold for each city and keep the smallest count, breaking ties by the larger index. Isomap needs exactly this table of all pairs distances.
1514	Path with Maximum Probability	Medium	Dijkstra that multiplies probabilities and keeps the largest	Use a max heap on probability. Moving along an edge multiplies the probability, which can only shrink it, so the first time you pop the end node you have the best path. t-SNE also works with neighbour probabilities on a graph.
1631	Path With Minimum Effort	Medium	Dijkstra where a path costs its biggest step	The effort of a path is its largest height difference. Run Dijkstra with that cost: moving to a neighbour costs max(current effort, height difference). Paths through a surface that avoid big jumps are what manifold methods look for.
882	Reachable Nodes In Subdivided Graph	Hard	Dijkstra on the original nodes, then count partial edges	An edge with cnt new nodes has length cnt + 1. Run Dijkstra on the original nodes. Count every original node within maxMoves. For each edge, the new nodes reached from each end are the moves left there, and together they cannot exceed cnt.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

The swiss roll is a flat sheet rolled up in 3D. PCA squashes it flat and mixes the layers, so its best axis follows the position along the roll with a rank correlation of only 0.19. Isomap unrolls it almost perfectly (1.0), spectral embedding nearly so (0.998), and LLE reaches 0.916. All three need a sensible number of neighbours.

Lesson: Manifolds, and distance along the data

REMEMBER

From the 10 distances alone, MDS rebuilds the five towns so well that no distance is off by even 0.001, and A to E is 6.32 both ways. The rebuilt map may be turned or flipped compared with the true one, since nothing in a distance table fixes north.

Lesson: Multidimensional scaling: a map from distances

IN THE EXAM

The binary search finds the width 0.607 that gives a perplexity of exactly 3. The three nearest points share almost all the probability (0.437, 0.333, 0.222) and the far points get none. A larger perplexity would widen the Gaussian and spread the probability over more neighbours.

Lesson: t-SNE: maps that keep neighbours together

REMEMBER

In a 2D PCA map of 600 digit images, the 5 nearest neighbours of a point share its digit only 57.7% of the time. In the t-SNE map they do 94.2% of the time, and the trustworthiness (how many map neighbours were real neighbours) is 0.995 against 0.834. t-SNE keeps local structure far better.

Lesson: t-SNE: maps that keep neighbours together

REMEMBER

On these 400 digits, neighbour accuracy stays between 0.985 and 0.992 for every perplexity from 2 to 100, and trustworthiness between 0.981 and 0.992. The local structure is robust. What changes is the look of the map: the size, shape and spacing of the clusters.

Lesson: Reading t-SNE maps without being fooled

IN THE EXAM

In 10 dimensions the wide cluster is 10 times as spread as the tight one, but in the t-SNE map both have spread 4.3. The far cluster is 10 times farther than the wide one in 10D, but only about twice as far in the map. So the map tells you which points belong together, not how big groups are or how far apart.

Lesson: Reading t-SNE maps without being fooled

REMEMBER

A common pipeline is PCA to 50 dimensions first and then t-SNE or UMAP to 2. PCA removes noise and makes the neighbour search much faster, and the picture usually looks the same or better.

Lesson: Manifold learning compared with PCA

STEP 3 Unsupervised Learning · Patterns, anomalies and recommendations

3.1 Anomaly detection

Robust scores, Isolation Forest, LOF, one class SVM and elliptic envelope, and evaluation for rare events.

KEY IDEAS

- **Robust rules** Median and MAD score (x − med)/(1.4826 MAD), flag above 3.5. IQR fences Q1 − 1.5 IQR, Q3 + 1.5 IQR. z scores suffer masking.
- **Isolation Forest** Random feature, random cut. Anomalies isolate fast. $s = 2^{-E[h]/c(n)}$, near 1 anomalous. 256 row samples. contamination sets the cut off.
- **LOF** lrd = 1 / mean reachability distance. LOF = mean neighbour lrd / own lrd. Handles varying density. novelty=True for new points.
- **Other models** One class SVM (nu, gamma), elliptic envelope (one robust Gaussian, Mahalanobis), GMM log density. Match the shape of normal.
- **Evaluation** PR AUC, precision at k, recall at a budget. Not accuracy. Threshold from costs. Refit for drift.
- **Modes** Outlier detection on dirty data, novelty detection on clean data, supervised models when labelled anomalies are plentiful.

FORMULAS

Robust score Exam favourite

$$(x - \text{median}) / (1.4826 \times \text{MAD})$$

MAD = median of |x − median|. Flag above 3.5.

Lesson: What makes a point an anomaly

Isolation score Exam favourite

$$s(x, n) = 2^{-E[h(x)] / c(n)},\ c(n) = 2(\ln(n - 1) + 0.5772) - 2(n - 1)/n$$

h = path length, c(n) = average path length in a random tree.

Lesson: Isolation Forest

Reachability distance

$$\text{reach}_k(p, o) = \max(\text{k-distance}(o), d(p, o))$$

Smooths tiny distances inside dense groups.

Lesson: Local outlier factor: anomalies relative to their neighbourhood

Local reachability density Exam favourite

$$\text{lrd}(p) = 1 / \text{mean}_{o \in N_k(p)} \text{reach}_k(p, o)$$

How crowded p's neighbourhood is.

Lesson: Local outlier factor: anomalies relative to their neighbourhood

LOF Exam favourite

$$\text{LOF}(p) = \text{mean}_{o \in N_k(p)} \text{lrd}(o) / \text{lrd}(p)$$

About 1 normal, above 1 outlier.

Lesson: Local outlier factor: anomalies relative to their neighbourhood

z score

$$z = (x - \mu) / \sigma$$

Sensitive to the outliers it seeks.

Lesson: The maths, all in one place

Robust score Exam favourite

$$(x - \text{median}) / (1.4826\ \text{MAD})$$

1.4826 makes MAD match σ for normal data.

Lesson: The maths, all in one place

Mahalanobis distance Exam favourite

$$d_M(x) = \sqrt{((x - \mu)^T \Sigma^{-1} (x - \mu))}$$

Distance measured in units of the spread.

Lesson: The maths, all in one place

Isolation score Exam favourite

$$s = 2^{-E[h]/c(n)}$$

1 anomalous, 0.5 nothing special, lower normal.

Lesson: The maths, all in one place

LOF Exam favourite

$$\text{mean lrd(neighbours)} / \text{lrd}(p)$$

Relative density.

Lesson: The maths, all in one place

Precision at k

$$(\text{true anomalies in the top k}) / k$$

Matches the review budget.

Lesson: The maths, all in one place

TABLES

Table: kind of anomaly · example · typical method

Kind of anomaly	Example	Typical method
Point	One payment of 50,000 on a card that usually spends 50	Robust scores, Isolation Forest
Contextual	30 degrees is normal in July but odd in January	Model the context, then score the residual
Collective	A run of normal looking packets that together form an attack	Sequence and time series models

Lesson: What makes a point an anomaly

Table: · robust stats · isolation forest · lof

	Robust stats	Isolation Forest	LOF	One class SVM	Elliptic envelope	GMM
Features	One at a time	Many	Many	Many	Many	Many
Shape of normal	One centre	Any	Any, varying density	Any (kernel)	One ellipse	Several ellipses
Speed on big data	Very fast	Fast	Slow (neighbours)	Slow (kernel)	Fast	Moderate
Main setting	Threshold	contamination	n_neighbors	nu, gamma	contamination	n_components
New points	Yes	Yes	novelty=True	Yes	Yes	Yes

Lesson: Anomaly detection methods compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
136	Single Number	Easy	XOR cancels every pair	XOR all the numbers. Each pair cancels to 0, so the number that appears once is what is left. The odd one out stands out because everything else matches.
268	Missing Number	Easy	Compare the expected sum with the actual sum	The numbers 0 to n add up to n(n + 1)/2. Subtract the actual sum and the gap is the missing number. Comparing what you expect with what you see is the core of anomaly detection.
219	Contains Duplicate II	Easy	Remember where each value was last seen	Keep the last index of each value in a dictionary. If a value comes back within k positions, answer true. It is the same pattern as flagging a repeated event inside a time window.
287	Find the Duplicate Number	Medium	Treat values as next pointers and find the cycle	Read nums as a linked list where i points to nums[i]. The duplicate value is where a cycle starts. Floyd's tortoise and hare finds it in O(1) extra memory.
442	Find All Duplicates in an Array	Medium	Mark seen values by flipping signs in place	Values are 1 to n, so value v can mark position v − 1 by making it negative. Meeting an already negative mark means v is a duplicate. Ours sorts the answer so it can be checked, and LeetCode accepts any order.
41	First Missing Positive	Hard	Place each value at its own index, then scan	Swap each value v in 1 to n into position v − 1. Afterwards the first position i that does not hold i + 1 gives the answer. It runs in O(n) time with O(1) extra memory.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

Two typing mistakes (900 and 950 orders on days that usually see about 50) drag the mean to 185 and the standard deviation to 316. Their z scores are only 2.26 and 2.42, so a $|z| > 3$ rule misses both. This is called **masking**. The median (51) and MAD (4.45) barely move, so the robust scores are about 190 and 200 and both days are flagged, as they are by the IQR fences 41.5 to 61.5.

Lesson: What makes a point an anomaly

IN THE EXAM

With 100 values, the far value 95 is isolated after 1.39 random cuts on average, while a normal value needs about 10. Turning path length into a score with $s = 2^{-h/c(n)}$ gives 0.89 for the far value and 0.44 for the normal one. Scores near 1 are anomalies, and scores below 0.5 are normal.

Lesson: Isolation Forest

REMEMBER

The forest separates the planted points very well: ROC AUC 0.998. With contamination set to the true share it flags 14 points, of which 11 are planted anomalies (precision and recall 0.786). The misses are planted points that happened to land close to the normal cloud.

Lesson: Isolation Forest

IN THE EXAM

A point at distance 1.2 from a group of spread 0.2 is very unusual locally, but it is still much closer to the other points than the edges of the loose group are. LOF ranks it as the single most unusual of 401 points, with an LOF of 4.77 against a typical 1.04. Isolation Forest, which looks at global isolation, ranks it only 13th.

Lesson: Local outlier factor: anomalies relative to their neighbourhood

IN THE EXAM

The normal data is two round blobs. The Gaussian mixture, one class SVM and LOF all score almost perfectly, because they can model two groups. The elliptic envelope assumes one ellipse around everything, so the empty space between the blobs looks normal to it and its PR AUC drops to 0.92. Isolation Forest is close behind at 0.91. Match the method to the shape of normal.

Lesson: One class SVM, robust covariance and a fair comparison

IN THE EXAM

ROC AUC is 0.957, which sounds excellent. But PR AUC is 0.496, and of the top 100 alerts only half are real. With a budget of 500 alerts the team catches 81% of anomalies but 84% of its work is false alarms. PR AUC and precision at k tell that story, and ROC AUC hides it.

Lesson: Evaluating detectors when anomalies are rare

REMEMBER

Supervised classification beats all of these when you have enough labelled anomalies of every kind you care about. Anomaly detection is for when you do not, or when new kinds of anomalies keep appearing.

Lesson: Anomaly detection methods compared

3.2 Association rules and market basket analysis

Support, confidence and lift, Apriori and pruning, rule tables, FP-Growth, and reading rules safely.

KEY IDEAS

- **Measures** support P(A ∩ B), confidence P(B | A), lift P(A ∩ B)/P(A)P(B). Lift 1 independent, below 1 negative.
- **Trap** High confidence with a very common B can still mean lift below 1. Always check lift.
- **Apriori** Subsets of frequent sets are frequent. Count level by level, join, prune candidates with infrequent subsets.
- **Rules** Each frequent k itemset gives $2^k - 2$ rules. Filter by support, confidence, lift. Leverage and conviction.
- **Scale** FP-Growth prefix tree, no candidates, two scans. PCY and SON for massive data. Pair counts via X^TX.
- **Caution** Many tests give chance patterns. Rules are correlations, not causes.

FORMULAS

Support Exam favourite

$$\text{supp}(A \Rightarrow B) = P(A \cap B)$$

Share of baskets with both.

Lesson: Items that go together: support, confidence and lift

Confidence Exam favourite

$$\text{conf}(A \Rightarrow B) = P(B \mid A) = \text{supp}(A \cap B) / \text{supp}(A)$$

Not symmetric.

Lesson: Items that go together: support, confidence and lift

Lift Exam favourite

$$\text{lift} = \text{conf}(A \Rightarrow B) / \text{supp}(B) = P(A \cap B) / (P(A) P(B))$$

Symmetric. 1 means independent.

Lesson: Items that go together: support, confidence and lift

Support Exam favourite

$$\text{supp}(X) = |\{\{t : X \subseteq t\}\}| / n$$

Share of transactions t containing itemset X.

Lesson: The maths, all in one place

Confidence Exam favourite

$$\text{conf}(A \Rightarrow B) = \text{supp}(A \cup B) / \text{supp}(A)$$

A ∪ B here means both itemsets in the basket.

Lesson: The maths, all in one place

Lift Exam favourite

$$\text{lift}(A \Rightarrow B) = \text{supp}(A \cup B) / (\text{supp}(A)\ \text{supp}(B))$$

1 = independent.

Lesson: The maths, all in one place

Leverage

$$\text{supp}(A \cup B) - \text{supp}(A)\ \text{supp}(B)$$

0 = independent.

Lesson: The maths, all in one place

Conviction

$$(1 - \text{supp}(B)) / (1 - \text{conf}(A \Rightarrow B))$$

Infinite for a rule that is never wrong.

Lesson: The maths, all in one place

Apriori property Exam favourite

$$X \subseteq Y \Rightarrow \text{supp}(X) \geq \text{supp}(Y)$$

Support never rises when a set grows.

Lesson: The maths, all in one place

TABLES

Table: measure · formula · reads as

Measure	Formula	Reads as
Support	P(A ∩ B)	How often the rule applies
Confidence	P(B | A)	How often it is right when it applies
Lift	P(A ∩ B) / P(A)P(B)	Times more likely than chance
Leverage	P(A ∩ B) − P(A)P(B)	Extra share of baskets beyond chance
Conviction	(1 − P(B)) / (1 − P(B | A))	How much more often the rule would be wrong if A and B were independent

Lesson: From itemsets to a table of rules

Table: · association rules · clustering · collaborative filtering

	Association rules	Clustering	Collaborative filtering	Classification
Labels needed	No	No	No (uses ratings or clicks)	Yes
Finds	Items that co-occur	Groups of similar records	Items a person may like	A label for each record
Output	If A then B rules	Cluster for each record	Ranked list per person	Predictions
Personal	No, the same for everyone	No	Yes	Per record
Readable	Very	Fairly	Less	Depends on the model

Lesson: Association rules compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1	Two Sum	Easy	Remember each value you have seen	Walk the list and keep a dictionary from value to index. For each number, check whether target minus it has been seen. Finding pairs that belong together quickly is the heart of pair counting.
349	Intersection of Two Arrays	Easy	Set intersection	Turn both lists into sets and intersect them. Support counting asks the same question many times: is every item of this set in that basket. Ours sorts the answer so it can be checked.
78	Subsets	Medium	Each item is in or out	Start with the empty set and, for every item, add it to every subset found so far. n items give 2^n subsets, which is exactly why Apriori avoids listing them all. Ours sorts the subsets so the answer can be checked.
77	Combinations	Medium	Backtrack, choosing numbers in increasing order	Build each combination by adding numbers larger than the last one, and record it once it has k numbers. Generating all size k sets in order is how candidate itemsets are listed.
49	Group Anagrams	Medium	Use the sorted letters as a key	Words with the same letters share the same sorted string. Group by that key in a dictionary. Turning a basket into a sorted key is also how identical itemsets are counted together. Ours sorts the groups so the answer can be checked.
1178	Number of Valid Words for Each Puzzle	Hard	Count word letter sets as bitmasks, then enumerate each puzzle's subsets	Turn each word into a bitmask of its letters and count masks. A puzzle has only 7 letters, so list all subsets of its letters that contain its first letter (at most 64) and add up their counts. Enumerating subsets of a small itemset is exactly how rules are generated from a frequent itemset.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

Bread is in 7 of 10 baskets, butter in 5, and both in 5. So every basket with butter also has bread. Confidence(bread ⇒ butter) = 0.5 / 0.7 = 0.714, and lift = 0.714 / 0.5 = 1.43: buying bread makes butter 43% more likely than in a random basket.

Lesson: Items that go together: support, confidence and lift

IN THE EXAM

72% of cereal buyers also buy milk, which sounds like a strong rule. But 80% of all shoppers buy milk. Cereal buyers are slightly less likely to buy milk than everyone else, so the lift is 0.9. Confidence alone can mislead whenever the right hand side is very common.

Lesson: Items that go together: support, confidence and lift

IN THE EXAM

With a minimum of 3 baskets, 5 of the 6 items are frequent (biscuits is in only 2). The 5 frequent items give 10 candidate pairs, and 4 pairs survive. Joining those pairs gives triples such as bread, butter and jam, but butter and jam appear together only twice, so that triple is skipped without counting. Only bread, butter and milk is counted, and it is frequent.

Lesson: Apriori: finding frequent itemsets fast

REMEMBER

For pairs, one matrix product does all the counting: with X the basket by item table of 0s and 1s, $X^T X$ holds the number of baskets containing each pair of items, with single item counts on the diagonal. Bread and butter appear together in 5 baskets, a support of 0.5.

Lesson: Apriori: finding frequent itemsets fast

IN THE EXAM

The three habits built into the simulation come out on top: bread with butter (lift 1.97), tea with biscuits (1.82) and coffee with sugar (1.79). Each pair appears in both directions with the same lift but different confidence, because lift is symmetric and confidence is not. 38 of the 44 rules with support of at least 5% have lift below 1, so support alone is a poor filter.

Lesson: From itemsets to a table of rules

REMEMBER

A rule found in the data is a pattern, not a cause. "Diapers and beer" type stories need a test, such as an experiment with shelf placement, before acting on them.

Lesson: FP-Growth, big data and pitfalls

3.3 Recommender systems

Content based and collaborative filtering, matrix factorisation by SGD and ALS, ranking metrics, cold start and production systems.

KEY IDEAS

- **Data** A huge sparse user by item matrix. Explicit ratings or implicit clicks. Missing is unknown, not a dislike.
- **Content based** Item features, taste profile, cosine similarity. Works for new items, but only finds more of the same.
- **Neighbours** Centre on user means, centred cosine (Pearson) on co rated items, top k, $\hat{r} = \bar{r}_u + \Sigma \text{ sim } (r - \bar{r}_v) / \Sigma |\text{sim}|$.
- **Factorisation** $\hat{r} = \mu + b_u + b_i + p_u \cdot q_i$. Loss on known cells plus λ penalty. SGD or ALS. Tune k and λ on held out data.
- **Evaluation** Precision@k, recall@k, NDCG@k, MRR on a time based split. RMSE ignores order.
- **Practice** Cold start fallbacks, shrunk popularity, two stage retrieval and ranking, feedback loops and popularity bias.

FORMULAS

Cosine similarity Exam favourite

$$\cos(a, b) = a \cdot b / (\|a\| \, \|b\|)$$

1 for the same direction, 0 for no shared features.

Lesson: What a recommender does, and content based filtering

Taste profile

$$\text{profile}(u) = \text{mean of } x_i \text{ over items i liked by u}$$

Ratings can be used as weights.

Lesson: What a recommender does, and content based filtering

Centred cosine (Pearson) Exam favourite

$$\text{sim}(u, v) = \Sigma_i (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v) / \sqrt{(\Sigma_i (r_{ui} - \bar{r}_u)^2 \, \Sigma_i (r_{vi} - \bar{r}_v)^2)}$$

Sums over items both users rated.

Lesson: The maths, all in one place

User based prediction Exam favourite

$$\hat{r}_{ui} = \bar{r}_u + \Sigma_v \text{ sim}(u, v)(r_{vi} - \bar{r}_v) / \Sigma_v |\text{sim}(u, v)|$$

v runs over the k nearest users who rated i.

Lesson: The maths, all in one place

Matrix factorisation Exam favourite

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u \cdot q_i$$

k hidden factors per user and item.

Lesson: The maths, all in one place

Training loss Exam favourite

$$\min \Sigma_{(u,i) \text{ known}} (r_{ui} - \hat{r}_{ui})^2 + \lambda(\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2)$$

Only known cells count.

Lesson: The maths, all in one place

SGD step

$$e = r - \hat{r},\ p_u \leftarrow p_u + \eta(e\ q_i - \lambda\ p_u),\ q_i \leftarrow q_i + \eta(e\ p_u - \lambda\ q_i)$$

Use the old p_u when updating q_i.

Lesson: The maths, all in one place

ALS step

$$p_u = (Q_u^T Q_u + \lambda I)^{-1} Q_u^T r_u$$

Q_u holds the factors of items u rated. A ridge regression per user.

Lesson: The maths, all in one place

NDCG@k Exam favourite

$$DCG@k = \Sigma_{i=1}^{k} rel_i / \log_2(i + 1),\ NDCG@k = DCG@k / IDCG@k$$

Between 0 and 1.

Lesson: The maths, all in one place

Shrunk average

$$score = (n\ \bar{r} + m\ C) / (n + m)$$

m acts as m extra ratings at the overall mean C.

Lesson: The maths, all in one place

TABLES

Table: kind of feedback · examples · what it tells us

Kind of feedback	Examples	What it tells us
Explicit	Star ratings, thumbs up or down	Clear likes and dislikes, but few people give them
Implicit	Clicks, plays, purchases, time watched	Plenty of data, but a missing click is not a dislike

Lesson: What a recommender does, and content based filtering

Table: · user based · item based

	User based	Item based
Similarity between	Rows (users)	Columns (items)
Stable over time	No, tastes and users change fast	Yes, item links change slowly, so they can be computed offline
Typical scale	Many more users than items makes it slow	Used by Amazon for item to item recommendations
Explains itself as	"People like you liked this"	"Because you liked X"

Lesson: Collaborative filtering: people like you and items like this

Table: metric · formula · cares about order

Metric	Formula	Cares about order
Precision@k	hits in top k / k	No
Recall@k	hits in top k / number relevant	No
NDCG@k	DCG@k / ideal DCG@k, with DCG = $\Sigma\ rel_i / \log_2(i + 1)$	Yes
MRR	mean of 1 / rank of first hit	Yes, first hit only
RMSE	$\sqrt{}$(mean of $(r - \hat{r})^2$)	No, judges ratings not lists

Lesson: Measuring recommendations and starting cold

Table: · popularity · content based · neighbour collaborative filtering

	Popularity	Content based	Neighbour collaborative filtering	Matrix factorisation
Data needed	Counts only	Item features	Ratings or clicks	Ratings or clicks
Personal	No	Yes	Yes	Yes
New item	Needs some ratings	Works at once	Fails	Fails
New user	Works at once	Needs a few likes	Fails	Fails
Finds surprising items	No	Rarely	Yes	Yes
Explains itself	Very easily	Easily	Fairly	Hardly

Lesson: Recommendation methods compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
506	Relative Ranks	Easy	Sort indices by score, best first	Sort the positions by score from high to low, then hand out "Gold Medal", "Silver Medal", "Bronze Medal" and numbers in that order. Turning scores into ranks is the last step of every recommender.
1636	Sort Array by Increasing Frequency	Easy	Sort by a key made of the count and the negative value	Count each value, then sort by (count, minus value). A sort key with a tie breaker is how popularity lists are ordered when two items have the same score.
1311	Get Watched Videos by Your Friends	Medium	Breadth first search to the given level, then count	Walk the friend graph level by level from the person, keeping a visited set, and stop at the asked level. Count the videos those friends watched and sort by count, then by name. This is the simplest social recommender: suggest what your friends watched.
1268	Search Suggestions System	Medium	Sort the products, then binary search each prefix	After sorting, all products with a given prefix sit next to each other. For each prefix typed so far, find the first match with bisect and take up to three products that still start with the prefix. Suggestions while typing are a recommender with a very simple score.
1366	Rank Teams by Votes	Medium	Count votes per position and sort by the whole count list	For each team, count how many voters put it in each position. Sort teams by those counts, highest first position by position, and by name to break full ties. Combining many people's rankings into one is the core of popularity and group recommendations.
2102	Sequentially Ordinal Rank Tracker	Hard	Two heaps: the items already shown, and the rest	Keep the i best items seen so far in a heap whose top is the worst of them, and every other item in a heap whose top is the best. To add, push into the first heap and move its worst item to the second. To get, move the best of the second heap into the first and return it. Each call costs O(log n), the pattern behind serving a ranked feed that keeps changing.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

The person liked one action sci fi film and one romantic sci fi film, so the profile is 1 for sci fi and 0.5 for romance and action. Star Siege shares sci fi and action and scores 0.866. Every other film shares no feature with the profile and scores 0. This shows the main weakness of content based filtering: it only finds more of the same, and it can never suggest a comedy to someone who has not watched one yet.

Lesson: What a recommender does, and content based filtering

IN THE EXAM

User 1 agrees with user 0 (similarity 0.776) and rated the film 1, well below user 1's mean of 2.75. User 2 disagrees with user 0 (similarity −0.653) and loved the film. Both signals point the same way, so the guess for user 0 is 1.81, far below user 0's mean of 3.33. A negative similarity flips the neighbour's opinion. Many systems simply keep only positive similarities.

Lesson: Collaborative filtering: people like you and items like this

REMEMBER

Space Race and Star Siege are rated almost the same way by everyone (cosine 0.97), and so are the three horror films. User 0 loved the two sci fi films and gave Ghost Camp a 1, so the unseen horror films get low predictions of 1.95 and 2.25. This version uses plain cosine on raw ratings, which treats a missing rating as 0 and makes every similarity positive. Sarwar and colleagues found that the adjusted cosine, which subtracts each user's mean first, works better.

Lesson: Collaborative filtering: people like you and items like this

IN THE EXAM

Guessing the average gives a test RMSE of 1.068. For the first 10 epochs SGD is no better, because the small random factors need time to grow. By epoch 20 test RMSE falls to 0.689 and by epoch 60 to 0.36, while train RMSE is 0.2. Run the same program with 30 users, 20 films, only 30% of ratings seen and $\lambda = 0.02$, and it overfits: train RMSE 0.194 but test RMSE 1.145, worse than the average guess. With sparse data the penalty is not optional.

Lesson: Matrix factorisation: hidden tastes learned from ratings

REMEMBER

ALS reaches a test RMSE of 0.319 after only 3 sweeps and 0.259 after 10, against 1.068 for the average guess. Each sweep solves small k by k linear systems exactly, so it needs far fewer passes than SGD. ALS also parallelises well, since every user can be solved at the same time, which is why Spark MLlib uses it.

Lesson: Matrix factorisation: hidden tastes learned from ratings

IN THE EXAM

Two of the five recommendations (A and C) are relevant, so precision@5 = 0.4, and two of the three relevant items are found, so recall@5 = 0.667. The hits are at positions 1 and 3, giving DCG = $1 + 1/\log_2 4 = 1.5$. The best possible list would put all three relevant items first, with ideal DCG 2.131, so NDCG@5 = 0.704. The first item is a hit, so the reciprocal rank is 1.

Lesson: Measuring recommendations and starting cold

REMEMBER

Sorting by plain average favours items with very few ratings: one 5 star rating would beat everything. The shrunk score $(n \times \text{mean} + m \times C) / (n + m)$ pulls items with few ratings towards the overall mean C. Haunted Inn has only 3 ratings, so its average of 3.33 moves to 3.32 here. With one rating it would move much more. This is the score IMDb used for its top 250 list.

Lesson: Measuring recommendations and starting cold

REMEMBER

Popularity bias and feedback loops: items that are shown get clicks, clicks make them shown more, and items that were never shown never get a chance. Exploration (sometimes showing less certain items) and corrections such as inverse propensity weighting reduce this. Measure variety and coverage, not only clicks.

Lesson: Recommenders at scale and their side effects

STEP 1 Reinforcement Learning · RL basics

1.1 Agents, rewards and returns

The agent and environment loop, discounted returns, policies and the explore or exploit choice, and the Gymnasium interface.

KEY IDEAS

- **Loop** State, action, reward, next state. The agent picks actions with a policy $\pi(a \mid s)$.
- **Return** $G_t = r_{t+1} + \gamma\, G_{t+1}$. γ in [0, 1]. Constant reward r gives $r / (1 - \gamma)$.
- **Goal** Maximise the expected return. The best policy is π^*.
- **Trade off** Explore to learn, exploit to earn. ε greedy is the simplest balance.
- **Gymnasium** reset(seed) gives obs and info. step(a) gives obs, reward, terminated, truncated, info. Truncated is not terminal.

- **When to use** Actions change future data and feedback is a delayed score. Watch for reward hacking and sample cost.

FORMULAS

Return Exam favourite

$$G_t = \Sigma_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

The quantity RL maximises in expectation.

Lesson: The return and the discount factor

Recursive form Exam favourite

$$G_t = r_{t+1} + \gamma G_{t+1}$$

The seed of the Bellman equation.

Lesson: The return and the discount factor

Constant reward

$$r + \gamma r + \gamma^2 r + \ldots = r / (1 - \gamma)$$

Bounded whenever $\gamma < 1$.

Lesson: The return and the discount factor

Effective horizon

$$\text{about } 1 / (1 - \gamma) \text{ steps}$$

$\gamma = 0.99$ looks about 100 steps ahead.

Lesson: The return and the discount factor

Interaction

$$s_0, a_0, r_1, s_1, a_1, r_2, s_2, \ldots$$

A trajectory.

Lesson: The maths, all in one place

Policy Exam favourite

$$\pi(a \mid s) = P(A_t = a \mid S_t = s)$$

Stochastic in general.

Lesson: The maths, all in one place

Return Exam favourite

$$G_t = \Sigma_{k \geq 0} \gamma^k r_{t+k+1} = r_{t+1} + \gamma G_{t+1}$$

$0 \leq \gamma \leq 1$.

Lesson: The maths, all in one place

Goal Exam favourite

$$\text{find } \pi \text{ that maximises } E_\pi[G_0]$$

The expected return from the start.

Lesson: The maths, all in one place

Geometric sum

$$\Sigma_{k \geq 0} \gamma^k = 1 / (1 - \gamma)$$

For $\gamma < 1$. Bounds every return by $r_{max} / (1 - \gamma)$.

Lesson: The maths, all in one place

Gambler's ruin

$$P(\text{reach } N \text{ before } 0 \text{ from } k) = k / N, E[\text{steps}] = k(N - k)$$

For a fair random walk.

Lesson: The maths, all in one place

TABLES

Table: term · meaning · in the corridor

Term	Meaning	In the corridor
State s	What the agent observes now	Its cell number
Action a	A choice the agent can make	Left or right
Reward r	A number the environment gives after each action	−1 per move, +10 treasure, −10 pit
Policy π	The rule that maps states to actions	"Go right with chance 0.5"
Episode	One run from a start to an end	Start in cell 2, stop at the pit or the treasure

Lesson: Learning from rewards: the agent and its environment

Table: · exploit · explore

	Exploit	Explore
Does	Pick the best action found so far	Try an action that may be worse
Gains	Reward now	Information for later
Risk	Stuck with a second best choice	Lost reward while testing
Simple rule	Greedy	ε greedy: random action with chance ε

Lesson: Policies, value and the explore or exploit choice

Table: kind of task · ends? · examples · discount

Kind of task	Ends?	Examples	Discount
Episodic	Yes, in a terminal state	A game of chess, one maze run	$\gamma = 1$ is allowed
Continuing	No	Stock trading, heating control	$\gamma < 1$ needed

Lesson: Environments in code: the Gymnasium interface

Table: · supervised · unsupervised · reinforcement

	Supervised	Unsupervised	Reinforcement
Feedback	The right answer for every example	None	A reward, often delayed
Data	A fixed labelled dataset	A fixed unlabelled dataset	Created by the agent's own actions
Goal	Predict labels	Find structure	Maximise total reward
Main difficulty	Generalising	Deciding what structure matters	Exploration and credit for delayed rewards
Example	Spam filter	Customer segments	Game playing agent

Lesson: Reinforcement learning compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
657	Robot Return to Origin	Easy	Count moves in each direction	The robot is back at the start when right moves equal left moves and up moves equal down moves. Tracking where actions take the agent is the job of an environment's step function.
70	Climbing Stairs	Easy	Ways to reach step n = ways to reach n − 1 + ways to reach n − 2	Build the answer from smaller ones and keep only the last two numbers. The same idea of reusing the answer from the next step gives returns backwards: $G_t = r + \gamma\, G_{t+1}$.
874	Walking Robot Simulation	Medium	Simulate one square at a time and check a set of obstacles	Keep a direction index into lists of x and y steps, turn by changing the index, and move one square at a time unless the next square is blocked. Track the largest squared distance. Writing a faithful step by step simulator is the first job in any RL project.
2400	Number of Ways to Reach a Position After Exactly k Steps	Medium	Count how many of the k steps must go right	If the distance is d, you need (k + d) / 2 steps right and the rest left, and the parity must match. The answer is a binomial coefficient. Walks on a line are the corridor of this chapter.
576	Out of Boundary Paths	Medium	Spread path counts one move at a time	Keep the number of ways to stand on each cell after each move. Every move that leaves the grid adds its count to the answer. Pushing counts (or probabilities) forward step by step is how the distribution of states under a policy is computed.
1269	Number of Ways to Stay in the Same Place After Some Steps	Hard	Dynamic programming over position, capped at steps / 2 + 1 cells	Each step you may go left, go right or stay. Keep the number of ways to be at each position. You can never usefully go further than steps / 2 cells, which keeps the array small even when the corridor is huge.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

The random agent stepped right, left, left, right, right, right, right. Six moves cost −1 each and the treasure gave +10, so the total was 4. Going right three times in a row would have scored 8. The agent did not know that. Learning means finding it out from experience.

Lesson: Learning from rewards: the agent and its environment

IN THE EXAM

The rewards −1, −1, −1, +10 add up to 7. With $\gamma = 0.9$ the return is 4.58, because the treasure three steps away counts as $0.9^3 \times 10 = 7.29$. With $\gamma = 0$ only the first reward counts, −1, and the treasure is invisible. Working backwards gives the return from every step at once: 10, 8, 6.2 and 4.58.

Lesson: The return and the discount factor

IN THE EXAM

Over 2,000 episodes: always right earns 8, its exact best path. Right 80% of the time earns 5.29, random earns −7.03 and always left −11. A theory check: a fair random walk from cell 2 between 0 and 5 reaches the treasure first with chance 2/5 and takes $2 \times 3 = 6$ steps on average, so its expected total is $-5 + 0.4 \times 10 + 0.6 \times (-10) = -7$, just as the simulation found.

Lesson: Policies, value and the explore or exploit choice

IN THE EXAM

The agent leaned right but did not reach cell 6 within 20 steps, so the episode was **truncated**, not terminated. Its last state, cell 2, still had a future. A learner that treated this as a true end would wrongly believe cell 2 is worth nothing after this step.

Lesson: Environments in code: the Gymnasium interface

1.2 Multi armed bandits

Running averages and regret, epsilon greedy, UCB, Thompson sampling against A/B tests, and contextual bandits.

KEY IDEAS

- **Problem** One state, k arms, unknown means. Maximise total reward or minimise regret $T\mu^* - \Sigma\ \mu_{a_t}$.
- **Update** $Q \leftarrow Q + (r - Q)/N$. Constant α for drifting arms.
- **ε greedy** Random arm with chance ε, else greedy. Simple, but linear regret with fixed ε.
- **UCB** argmax $Q + c\sqrt{(\ln t / N)}$. Untried arms first. Logarithmic regret. Tune c.
- **Thompson** Beta(1 + wins, 1 + losses). Draw from each, pull the best draw. Probability matching.
- **Context** Contextual bandits (LinUCB) choose per context. Bandits beat fixed A/B tests on reward, A/B tests give cleaner answers.

FORMULAS

Running average Exam favourite

$$Q_{n+1} = Q_n + (r_n - Q_n) / n$$

New estimate = old + step × (target − old).

Lesson: The bandit problem and running averages

Constant step size

$$Q_{n+1} = Q_n + \alpha(r_n - Q_n)$$

Weights reward k pulls ago by $\alpha(1 - \alpha)^k$.

Lesson: The bandit problem and running averages

Regret Exam favourite

$$\text{regret}(T) = T\ \mu^* - \Sigma_t\ \mu_{a_t}$$

Reward lost against always pulling the best arm.

Lesson: The bandit problem and running averages

UCB score Exam favourite

$$A_t = \text{argmax}_a\ [Q(a) + c\ \sqrt{(\ln t / N(a))}]$$

Optimism in the face of uncertainty.

Lesson: Upper confidence bounds: optimism under uncertainty

UCB1

$$Q(a) + \sqrt{(2 \ln t / N(a))}$$

Auer and colleagues, rewards in [0, 1].

Lesson: Upper confidence bounds: optimism under uncertainty

Hoeffding bound

$$P(\mu > Q + u) \leq e^{-2\ N\ u^2}$$

Where the bonus comes from.

Lesson: Upper confidence bounds: optimism under uncertainty

Action value Exam favourite

$$q^*(a) = E[R_t \mid A_t = a]$$

The true mean of arm a.

Lesson: The maths, all in one place

Sample average

$$Q_t(a) = (\text{sum of rewards from a}) / N_t(a)$$

Converges to q*(a) by the law of large numbers.

Lesson: The maths, all in one place

Incremental form Exam favourite

$$Q \leftarrow Q + (1/N)(r - Q)$$

Any step size α_n with $\Sigma\ \alpha_n = \infty$ and $\Sigma\ \alpha_n{}^2 < \infty$ converges.

Lesson: The maths, all in one place

ε greedy choice

$$P(\text{greedy arm}) = 1 - \varepsilon + \varepsilon/k$$

The random pick can also land on it.

Lesson: The maths, all in one place

UCB Exam favourite

$$\text{argmax}_a\ Q(a) + c\ \sqrt{(\ln t / N(a))}$$

Regret O(log T).

Lesson: The maths, all in one place

Beta posterior Exam favourite

$$\text{Beta}(1, 1) + s \text{ clicks and } f \text{ misses} \Rightarrow \text{Beta}(1 + s, 1 + f), \text{mean } (1 + s)/(2 + s + f)$$

Conjugate to the Bernoulli.

Lesson: The maths, all in one place

Lai and Robbins bound

$$\text{regret}(T) \geq (\Sigma_{a:\ \Delta_a>0}\ \Delta_a / \text{KL}(a, a^*))\ \ln T, \text{ as } T \text{ grows}$$

No rule can beat log T.

Lesson: The maths, all in one place

TABLES

Table: rule · explores by · weakness

Rule	Explores by	Weakness
Greedy	Never	Stuck on an early lucky arm
ε greedy	Random arm with chance ε	Explores bad arms as often as promising ones, forever
Optimistic start	High first estimates force early tries	Only helps at the start
UCB	Bonus for rarely pulled arms	Needs a bonus size c
Thompson sampling	Random draws from beliefs	Needs a model of the rewards

Lesson: Greedy and epsilon greedy

Table: · fixed a/b test · bandit (thompson or ucb)

	Fixed A/B test	Bandit (Thompson or UCB)
Traffic split	Fixed, often even	Shifts towards the winner as data arrives
Lost reward while testing	High	Low
Clean significance test	Yes	Harder, the samples are not balanced
Best for	Learning which is better, with a clear answer	Earning while learning, many options, short campaigns

Lesson: Thompson sampling and A/B tests

Table: problem · states · context · actions change the future

Problem	States	Context	Actions change the future
Multi armed bandit	One	No	No
Contextual bandit	One per round	Yes	No
Full reinforcement learning	Many	Yes (the state)	Yes

Lesson: Contextual bandits and where bandits fit

Table: · greedy · ε greedy · ucb

	Greedy	ε greedy	UCB	Thompson sampling
Explores	No	At random, a fixed share	Where uncertainty is high	In proportion to the chance of being best
Settings	None	ε	c	A prior
Regret over time	Can grow linearly	Grows linearly with fixed ε	Logarithmic	Logarithmic
Randomness	None	Yes	None, deterministic	Yes
Typical use	Never alone	A quick baseline	Theory and simple systems	Web experiments, ads

Lesson: Bandit strategies compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1431	Kids With the Greatest Number of Candies	Easy	Compare each value plus the extra with the current maximum	Find the maximum once, then check each kid against it. A greedy bandit makes this comparison every round: which estimate is the highest right now.
1672	Richest Customer Wealth	Easy	The largest row sum	Add up each customer's accounts and take the maximum. Total payout per arm, then the best arm, is the same calculation.
398	Random Pick Index	Medium	Reservoir sampling: keep the i-th match with chance 1/i	Walk the list. At the i-th time you meet the target, replace your choice with chance 1/i. Every matching index ends up equally likely, and no extra memory is needed. Our helper picks many times and returns the set of indices seen, so the answer can be checked.
470	Implement Rand10() Using Rand7()	Medium	Build a uniform number from 1 to 49, reject anything above 40	Two calls of rand7 give 49 equally likely outcomes. Keep only the first 40 and map them to 1 to 10. Rejection sampling like this is how random draws from one distribution are turned into another, as Thompson sampling needs. Our check confirms all ten values appear about equally often.
710	Random Pick with Blacklist	Hard	Draw from the first n minus b numbers and remap blacklisted ones	Let size be the count of allowed numbers. Draw uniformly from 0 to size minus 1. Any blacklisted number in that range is mapped once, at the start, to an allowed number above the range. Each pick is then one random draw and one dictionary lookup. Our helper returns the set of values picked.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

The running average gives 4, 5, 5, 6, 5, exactly the plain average at each point, without storing the rewards. With a constant step size of 0.5 the estimate forgets the past: after three rewards of 1 and then rewards of 10 it climbs to 8.859 within three pulls. A constant step size suits arms whose payouts drift over time.

Lesson: The bandit problem and running averages

IN THE EXAM

Averaged over 500 experiments of 1,000 pulls, greedy chooses the best arm only 25% of the time and earns 0.497 per pull. ε = 0.1 earns the most, 0.705, picking the best arm 68% of the time. ε = 0.3 explores too much: it knows the best arm but still pulls a random one 30% of the time.

Lesson: Greedy and epsilon greedy

IN THE EXAM

Over 2,000 pulls, UCB with c = 1 loses 67.3 in regret, about half of ε greedy's 141.2. But c = 2 does worse than ε greedy (167.7) because its bonus keeps forcing pulls of bad arms, and c = 0.5 lands in between (96.2). UCB is better in principle, yet its bonus size still has to fit the noise in the rewards.

Lesson: Upper confidence bounds: optimism under uncertainty

IN THE EXAM

Over 5,000 visitors and 200 repeats, Thompson sampling earned 323 clicks against 305.3 for an A/B test that split the first half of visitors evenly and then showed the winner. Thompson showed the worst design only 354 times on average, against 844 in the A/B test. The best possible was 350. The A/B test gives a cleaner statistical answer, Thompson gives more clicks while learning.

Lesson: Thompson sampling and A/B tests

1.3 Markov decision processes and Bellman equations

Markov chains, reward processes, MDPs, state and action values, and the Bellman expectation and optimality equations.

KEY IDEAS

- **Markov** Next state depends only on the current state. Chain: P with rows adding to 1. $d_{t+k} = d_t P^k$, stationary $\pi = \pi P$.
- **MDP** (S, A, p, R, γ). MDP + fixed policy = MRP. MRP: $V = R + \gamma PV$, $V = (I - \gamma P)^{-1} R$.
- **Values** $v_\pi(s) = E[G \mid s]$, $q_\pi(s, a) = E[G \mid s, a]$. $v = \Sigma\, \pi\, q$, $q = \Sigma\, p\, (r + \gamma v)$.
- **Optimality** $v^*(s) = \max_a \Sigma\, p\, (r + \gamma v^*)$. Greedy on q^* is optimal. Nonlinear, solved iteratively.
- **Convergence** Bellman operator is a γ contraction, unique fixed point, error shrinks by γ per sweep.
- **Gridworld** Random policy values −14 to −22, optimal values = −distance to exit. Greedy on v_π improves π.

FORMULAS

Markov property

Exam favourite

$$P(S_{t+1} \mid S_t, S_{t-1}, \ldots, S_0) = P(S_{t+1} \mid S_t)$$

The present holds all that matters.

Lesson: The Markov property and Markov chains

k steps ahead

Exam favourite

$$d_{t+k} = d_t\, P^k$$

Row vector times matrix powers.

Lesson: The Markov property and Markov chains

Stationary distribution

Exam favourite

$$\pi = \pi\, P,\ \Sigma\, \pi_i = 1$$

Left eigenvector for eigenvalue 1.

Lesson: The Markov property and Markov chains

State value

Exam favourite

$$v_\pi(s) = E_\pi[G_t \mid S_t = s]$$

How good it is to be in s.

Lesson: State values and action values for a policy

Action value

Exam favourite

$$q_\pi(s, a) = E_\pi[G_t \mid S_t = s, A_t = a]$$

How good it is to take a in s.

Lesson: State values and action values for a policy

Links

Exam favourite

$$v_\pi(s) = \Sigma_a\, \pi(a|s)\, q_\pi(s, a),\ q_\pi(s, a) = \Sigma_{s',r}\, p(s', r \mid s, a)\, [r + \gamma\, v_\pi(s')]$$

Average over actions, then over outcomes.

Lesson: State values and action values for a policy

MDP

Exam favourite

$$(S, A, p, R, \gamma),\ p(s', r \mid s, a) = P(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a)$$

The four argument dynamics function.

Lesson: The maths, all in one place

MRP solution

Exam favourite

$$V = (I - \gamma P)^{-1}\, R$$

Cost $O(n^3)$, fine for small n.

Lesson: The maths, all in one place

Bellman expectation

Exam favourite

$$v_\pi(s) = \Sigma_a\, \pi(a|s)\, \Sigma_{s',r}\, p(s', r \mid s, a)[r + \gamma\, v_\pi(s')]$$

Linear in v_π.

Lesson: The maths, all in one place

Bellman optimality

Exam favourite

$$v^*(s) = \max_a\, \Sigma_{s',r}\, p(s', r \mid s, a)[r + \gamma\, v^*(s')]$$

Nonlinear because of the max.

Lesson: The maths, all in one place

Optimal action value

$$q^*(s, a) = \Sigma_{s',r}\, p(s', r \mid s, a)[r + \gamma\, \max_{a'}\, q^*(s', a')]$$

Acting: $\pi^*(s) = \text{argmax}_a\, q^*(s, a)$.

Lesson: The maths, all in one place

Contraction

$$\|TU - TV\|_\infty \le \gamma\, \|U - V\|_\infty$$

Hence a unique fixed point.

Lesson: The maths, all in one place

Number of policies

$$|A|^{|S|} \text{ deterministic policies}$$

Far too many to try one by one.

Lesson: The maths, all in one place

TABLES

Table: model · has · answers

Model	Has	Answers
Markov chain	States, transitions	Where will I be?
Markov reward process	+ rewards, γ	How good is each state?
Markov decision process	+ actions	What should I do in each state?
Partially observable MDP	+ observations that hide the state	What should I do when I cannot see the state?

Lesson: Markov reward processes and decision processes

Table: equation · relates · linear? · solved by

Equation	Relates	Linear?	Solved by
Bellman expectation for v_π	A state and its successors under π	Yes	Matrix solve or iterative evaluation
Bellman optimality for v*	A state and its successors under the best action	No (max)	Value iteration, policy iteration
Bellman optimality for q*	An action and the best next actions	No (max)	Q learning, DQN

Lesson: Optimal values and the Bellman optimality equation

Table: · markov chain · markov reward process · markov decision process

	Markov chain	Markov reward process	Markov decision process	Bandit
States	Many	Many	Many	One
Actions	None	None	Yes	Yes
Rewards	None	Per state	Per state and action	Per action
Question	Long run behaviour	Value of each state	Best policy	Best arm
Main equation	π = πP	V = R + γPV	v* = max over actions of the Bellman backup	Q ← Q + (r − Q)/N

Lesson: Markov models compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
509	Fibonacci Number	Easy	Each value comes from the two before it	Keep only the last two numbers and step forward. A value defined by earlier values is the simplest recursion, the same shape as a Bellman equation.
746	Min Cost Climbing Stairs	Easy	Best cost to reach step i = min of the two ways in	The cheapest cost to stand on step i is the cheaper of coming from i − 1 or i − 2, plus that step's cost. Choosing the best of several moves and adding a cost is a Bellman optimality backup with costs instead of rewards.
64	Minimum Path Sum	Medium	Work backwards from the goal	The best total from a cell is its own cost plus the better of the best totals to the right and below. Filling the table from the goal backwards is exactly how optimal values are found in a deterministic gridworld.
120	Triangle	Medium	Collapse the triangle from the bottom row up	Start with the bottom row as the best totals. Each number above adds the smaller of its two children. One pass from the bottom gives the answer at the top, in the style of a backward Bellman sweep.
935	Knight Dialer	Medium	Count numbers by pushing counts through the knight's moves	Keep how many numbers end on each digit. Each step, every digit passes its count to the digits a knight can jump to. This is a Markov chain transition applied n − 1 times, with counts instead of probabilities.
174	Dungeon Game	Hard	Go backwards from the princess: health needed on entering each room	Forward search fails because the best path depends on what comes later. Instead compute, from the goal backwards, the least health needed when entering each room: one less than the smaller need of the next rooms minus this room's value, but at least 1. Backward induction like this is how finite horizon MDPs are solved.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

Walking the chain for 10,000 days gives shares 0.468, 0.304 and 0.228. Pushing the starting distribution through P 50 times gives 0.462, 0.308 and 0.231, the same as the left eigenvector of P for eigenvalue 1. Two days ahead from rainy, sunny has chance $0.2 \times 0.7 + 0.4 \times 0.3 + 0.4 \times 0.2 = 0.34$.

Lesson: The Markov property and Markov chains

IN THE EXAM

Solving the linear system gives Work 9.179, Gym 4.198, Wake −0.138 and Couch −2.282. Repeating $V \leftarrow R + \gamma PV$ from all zeros reaches the same numbers: within 0.01 after 20 sweeps and exactly after 100. Check Work by hand: $10 + 0.9 \times (0.4 \times (-2.282) + 0.6 \times 0) = 9.179$.

Lesson: Markov reward processes and decision processes

IN THE EXAM

These are the values from Sutton and Barto's example 4.1. Next to an exit the random walker still needs 14 moves on average (value −14), and in the far corners 22. Values are expected costs to finish, so they tell you how hopeless the random policy is, not how short the best path is.

Lesson: State values and action values for a policy

REMEMBER

In cell 6 the four action values are −21, −19, −19 and −21, and their average is −20, exactly $v_\pi(6)$. Picking the best action in every cell gives arrows that already lead straight to the exits. Acting greedily on the values of even a poor policy gives a better policy. This is policy improvement, the next chapter's main idea.

Lesson: State values and action values for a policy

2.1 Dynamic programming: policy and value iteration

Policy evaluation, policy iteration, value iteration on FrozenLake and the gambler, and generalised policy iteration.

KEY IDEAS

- **DP** Known model, sweeps of Bellman backups over all states. Stop when max change < θ.
- **Evaluation** v(s) ← Σ π(a|s) Σ p (r + γv(s')). Error shrinks by γ per sweep. In place is fine.
- **Policy iteration** Evaluate, then greedy improve. Improvement theorem. Few iterations, stops when the policy is stable.
- **Value iteration** v(s) ← $\max_a$ Σ p (r + γv(s')), then greedy extraction. Many cheap sweeps.
- **Examples** Gridworld −14 to −22, FrozenLake start 0.542 (slippery, γ 0.99), gambler bets boldly at 25, 50, 75.
- **Limits** GPI is the pattern of all RL. DP needs a model and suffers the curse of dimensionality. LP solves MDPs in polynomial time.

FORMULAS

Policy evaluation update Exam favourite

$$v_{k+1}(s) = \Sigma_a \pi(a|s) \Sigma_{s',r} p(s', r \mid s, a)[r + \gamma v_k(s')]$$

Repeat until max change < θ.

Lesson: Dynamic programming and policy evaluation

Error after k sweeps

$$\|v_k - v_\pi\|_\infty \le \gamma^k \|v_0 - v_\pi\|_\infty$$

Geometric convergence for γ < 1.

Lesson: Dynamic programming and policy evaluation

Policy evaluation Exam favourite

$$v_{k+1} = r_\pi + \gamma P_\pi v_k$$

Matrix form for a fixed policy.

Lesson: The maths, all in one place

Greedy improvement Exam favourite

$$\pi'(s) = \operatorname{argmax}_a q_\pi(s, a)$$

Never worse than π.

Lesson: The maths, all in one place

Policy improvement theorem Exam favourite

$$q_\pi(s, \pi'(s)) \ge v_\pi(s) \text{ for all } s \Rightarrow v_{\pi'}(s) \ge v_\pi(s) \text{ for all } s$$

Sutton and Barto section 4.2.

Lesson: The maths, all in one place

Value iteration Exam favourite

$$v_{k+1}(s) = \max_a \Sigma_{s',r} p(s', r \mid s, a)[r + \gamma v_k(s')]$$

The optimality operator applied once per sweep.

Lesson: The maths, all in one place

Sweeps for accuracy ε

$$k \ge \log(\varepsilon / e_0) / \log \gamma$$

From $e_k \le \gamma^k e_0$.

Lesson: The maths, all in one place

Stopping rule

$$\text{if } \|v_{k+1} - v_k\| < \theta(1 - \gamma)/(2\gamma) \text{ then the greedy policy is } \theta \text{ optimal}$$

Williams and Baird (1993).

Lesson: The maths, all in one place

Cost of one sweep

$$O(|S|^2 |A|)$$

Less with sparse transitions.

Lesson: The maths, all in one place

TABLES

Table: method · model needed · backup · per step uses

Method	Model needed	Backup	Per step uses
Policy evaluation	Yes	Expected, under π	All successor states
Policy iteration	Yes	Full evaluation, then greedy	All successor states
Value iteration	Yes	Expected, max over actions	All successor states
Monte Carlo	No	Sampled full return	One episode
TD and Q learning	No	Sampled one step, bootstrapped	One transition

Lesson: Generalised policy iteration and the limits of DP

Table: · policy iteration · value iteration

	Policy iteration	Value iteration
Each iteration	Full evaluation, then greedy improvement	One sweep of max backups
Iterations to finish	Few (often under 10)	Many (hundreds when γ is near 1)
Cost per iteration	High (a full evaluation)	Low (one sweep)
Stops when	The policy does not change	Values change less than θ
Gives	Exact optimal policy	Near optimal values, then the greedy policy
Leads on to	Actor critic methods	Q learning

Lesson: Policy iteration and value iteration compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
1137	N-th Tribonacci Number	Easy	Carry the last three values forward	Each term is the sum of the three before it. Update three variables at once. Carrying a small state forward and updating it again and again is what every DP sweep does.
118	Pascal's Triangle	Easy	Each row comes from the row above	Every inner number is the sum of the two numbers above it. Building a new row entirely from the previous one is a synchronous sweep: the old values are read, the new ones written.
198	House Robber	Medium	Two values: best total if you rob this house, and if you skip it	At each house choose the better of robbing it (the previous skip total plus its money) or skipping it (the better of the two previous totals). A max over two actions at every step is a Bellman optimality backup.
322	Coin Change	Medium	Fewest coins for each amount, built from smaller amounts	best[a] is the minimum over coins c of best[a − c] + 1. Each amount is a state, each coin an action with cost 1, and the table is filled like value iteration on a deterministic MDP.
787	Cheapest Flights Within K Stops	Medium	Bellman Ford for k + 1 rounds, reading only last round's costs	Run k + 1 rounds of relaxing every flight, but compute each round from a copy of the previous costs, so a round adds at most one flight. Bellman Ford is value iteration for shortest paths: each round is one sweep of the Bellman backup with costs.
1463	Cherry Pickup II	Hard	State = the two robots' columns, swept row by row	Both robots move down one row at a time, so the state is the pair of columns. For each row, every pair of moves (9 of them) updates the best total for the new pair, counting a shared cell once. Sweeping a state table forward row by row with a max over joint actions is DP over a joint state space.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM After sweep 1 every cell is −1. After sweep 2 a cell next to an exit is 0.25 × (−1) + 0.75 × (−2) = −1.75, and others are −2. With γ = 1 it takes 173 sweeps for the largest change to fall below 0.0001, and the final values are the −14 to −22 seen in the last chapter.

Lesson: Dynamic programming and policy evaluation

IN THE EXAM Starting from "always go left", the value of the start rises from 0 to 0.121, 0.52, 0.532 and finally 0.542 after 7 iterations, when the policy stops changing. The optimal first move at the start is left, into the wall. On ice where you slide sideways two times in three, pushing against the wall means you can only slide down or stay put, never into danger. Good policies on slippery ice look odd but are safe.

Lesson: Policy improvement and policy iteration

IN THE EXAM Value iteration reaches exactly the same policy and the same start value, 0.542, as policy iteration, but needs 324 sweeps against 7 policy iterations (each of which hides many evaluation sweeps). The start value stays 0 for the first 5 sweeps because the reward is 6 steps away. The best cells are the two on the bottom row just before the goal: 0.863 and 0.742.

Lesson: Value iteration

REMEMBER Sutton and Barto's gambler with a 0.4 coin: the best play is bold. With 50 you stake everything (win chance 0.4). With 25 you stake 25 to reach 50, so the win chance is 0.4 × 0.4 = 0.16. With 75 you stake 25 to reach 100 or fall back to 50, giving 0.4 + 0.6 × 0.4 = 0.64. When the odds are against you, fewer bets mean less time for the house edge to work.

Lesson: Value iteration

2.2 Monte Carlo and temporal difference learning

Monte Carlo and TD(0) prediction, their bias and variance, batch updating, n step TD and TD(lambda).

KEY IDEAS

- **MC** $V(s) \leftarrow V(s) + \alpha(G - V(s))$. Full returns, no model, unbiased, high variance, needs episode ends.
- **TD(0)** $\delta = r + \gamma V(s') - V(s)$, $V(s) \leftarrow V(s) + \alpha\delta$. Bootstraps, online, lower variance, biased.
- **Compare** TD usually faster on Markov problems. Batch MC fits returns, batch TD fits the Markov model (example 6.4).
- **Step size** Constant α tracks change but fluctuates. Robbins Monro: $\Sigma\,\alpha = \infty$, $\Sigma\,\alpha^2 < \infty$.
- **n step and λ** $G_{t:t+n}$ mixes n rewards and a bootstrap. TD(λ) weights $(1 - \lambda)\lambda^{n-1}$. Traces decay by $\gamma\lambda$.
- **Warning** Deadly triad: function approximation + bootstrapping + off policy can diverge.

FORMULAS

Monte Carlo update Exam favourite

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

Target: the full return G_t. Waits for the episode to end.

Lesson: Learning without a model: Monte Carlo prediction

Sample average form

$$\alpha = 1 / N(S_t)$$

Converges to v_π by the law of large numbers.

Lesson: Learning without a model: Monte Carlo prediction

TD(0) update Exam favourite

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

Target: one reward plus a guess.

Lesson: Temporal difference learning: TD(0)

TD error Exam favourite

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

Also the signal dopamine neurons appear to carry (Schultz, Dayan and Montague, 1997).

Lesson: Temporal difference learning: TD(0)

MC error as TD errors

$$G_t - V(S_t) = \Sigma_{k \geq t} \gamma^{k-t} \delta_k$$

If V does not change during the episode.

Lesson: Temporal difference learning: TD(0)

Return

Exam favourite

$$G_t = R_{t+1} + \gamma G_{t+1}$$

The Monte Carlo target.

Lesson: The maths, all in one place

TD(0)

Exam favourite

$$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$$

The TD target bootstraps.

Lesson: The maths, all in one place

n step return

Exam favourite

$$G_{t:t+n} = \Sigma_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V(S_{t+n})$$

n = 1 TD(0), n = ∞ MC.

Lesson: The maths, all in one place

λ return

$$G_t^{\lambda} = (1 - \lambda) \Sigma_{n \geq 1} \lambda^{n-1} G_{t:t+n}$$

λ = 0 TD(0), λ = 1 MC.

Lesson: The maths, all in one place

Accumulating trace

$$e_t(s) = \gamma \lambda e_{t-1}(s) + 1[S_t = s]$$

Recent and frequent states get more credit.

Lesson: The maths, all in one place

Robbins Monro conditions

$$\Sigma \alpha_t = \infty, \Sigma \alpha_t^2 < \infty$$

Step sizes that guarantee convergence, such as 1/t.

Lesson: The maths, all in one place

Random walk truth

$$v(k) = k / 6 \text{ for states } k = 1 \text{ to } 5$$

Gambler's ruin with a fair coin.

Lesson: The maths, all in one place

TABLES

Table: method · target · bias · variance

Method	Target	Bias	Variance	Updates
TD(0)	r + γV(s')	Higher	Lowest	Every step
n step TD	n rewards + $\gamma^n V$	Medium	Medium	n steps late
TD(λ)	Weighted mix of all n step returns	Tunable	Tunable	Every step, with traces
Monte Carlo	Full return G	None	Highest	End of episode

Lesson: Between the two: n step TD and TD(λ)

Table: · dynamic programming · monte carlo · temporal difference

	Dynamic programming	Monte Carlo	Temporal difference
Needs a model	Yes	No	No
Bootstraps	Yes	No	Yes
Samples	No (expected updates)	Yes (whole episodes)	Yes (single steps)
Works without episode ends	Yes	No	Yes
Bias, variance	No sampling noise	Unbiased, high variance	Biased, low variance
Uses the Markov property	Yes	No	Yes

Lesson: Prediction methods compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
933	Number of Recent Calls	Easy	Keep a queue and drop old calls from the front	Add each call to the back and remove calls older than 3,000 milliseconds from the front. Keeping only recent history is what a constant step size does to an estimate: old data fades out. Our helper returns the count after each ping.
1716	Calculate Money in Leetcode Bank	Easy	Day d deposits d // 7 + d % 7 + 1	Each Monday starts one higher than the last, and each day adds one more. Adding up rewards day by day is how a Monte Carlo return is built.
1352	Product of the Last K Numbers	Medium	Prefix products, reset at every zero	Keep running products. The product of the last k numbers is the last prefix divided by the one k places earlier. A zero wipes the history, so restart the list. Running quantities that let you answer about any recent window are the idea behind n step returns. Our helper runs a list of operations.
1423	Maximum Points You Can Obtain from Cards	Medium	Taking k cards from the ends leaves a middle window of n minus k cards	Find the middle window with the smallest sum by sliding it across, and subtract it from the total. Sliding a fixed size window and updating its sum in O(1) per step is the same bookkeeping an n step TD learner keeps.
239	Sliding Window Maximum	Hard	A deque of indices with decreasing values	Before adding a new index, pop smaller values from the back, since they can never be a maximum again. Pop the front once it leaves the window. The front is always the current maximum. Each index enters and leaves once, so the whole pass is O(n).

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

The walk starts in C and moves left or right at random. Reaching the right end gives 1, so the true value of a state is its chance of finishing on the right: 1/6 to 5/6. After one episode MC knows only that one walk ended on the right. After 1,000 episodes its averages are within 0.011 of the truth (RMS error). MC is unbiased but needs many episodes because single returns vary a lot.

Lesson: Learning without a model: Monte Carlo prediction

IN THE EXAM

TD reaches an error of 0.075 after 100 episodes, against MC's 0.095 in the previous program. With a constant α of 0.1 it then keeps fluctuating around the truth (0.071 after 1,000 episodes) instead of settling, while MC with sample averages kept improving. A constant step size trades exact convergence for the ability to track change.

Lesson: Temporal difference learning: TD(0)

IN THE EXAM

Averaged over 100 runs, TD with α = 0.05 reaches an error of 0.035 after 100 episodes, against 0.094 for MC with either step size. TD with a large α of 0.15 is fastest early (0.095 after 10 episodes) but then stalls near 0.07, because each step jumps too far. Step size matters as much as the method.

Lesson: Monte Carlo against TD: bias, variance and batch learning

REMEMBER

Eight episodes: A then B with no reward once, then B alone six times with reward 1 and once with 0. Both methods give V(B) = 0.75. MC gives V(A) = 0, because the only return seen after A was 0. Batch TD gives V(A) = 0.75, because A always led to B and B is worth 0.75. TD's answer is the one a Markov model of the data implies, and it usually predicts future data better.

Lesson: Monte Carlo against TD: bias, variance and batch learning

2.3 SARSA and Q learning

SARSA, Q learning and expected SARSA on the cliff walk, on and off policy learning, and double Q learning.

KEY IDEAS

- **Action values** Learn Q(s, a), act with argmax. No model needed. Explore with ε greedy.

- **SARSA** Target r + γQ(s', a'), a' actually taken. On policy, safe under exploration.
- **Q learning** Target r + γ max Q(s', ·). Off policy, learns q*. Cliff: edge path, worse reward while learning.
- **Expected SARSA** Target r + γ Σ π Q. Lower variance. Greedy π gives Q learning.
- **Double Q** Max of noisy estimates is biased up. Select with Q_1, evaluate with Q_2.
- **Convergence** GLIE exploration, Robbins Monro step sizes, every pair visited. Tabular only. Deep versions need more care.

FORMULAS

SARSA update Exam favourite

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$$

A' is the action actually taken next.

Lesson: From prediction to control: action values and SARSA

ε greedy Exam favourite

$$\pi(a|s) = 1 - \varepsilon + \varepsilon/|A| \text{ for the greedy a, and } \varepsilon/|A| \text{ for others}$$

Keeps exploring.

Lesson: From prediction to control: action values and SARSA

Q learning update Exam favourite

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$$

The sample version of the Bellman optimality equation.

Lesson: Q learning: learning the best policy while exploring

Bias of the max Exam favourite

$$E[\max_a \hat{Q}(a)] \geq \max_a E[\hat{Q}(a)]$$

Jensen's inequality: max is convex.

Lesson: Maximisation bias and double Q learning

Double Q target Exam favourite

$$r + \gamma\, Q_2(s', \operatorname{argmax}_a Q_1(s', a))$$

Select with one table, evaluate with the other.

Lesson: Maximisation bias and double Q learning

Two noisy zeros

$$E[\max(X_1, X_2)] = 1/\sqrt{\pi} \approx 0.564 \text{ for } X_i \sim N(0, 1)$$

Although both true values are 0.

Lesson: Maximisation bias and double Q learning

SARSA Exam favourite

$$Q(s, a) \mathrel{+}= \alpha[r + \gamma Q(s', a') - Q(s, a)]$$

On policy TD control.

Lesson: The maths, all in one place

Q learning Exam favourite

$$Q(s, a) \mathrel{+}= \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Off policy TD control.

Lesson: The maths, all in one place

Expected SARSA Exam favourite

$$Q(s, a) \mathrel{+}= \alpha[r + \gamma \Sigma_{a'} \pi(a'|s')Q(s', a') - Q(s, a)]$$

Averages out the next action.

Lesson: The maths, all in one place

Double Q learning

$$Q_1(s, a) \mathrel{+}= \alpha[r + \gamma Q_2(s', \operatorname{argmax} Q_1(s', \cdot)) - Q_1(s, a)]$$

Swap roles at random.

Lesson: The maths, all in one place

GLIE

$$\text{every } (s, a) \text{ tried infinitely often and } \varepsilon_k \to 0, \text{ for example } \varepsilon_k = 1/k$$

Greedy in the limit with infinite exploration: makes SARSA converge to q*.

Lesson: The maths, all in one place

Convergence of Q learning

$$\Sigma_t \alpha_t(s, a) = \infty, \Sigma_t \alpha_t(s, a)^2 < \infty \text{ for every pair}$$

Watkins and Dayan (1992), tabular case.

Lesson: The maths, all in one place

TABLES

Table: method · target · policy learned · on the cliff

Method	Target	Policy learned	On the cliff
SARSA	$r + \gamma Q(s', a')$, a' sampled	The exploring policy (on policy)	Safe path, good reward while learning
Expected SARSA	$r + \gamma \sum \pi(a'\|s')Q(s', a')$	The exploring policy, less noise	Safe path, best reward while learning
Q learning	$r + \gamma \max Q(s', a')$	The greedy policy (off policy)	Optimal edge path, worse reward while learning

Lesson: Expected SARSA, and comparing the three

Table: · monte carlo control · sarsa · expected sarsa

	Monte Carlo control	SARSA	Expected SARSA	Q learning
Target	Full return G	$r + \gamma Q(s', a')$	$r + \gamma E_\pi[Q(s', \cdot)]$	$r + \gamma \max Q(s', \cdot)$
On or off policy	On (or off with importance sampling)	On	Either	Off
Learns	v of the exploring policy	q of the exploring policy	q of the target policy	q*
Variance	High	Medium	Low	Low
Risk	Slow	None special	More computation	Maximisation bias, unsafe exploration

Lesson: TD control methods compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
2373	Largest Local Values in a Matrix	Easy	Take the max of every 3 by 3 block	For each top left corner, take the largest of the nine values below and to the right. A max over a small neighbourhood is what Q learning computes over the next state's actions.
62	Unique Paths	Medium	Paths to a cell = paths from above + paths from the left	Keep one row of counts and add the left neighbour into each cell, row after row. Counting routes on a grid shows how quickly the number of possible paths grows, which is why RL learns values instead of listing paths.
1091	Shortest Path in Binary Matrix	Medium	Breadth first search over 8 directions	With equal cost moves, breadth first search finds the shortest path, visiting cells in order of distance. Q learning on a grid with −1 per move learns the same distances as negative values, but from trial and error instead of a known map.
1926	Nearest Exit from Entrance in Maze	Medium	Breadth first search until you reach a border cell other than the entrance	Search outward from the entrance and stop at the first open cell on the border. A maze with exits is a gridworld: the answer is minus the optimal value of the entrance when each move costs 1.
1293	Shortest Path in a Grid with Obstacles Elimination	Hard	Add the number of eliminations left to the state	Position alone is not enough to decide what to do next: you also need to know how many walls you can still break. Search over (row, column, eliminations left). Choosing a state that holds everything needed for the future is exactly the Markov property from chapter 1.3.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

On the cliff walk SARSA earns an average of −24.42 per episode over its last 100 episodes while still exploring with $\varepsilon = 0.1$. Its greedy path climbs to the top row before heading right. The path is longer than needed, but a random step there costs one move, not a fall of −100.

Lesson: From prediction to control: action values and SARSA

IN THE EXAM Q learning finds the truly shortest path, straight along the cliff edge (13 moves, reward −13). Yet while it still explores, its average reward is only −70.01, far worse than SARSA's, because ε greedy steps near the edge keep sending it over the cliff. Q learning values the policy it would follow without exploring, SARSA values the policy it really follows.

Lesson: Q learning: learning the best policy while exploring

IN THE EXAM Averaged over 5 runs, the reward per episode while learning (last 100 of 200 episodes) is −49.2 for Q learning, −34.6 for SARSA and −21.1 for expected SARSA. Expected SARSA learns the safe route like SARSA, with steadier updates. Results vary from run to run, so real comparisons average many seeds.

Lesson: Expected SARSA, and comparing the three

IN THE EXAM In state A, going left leads to B, where every action pays −0.1 on average with a lot of noise, so left is a mistake. Plain Q learning still goes left in 86% of episodes at episode 10 and 16% at episode 300, because the max over B's noisy estimates looks positive. Double Q learning goes left only 40% of the time at episode 10 and settles at the best possible 5% (the random ε moves) by episode 50.

Lesson: Maximisation bias and double Q learning

STEP 3 Reinforcement Learning · Modern RL

3.1 Function approximation and deep Q networks

Value function approximation, tile coding, DQN with replay and a target network, its improvements, and the deadly triad.

KEY IDEAS

- **Function approximation** $\hat{v}(s, w)$ with few weights. Generalises. Objective VE weighted by $\mu(s)$.
- **Semi gradient TD** $w \mathrel{+}= \alpha[r + \gamma\hat{v}(s') - \hat{v}(s)]\nabla\hat{v}(s)$. Target treated as fixed. Linear gradient = features.
- **Tile coding** Several shifted grids, one active tile each. α about 0.5/n. Mountain car: 1944 weights.
- **DQN** Network $Q(s, a, \theta)$, experience replay, target network θ^- copied every C steps, Huber loss.
- **Improvements** Double DQN, dueling V + A − mean A, prioritised replay by $|\delta|$, multi step, distributional, noisy nets = Rainbow.
- **Deadly triad** Approximation plus bootstrapping plus off policy can diverge. Baird's counterexample. Fixes: target networks, gradient TD.

FORMULAS

Prediction objective Exam favourite

$$\mathrm{VE}(\mathbf{w}) = \Sigma_s\, \mu(s)\, [v_\pi(s) - \hat{v}(s, \mathbf{w})]^2$$

$\mu(s)$ is the share of time spent in s.

Lesson: Why tables break, and learning a value function

Gradient Monte Carlo Exam favourite

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha[G_t - \hat{v}(S_t, \mathbf{w})]\, \nabla\hat{v}(S_t, \mathbf{w})$$

Unbiased target, converges to a local minimum of VE.

Lesson: Why tables break, and learning a value function

Semi gradient TD(0) Exam favourite

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha[R + \gamma\hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})]\, \nabla\hat{v}(S, \mathbf{w})$$

The target also depends on w, but we ignore that in the gradient.

Lesson: Why tables break, and learning a value function

Linear value Exam favourite

$$\hat{v}(s, \mathbf{w}) = \mathbf{w}^{\mathrm{T}}\mathbf{x}(s), \quad \nabla\hat{v} = \mathbf{x}(s)$$

The gradient is the feature vector.

Lesson: Linear features and tile coding

Tile coding step size

$$\alpha = 0.5 / n \text{ for } n \text{ tilings}$$

Each update moves the value about half way to the target.

Lesson: Linear features and tile coding

DQN loss

Exam favourite

$$L(\theta) = E_{(s, a, r, s') \sim D}[(r + \gamma \max_{a'} Q(s', a', \theta^-) - Q(s, a, \theta))^2]$$

D is the replay memory, θ^- the target weights.

Lesson: Deep Q networks: replay memory and a target network

Huber loss

$$L_\delta(e) = e^2/2 \text{ if } |e| \le 1, \text{ else } |e| - 1/2$$

Same as clipping the error to [−1, 1] in the gradient.

Lesson: Deep Q networks: replay memory and a target network

Double DQN target

Exam favourite

$$y = r + \gamma Q(s', \text{argmax}_a\, Q(s', a, \theta), \theta^-)$$

Online network selects, target network evaluates.

Lesson: Double DQN, dueling networks, prioritised replay and Rainbow

Dueling aggregation

Exam favourite

$$Q(s, a) = V(s) + A(s, a) - (1/|A|)\, \Sigma_{a'}\, A(s, a')$$

Subtracting the mean makes V and A identifiable.

Lesson: Double DQN, dueling networks, prioritised replay and Rainbow

Prioritised sampling

$$P(i) = p_i^\alpha / \Sigma_k\, p_k^\alpha, \quad w_i = (N\, P(i))^{-\beta} / \max_j w_j$$

α = 0 gives uniform sampling.

Lesson: Double DQN, dueling networks, prioritised replay and Rainbow

Importance ratio

Exam favourite

$$\rho = \pi(a|s) / b(a|s)$$

Reweights off policy samples. In Baird's example ρ = 7 for solid and 0 for dashed.

Lesson: The deadly triad: when approximation diverges

Linear TD fixed point

$$\mathbf{w}_{TD} = \mathbf{A}^{-1}\mathbf{b}, \ \mathbf{A} = E[\mathbf{x}_t(\mathbf{x}_t - \gamma \mathbf{x}_{t+1})^T], \ \mathbf{b} = E[R_{t+1}\mathbf{x}_t]$$

Stable when A is positive definite, which holds on policy.

Lesson: The deadly triad: when approximation diverges

TD error bound

$$VE(\mathbf{w}_{TD}) \le VE(\text{best } \mathbf{w}) / (1 - \gamma)$$

On policy linear TD, Tsitsiklis and Van Roy (1997).

Lesson: The deadly triad: when approximation diverges

Value error

Exam favourite

$$VE(\mathbf{w}) = \Sigma_s\, \mu(s)[v_\pi(s) - \hat{v}(s, \mathbf{w})]^2$$

The prediction objective.

Lesson: The maths, all in one place

Semi gradient TD(0)

Exam favourite

$$\mathbf{w} \mathrel{+}= \alpha\delta\nabla\hat{v}(S, \mathbf{w}), \ \delta = R + \gamma\hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$$

Linear: $\nabla\hat{v} = \mathbf{x}(S)$.

Lesson: The maths, all in one place

Semi gradient SARSA

Exam favourite

$$\mathbf{w} \mathrel{+}= \alpha[R + \gamma\hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})]\nabla\hat{q}(S, A, \mathbf{w})$$

Control with features.

Lesson: The maths, all in one place

DQN target

Exam favourite

$$y = r + \gamma(1 - \text{done}) \max_{a'} Q(s', a', \theta^-)$$

Frozen target weights.

Lesson: The maths, all in one place

Dueling

$$Q = V + A - \text{mean}(A)$$

Splits state value and advantage.

Lesson: The maths, all in one place

TABLES

Table: features · how they work · good for

Features	How they work	Good for
State aggregation	One weight per group of states	A first try, easy to understand
Polynomials	Powers and products of state variables	Smooth values in a few dimensions
Fourier basis	Cosines of different frequencies	Smooth values, simple to set up
Tile coding	Several shifted grids, one active tile each	Low dimensional control, fast updates
Radial basis functions	Bumps centred at chosen points	Smooth local generalisation
Neural networks	Features are learned, not designed	Images, text, high dimensional input

Lesson: Linear features and tile coding

Table: rainbow part · what it changes · why it helps

Rainbow part	What it changes	Why it helps
Double Q learning	Select with online, evaluate with target	Removes overestimation
Prioritised replay	Sample by TD error	Learns faster from rare, surprising events
Dueling network	Q = V + A − mean A	Better values when actions matter little
Multi step returns	n step targets, n = 3	Rewards travel back faster
Distributional RL (C51)	Learns the whole spread of returns in 51 bins	Richer signal, more stable learning
Noisy nets	Learned noise in the weights	Exploration without a hand set ε

Lesson: Double DQN, dueling networks, prioritised replay and Rainbow

Table: · table · linear features · deep network

	Table	Linear features	Deep network
Parameters	One per state (and action)	One per feature	Thousands to millions
Generalisation	None	Through designed features	Through learned features
Convergence	Proven for TD and Q learning	Proven for on policy TD	No guarantee, needs tricks
Inputs	Small discrete states	Low dimensional states	Pixels, text, many sensors
Typical method	Q learning, SARSA	Semi gradient SARSA with tile coding	DQN, Double DQN, Rainbow

Lesson: Tables, linear models and deep networks compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
703	Kth Largest Element in a Stream	Easy	Keep a min heap of the k largest values	Push each new value and pop the smallest whenever the heap holds more than k. The top of the heap is then the kth largest. Keeping only the most important items from a stream is what prioritised replay does with surprising transitions. Our helper returns the answer after each add.
1046	Last Stone Weight	Easy	A max heap: smash the two heaviest each time	Python's heapq is a min heap, so store negative weights. Pop the two largest, push back their difference if it is not zero, and repeat. Always taking the biggest first is how prioritised replay picks the transitions with the largest TD error.
622	Design Circular Queue	Medium	A fixed array with a head index and a count	Store items in a list of fixed size. The next free slot is (head + count) mod size. This ring buffer is exactly how a DQN replay memory is stored: when it is full, the newest transition overwrites the oldest. Our helper runs a list of operations.
146	LRU Cache	Medium	An ordered dictionary: move used keys to the end, evict from the front	OrderedDict keeps keys in order of use. On get or put move the key to the end, and when the cache is too big remove the first key, the least recently used. Choosing what to forget when memory is full is the same question a replay memory answers, and DQN simply forgets the oldest. Our helper runs a list of operations and returns the results of get.
480	Sliding Window Median	Hard	Keep the window sorted and insert and remove with binary search	Hold the current window in a sorted list. Use bisect to insert the new value and remove the one leaving, then read the middle. Medians of recent results are a robust way to report RL progress, because single episodes can be wildly high or low.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

With 10 groups of 100 states, 10 weights stand in for 1000 values. After 2000 episodes gradient Monte Carlo reaches a root mean squared error of 0.058, close to the best any staircase can do. Semi gradient TD reaches 0.11 because its target leans on the next group's weight, which is itself only an average. Its steps near the two ends stay closer to 0.

Lesson: Why tables break, and learning a value function

IN THE EXAM

Mountain car with 8 tilings of 9 by 9 tiles and 3 actions uses 1944 weights. The first episode takes 2111 steps, because every weight starts at 0, which is optimistic when each step costs −1, so the car tries everything. By episodes 51 to 60 it needs about 149 steps on average. It has learned to rock back and forth to build up speed.

Lesson: Linear features and tile coding

IN THE EXAM

This DQN is written in plain NumPy: 4 inputs, 32 hidden units and 2 outputs, Adam, a memory of 10,000 and a target copy every 100 steps. It balances the pole for about 11 to 23 steps on average in the first 75 episodes, then 58.6, 131.1 and 171.4 in later blocks of 25 episodes. Ten greedy test runs last 134 to 147 steps out of 200. Deep RL is sensitive to settings and seeds, so real work compares several seeds.

Lesson: Deep Q networks: replay memory and a target network

IN THE EXAM

Hessel and colleagues found that prioritised replay and multi step returns mattered most: removing either one hurt Rainbow the most. Removing dueling or Double Q learning hurt the least on average.

Lesson: Double DQN, dueling networks, prioritised replay and Rainbow

IN THE EXAM

With linear features the size of the weight vector grows from 10.34 to 452.23 in 1000 steps, although every true value is 0. With a table (one value per state) and the same off policy data, it slowly shrinks, from 2.65 to 2.61. The problem is not off policy data alone, but all three parts of the triad together.

Lesson: The deadly triad: when approximation diverges

3.2 Policy gradients, actor critic and PPO

REINFORCE, baselines, actor critic, GAE, PPO and TRPO, and continuous control methods.

KEY IDEAS

- **Policy gradient** Learn $\pi(a|s, \theta)$ by gradient ascent on J. Stochastic and continuous policies. Short corridor: best p = 0.59.
- **REINFORCE** θ += $\alpha G_t \nabla \ln \pi$. Softmax: one hot(a) − π. Monte Carlo, high variance.
- **Baseline** Use G − b(s). Unbiased because $\Sigma_a \nabla \pi = 0$. Best simple choice v(s). Gives the advantage.
- **Actor critic** Critic TD error δ replaces G. GAE: $\hat{A}_t = \Sigma(\gamma\lambda)^l \delta_{t+l}$. A2C, A3C.
- **PPO and TRPO** PPO: min(rA, clip(r, 1 − ε, 1 + ε)A), ε = 0.2, several epochs per batch. TRPO: KL trust region.
- **Continuous control** Gaussian policy $(a - \mu)/\sigma^2$. DDPG, TD3 (double critics, delayed actor, smoothing), SAC (entropy).

FORMULAS

Softmax policy Exam favourite

$$\pi(a|s, \theta) = e^{h(s, a, \theta)} / \Sigma_b e^{h(s, b, \theta)}$$

h are action preferences.

Lesson: Learning the policy directly: REINFORCE

REINFORCE update Exam favourite

$$\theta \leftarrow \theta + \alpha G_t \nabla \ln \pi(A_t|S_t, \theta)$$

Monte Carlo policy gradient, Williams (1992).

Lesson: Learning the policy directly: REINFORCE

Softmax log gradient Exam favourite

$$\nabla_h \ln \pi(a) = \text{one hot}(a) - \pi$$

For two actions: a − p.

Lesson: Learning the policy directly: REINFORCE

Policy gradient theorem Exam favourite

$$\nabla J(\theta) \propto \Sigma_s \mu(s) \Sigma_a q_\pi(s, a) \nabla \pi(a|s, \theta)$$

No derivative of $\mu(s)$ needed.

Lesson: The policy gradient theorem and baselines

Score function form Exam favourite

$$\nabla J(\theta) = E_\pi[G_t \nabla \ln \pi(A_t|S_t, \theta)]$$

Uses the log derivative trick $\nabla \pi = \pi \nabla \ln \pi$.

Lesson: The policy gradient theorem and baselines

With baseline Exam favourite

$$\theta \mathrel{+}= \alpha(G_t - b(S_t))\nabla \ln \pi(A_t|S_t, \theta)$$

Unbiased for any b that ignores the action.

Lesson: The policy gradient theorem and baselines

One step actor critic Exam favourite

$$\theta \mathrel{+}= \alpha_\theta \delta \nabla \ln \pi(A|S, \theta), \quad \delta = R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$$

The TD error estimates the advantage.

Lesson: Actor critic methods

GAE Exam favourite

$$\hat{A}_t = \Sigma_{l \geq 0} (\gamma\lambda)^l \delta_{t+l}$$

λ = 0 gives the one step TD error, λ = 1 the Monte Carlo advantage.

Lesson: Actor critic methods

PPO clipped objective Exam favourite

$$L(\theta) = E[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t)]$$

$r_t = \pi_\theta(a_t|s_t) / \pi_{old}(a_t|s_t)$.

Lesson: PPO: small, safe policy updates

TRPO Exam favourite

$$\max E[r_t\hat{A}_t] \text{ subject to } E[KL(\pi_{old} \,||\, \pi_\theta)] \le \delta$$

A trust region on the policy.

Lesson: PPO: small, safe policy updates

Full PPO loss

$$L = L^{CLIP} - c_1(V_\theta(s) - V^{target})^2 + c_2\, H(\pi_\theta)$$

Value loss and entropy bonus.

Lesson: PPO: small, safe policy updates

Gaussian policy gradient Exam favourite

$$\nabla_\mu \ln \pi(a|s) = (a - \mu(s)) / \sigma(s)^2$$

Push the mean towards good actions.

Lesson: Continuous actions: Gaussian policies, DDPG, TD3 and SAC

Deterministic policy gradient

$$\nabla J = E[\nabla_a Q(s, a)|_{a = \mu(s)} \nabla_\theta \mu(s)]$$

Silver and colleagues (2014).

Lesson: Continuous actions: Gaussian policies, DDPG, TD3 and SAC

Maximum entropy objective

$$J(\pi) = E[\Sigma_t\, r_t + \alpha\, H(\pi(\cdot|s_t))]$$

SAC.

Lesson: Continuous actions: Gaussian policies, DDPG, TD3 and SAC

Objective Exam favourite

$$J(\theta) = v_{\pi_\theta}(s_0) = E_{\pi_\theta}[G_0]$$

Expected return from the start.

Lesson: The maths, all in one place

Log derivative trick Exam favourite

$$\nabla_\theta\, \pi(a|s, \theta) = \pi(a|s, \theta)\, \nabla_\theta \ln \pi(a|s, \theta)$$

Turns a gradient of a sum into an expectation.

Lesson: The maths, all in one place

REINFORCE with baseline Exam favourite

$$\theta \mathrel{+}= \alpha(G_t - b(S_t))\nabla \ln \pi(A_t|S_t)$$

Unbiased, lower variance.

Lesson: The maths, all in one place

Advantage Exam favourite

$$A(s, a) = q_\pi(s, a) - v_\pi(s)$$

Estimated by the TD error or by GAE.

Lesson: The maths, all in one place

GAE

$$\hat{A}_t = \delta_t + \gamma\lambda\hat{A}_{t+1}$$

Computed backwards through a batch.

Lesson: The maths, all in one place

PPO clip

$$\min(rA, \text{clip}(r, 1-\varepsilon, 1+\varepsilon)A)$$

No gradient where the minimum is flat.

Lesson: The maths, all in one place

TABLES

Table: method · year · key idea

Method	Year	Key idea
A3C	2016	Many actors run in parallel threads and update a shared network asynchronously
A2C	2016 onwards	The synchronous version of A3C: wait for all actors, then one batched update
GAE	2016	Advantage = sum of $(\gamma\lambda)^l\, \delta_{t+l}$, λ trades bias for variance
TRPO	2015	Limit each update by a KL divergence constraint, a trust region
PPO	2017	Keep updates small with a clipped probability ratio, simple and robust

Lesson: Actor critic methods

Table: method · policy · on or off policy · key trick

Method	Policy	On or off policy	Key trick
REINFORCE	Stochastic	On	Monte Carlo returns
PPO	Stochastic	On	Clipped ratio, several epochs per batch
DDPG	Deterministic plus noise	Off	Deterministic policy gradient through Q
TD3	Deterministic plus noise	Off	Clipped double Q, delayed actor, target smoothing
SAC	Stochastic	Off	Maximum entropy objective

Lesson: Continuous actions: Gaussian policies, DDPG, TD3 and SAC

Table: · value based (dqn) · policy based (reinforce) · actor critic (ppo, sac)

	Value based (DQN)	Policy based (REINFORCE)	Actor critic (PPO, SAC)
Learns	Q(s, a)	π(a|s)	Both π and V or Q
Actions	Discrete, needs a max	Discrete or continuous	Discrete or continuous
Stochastic policies	No, greedy plus ε	Yes	Yes
Variance	Low	High	Medium
Sample efficiency	Good, uses replay	Poor, on policy	PPO medium, SAC good

Lesson: Value based, policy based and actor critic compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
2016	Maximum Difference Between Increasing Elements	Easy	Track the smallest value seen so far	Walk left to right, keep the minimum so far, and record the largest value minus that minimum when it is positive. Comparing a value with a reference level is what an advantage does: it compares a return with the baseline.
55	Jump Game	Medium	Keep the furthest index you can reach	Scan once, updating the furthest reachable index. If you ever stand beyond it, you are stuck. Like the short corridor, whether you can reach the goal depends on which moves are allowed in each cell.
45	Jump Game II	Medium	Greedy breadth first search by layers	Treat each jump as a layer: from the current range find the furthest next reach, and count a jump each time you pass the end of the range. The answer is the shortest number of moves, the same as minus the best return when every move costs 1.
1696	Jump Game VI	Medium	Dynamic programming with a sliding window maximum	The best score at i is nums[i] plus the best score among the previous k cells. A deque of indices with decreasing scores gives that maximum in constant time. Adding the reward now to the best value reachable next is a Bellman backup.
1345	Jump Game IV	Hard	Breadth first search, and clear each value's list after first use	From index i you may step to i − 1, i + 1 or any index with the same value. Breadth first search finds the fewest steps. Emptying a value's index list once used stops the search from scanning it again. Shortcuts that connect distant states make credit travel fast, which is why the same idea appears in options and skills in hierarchical RL.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

Solving the Bellman equations for every p shows the best policy is p(right) = 0.59, worth −11.66 per episode, while p = 0.5 is worth −12. REINFORCE starts at 0.5 and, averaged over its last 500 episodes, sits at p = 0.593. Single episodes are noisy, so p wanders around the best value.

Lesson: Learning the policy directly: REINFORCE

IN THE EXAM For the fixed policy p = 0.3 the true gradient is 8.48. Over 20,000 episodes the plain REINFORCE estimate averages 8.53 with a standard deviation of 39.65, and with the baseline b(s) = v(s) it averages 8.59 with a standard deviation of 25.33. Same direction on average, about 36% less noise per episode.

Lesson: The policy gradient theorem and baselines

IN THE EXAM On the cliff walk from chapter 2.3, a tabular actor critic improves from an average reward of −47.1 in its first 100 episodes to about −19 in later blocks. Its most likely path takes 19 moves and visits every row, keeping well away from the cliff edge, much like SARSA. Like SARSA it learns about the policy it actually follows, and as its policy sharpens it explores less, so it earns more than SARSA's −24.42 while learning.

Lesson: Actor critic methods

IN THE EXAM With the clip, average episode length grows from 23.4 to 172.8 in 15 iterations of about 1000 steps, reusing each batch 10 times. The ratios still leave [0.8, 1.2] a little, because the clip only silences samples already outside the range while the others keep pushing. Without the clip this easy, linear problem also learns, but the ratios swing from 0.11 to 1.92 in the first iteration: the new policy moves far from the data, which with neural networks often ends in collapse.

Lesson: PPO: small, safe policy updates

3.3 RL in practice: models, offline data and RLHF

Model based planning, MCTS, offline RL, imitation, RLHF, DPO and safe deployment.

KEY IDEAS

- **Model based RL** Learn Model(s, a) = (s', r), plan with simulated steps. Dyna-Q: 1 real update plus n planning updates.
- **MCTS and AlphaZero** Select, expand, evaluate, back up. PUCT with a policy prior. Self play. MuZero learns the model.
- **Offline RL** Fixed log, no exploring. Unseen actions overestimated by the max. CQL pushes them down. BC, DAgger.
- **RLHF** SFT, then Bradley–Terry reward model $-\ln \sigma(r_w - r_l)$, then PPO on $r - \beta KL$. $\pi^* \propto \pi_{ref} e^{r/\beta}$.
- **DPO** $r = \beta \ln(\pi/\pi_{ref}) + \text{const}$. Loss $-\ln \sigma(\beta[\text{log ratio winner} - \text{log ratio loser}])$.
- **Safety** Reward hacking, side effects, safe exploration, distribution shift, sim to real, many seeds.

FORMULAS

Dyna-Q planning update Exam favourite

$$Q(S, A) \mathrel{+}= \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)], \text{ with } (S', R) = \text{Model}(S, A)$$

The same update as Q learning, on simulated data.

Lesson: Model based RL: learning a model and planning with it

Updates per real step

$$1 + n$$

One direct update and n planning updates.

Lesson: Model based RL: learning a model and planning with it

UCT selection Exam favourite

$$a = \text{argmax}\,[Q(s, a) + c\sqrt{(\ln N(s) / N(s, a))}]$$

Upper confidence for trees, like UCB for bandits.

Lesson: Planning at decision time: MCTS, AlphaZero and MuZero

AlphaZero selection (PUCT) Exam favourite

$$a = \text{argmax}\,[Q(s, a) + c\,P(s, a)\sqrt{N(s)} / (1 + N(s, a))]$$

P is the policy network's prior.

Lesson: Planning at decision time: MCTS, AlphaZero and MuZero

Behaviour cloning Exam favourite

$$\max_\theta \Sigma_{(s,\,a)\text{ in }D} \ln \pi_\theta(a|s)$$

Supervised learning on expert pairs.

Lesson: Offline RL and imitation learning

Compounding error Exam favourite

$$\text{cost of cloning} \le \text{cost of expert} + O(\varepsilon T^2),\ \text{DAgger: } O(\varepsilon T)$$

ε per step mistake rate, T the horizon.

Lesson: Offline RL and imitation learning

CQL penalty

$$\min_Q \alpha(E_{s\sim D}[\ln \Sigma_a e^{Q(s,\,a)}] - E_{(s,\,a)\sim D}[Q(s,\,a)]) + \text{TD error}$$

Lowers Q on actions outside the data.

Lesson: Offline RL and imitation learning

Bradley–Terry model Exam favourite

$$P(y_w \succ y_l) = \sigma(r(y_w) - r(y_l))$$

Preference probability from a score difference.

Lesson: RLHF: learning from human preferences

Reward model loss Exam favourite

$$L(r) = -E[\ln \sigma(r(x,\,y_w) - r(x,\,y_l))]$$

Logistic regression on pairs.

Lesson: RLHF: learning from human preferences

KL regularised objective Exam favourite

$$\max_\pi E[r(x,\,y)] - \beta\, KL(\pi(\cdot|x) \,\|\, \pi_{ref}(\cdot|x))$$

Solution: $\pi(y|x) \propto \pi_{ref}(y|x)\, e^{r(x,\,y)/\beta}$.

Lesson: RLHF: learning from human preferences

Implicit reward Exam favourite

$$r(x,\,y) = \beta \ln(\pi(y|x) / \pi_{ref}(y|x)) + \beta \ln Z(x)$$

The constant cancels in a comparison.

Lesson: DPO and learning from AI feedback

DPO loss Exam favourite

$$L = -E[\ln \sigma(\beta \ln(\pi(y_w)/\pi_{ref}(y_w)) - \beta \ln(\pi(y_l)/\pi_{ref}(y_l)))]$$

Rafailov and colleagues (2023).

Lesson: DPO and learning from AI feedback

Constrained MDP Exam favourite

$$\max_\pi E[\Sigma\, \gamma^t r_t] \text{ subject to } E[\Sigma\, \gamma^t c_t] \le d$$

Keep expected cost below a limit.

Lesson: Making RL work safely in the real world

Dyna planning Exam favourite

$$Q(s,\,a) \mathrel{+}= \alpha[\hat{r} + \gamma \max Q(\hat{s}',\,\cdot) - Q(s,\,a)],\ (\hat{s}',\,\hat{r}) = \text{Model}(s,\,a)$$

Simulated experience.

Lesson: The maths, all in one place

UCT Exam favourite

$$Q(s,\,a) + c\,\sqrt{(\ln N(s) / N(s,\,a))}$$

Tree search selection.

Lesson: The maths, all in one place

Bradley–Terry Exam favourite

$$P(y_w \succ y_l) = \sigma(r_w - r_l)$$

Preferences from scores.

Lesson: The maths, all in one place

KL regularised optimum Exam favourite

$$\pi^*(y) = \pi_{ref}(y)\, e^{r(y)/\beta} / Z$$

Z normalises.

Lesson: The maths, all in one place

DPO loss

$$-\ln \sigma(\beta[\ln \pi(y_w)/\pi_{ref}(y_w) - \ln \pi(y_l)/\pi_{ref}(y_l)])$$

Preference tuning without RL.

Lesson: The maths, all in one place

Cloning error bound

$$J(\pi_{BC}) \leq J(\pi_{expert}) + \varepsilon T^2$$

For costs in [0, 1], Ross and Bagnell (2010).

Lesson: The maths, all in one place

TABLES

Table: system · year · model of the world · main result

System	Year	Model of the world	Main result
TD-Gammon	1992	Rules of backgammon, TD learning	Near the best human players
AlphaGo	2016	Rules of Go, human games plus self play	Beat Lee Sedol 4 to 1
AlphaGo Zero	2017	Rules of Go, self play only	Beat AlphaGo Lee 100 to 0
AlphaZero	2018	Rules, one algorithm for three games	Superhuman at Go, chess and shogi
MuZero	2020	Learned model, no rules given	Matched AlphaZero and set new Atari records

Lesson: Planning at decision time: MCTS, AlphaZero and MuZero

Table: problem · example · common defence

Problem	Example	Common defence
Reward hacking	Circling for points instead of finishing a race	Better rewards, KL constraints, human review
Negative side effects	Knocking over a vase on the way to the goal	Penalise changes to the environment
Unsafe exploration	A robot trying a random torque near a person	Simulation, safe action sets, constrained RL
Distribution shift	A policy trained in summer used in winter	Monitoring, conservative offline RL, retraining
Sim to real gap	A simulated grasp that fails on a real arm	Domain randomisation, system identification
Noisy evaluation	An improvement that vanishes with new seeds	Many seeds, confidence intervals, fixed protocols

Lesson: Making RL work safely in the real world

Table: · model free online · model based · offline

	Model free online	Model based	Offline	From preferences
Data	Fresh interaction	Interaction plus simulated steps	A fixed log only	Human or AI comparisons
Reward	Given	Given or learned	In the log	Learned from comparisons
Strength	Simple, general	Few real steps needed	No risky exploration	Rewards that are hard to write
Main risk	Needs many samples	Model errors mislead planning	Overestimating unseen actions	Reward hacking
Examples	DQN, PPO, SAC	Dyna, MuZero, Dreamer	BC, CQL, IQL	RLHF with PPO, DPO

Lesson: Model free, model based, offline and preference based RL compared

Table: no. · problem · level · pattern

No.	Problem	Level	Pattern	Hint
997	Find the Town Judge	Easy	Count trust given and trust received	The judge is trusted by everyone else and trusts nobody, so their score of received minus given is n − 1. Turning many pairwise statements into one score per person is what a reward model does with preference pairs.
207	Course Schedule	Medium	Topological sort: repeatedly take courses with no remaining prerequisites	Count each course's prerequisites and start with those at zero. Taking a course frees the courses that depend on it. If some course is never freed, there is a cycle. Ordering steps so that each comes after what it needs is the heart of planning.
2115	Find All Possible Recipes from Given Supplies	Medium	Topological sort where recipes become supplies	Treat each recipe as waiting for its ingredients. When an ingredient becomes available, reduce the count of every recipe that needs it, and a recipe with nothing missing becomes a new supply. Building complex actions from simpler ones is how hierarchical planners work.
1557	Minimum Number of Vertices to Reach All Nodes	Medium	Every node with no incoming edge must be a start	In a directed acyclic graph, a node nobody points to can only be reached by starting there, and those nodes reach everything else. Knowing which states can be reached from which is the first question a planner asks of its model.
773	Sliding Puzzle	Hard	Breadth first search over board states	Write the 2 by 3 board as a string, find the blank, and swap it with each neighbour to get the next states. Breadth first search from the start finds the fewest moves. With an exact model of the puzzle, planning by search needs no learning at all.

Lesson: LeetCode practice

TIPS AND TRAPS

IN THE EXAM

Averaged over 5 runs, plain Q learning (n = 0) still needs 271.4 steps in episode 5, Dyna-Q with n = 5 needs 35.6 and with n = 50 only 19.4, against a shortest path of 14. Episode 1 is a random walk for every n, because no reward has been seen yet, so its length differs only by chance.

Lesson: Model based RL: learning a model and planning with it

IN THE EXAM

The log holds 2742 transitions from a cautious policy that takes a random move 30% of the time, covering 79 of the 96 state and action pairs. Behaviour cloning returns −9. Naive offline Q learning returns −51: one step after the start it picks "right", an action the log never tried, whose value stayed at its starting 0, which looks better than every tried action, and it falls straight into the pit. Pushing unseen actions down gives the best possible −7.

Lesson: Offline RL and imitation learning

IN THE EXAM

The reward model learns what the labellers prefer, including their mistake: the made up answer seems best to them and gets the highest learned reward, 2.82, though its true quality is −1. With $\beta = 2$ the tuned policy still improves true quality from −0.15 to 0.13. With $\beta = 0.1$ it puts all its probability on the made up answer and true quality falls to −1. The KL penalty limits how hard the policy can exploit reward model errors.

Lesson: RLHF: learning from human preferences

IN THE EXAM

On the same toy problem with honest labellers and $\beta = 1$, DPO trained directly on the 400 pairs and the RLHF route (fit a Bradley–Terry reward, then take $\pi \propto \pi_{ref} e^{r}$) give the same policy, with the largest difference in any probability below 0.0001. Expected quality rises from −0.15 to 0.97.

Lesson: DPO and learning from AI feedback

IN THE EXAM

Henderson and colleagues (2018) showed that the same deep RL algorithm, with the same settings, can give significantly different results with different random seeds, and that implementation details can matter as much as the algorithm. Report results over several seeds.

Lesson: Making RL work safely in the real world